

Soul of an Old Machine: Reverse-engineering the TMS 1000

A Thesis
Presented to
The Division of Mathematical and Natural Sciences
Reed College

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Arts

Kavi Stanley Wilson

May 2025

Approved for the Division
(Computer Science)

Erica Blum

Acknowledgements

I want to thank all the friends I made along the way: Connor Gilligan, Jaisy Gray, Josh Ogawa, Max Bresnan, Nadav Schul-Kutas, Matthew “Chewie” Honig, Matthew “Redford” Redford, Percy Okoben, Tiago Beck, Tucker R. Twomey, and many more (this is a very non-exhaustive list). I would be a different person if I didn’t know all of you.

I want to thank my advisor, Erica Blum, for being incredibly supportive over the course of this thesis, and the entire Reed College computer science community, especially the wonderful people on the reed cs & friends discord server. I have learned so much from you.

I want to thank my parents, Brent and Swati Wilson, for always supporting me, and enabling me to attend Reed.

Finally, I want to thank the vintage computing community at large, and especially Sean Riddle and Ken Shirriff, for bringing to light and freely sharing online a great deal of interesting and useful information about early Texas Instruments microcontrollers.

List of Abbreviations

ALU	Arithmetic and Logic Unit
BCD	Binary Coded Decimal
MOSFET	Metal Oxide Silicon Field Effect Transistor
NMOS	N-channel MOSFET
PLA	Programmable Logic Array
PMOS	P-channel MOSFET
RAM	Random Access Memory
ROM	Read Only Memory

Table of Contents

Introduction	1
0.1 Roadmap	1
0.2 Motivation	2
0.2.1 Why the TMS 1000?	3
0.3 History	4
0.3.1 A Note on Nomenclature	7
Chapter 1: Electricity and Semiconductors	9
1.1 Introduction	9
1.2 A Crash Course in Electronics	9
1.2.1 Electromagnetism and Charge Carriers	9
1.2.2 Voltage	11
1.2.3 Current	12
1.2.4 Conductance and Resistance	12
1.2.5 Circuits	14
1.2.6 Electrical Components	15
1.3 Semiconductor Doping	18
1.3.1 Introduction	18
1.3.2 How it works	19
1.3.3 P-N Junctions	20
1.3.4 Bias	21
1.4 Transistors	22
1.4.1 Introduction	22
1.4.2 P- and N-types	22
1.5 Introducing: the MOSFET	23
1.5.1 Field Effect Transistors	23
1.5.2 Metal Oxide Silicon FETs	23
1.5.3 MOSFET body regions	25
1.5.4 MOSFET Notation and Convention	26
1.5.5 MOSFET modes of operation	27
1.5.6 MOSFET current equations	30
1.5.7 MOSFET capacitors	30
1.5.8 MOSFET resistors	30
1.6 Integrated Circuits	31
1.6.1 Layers and Traces	32

1.6.2	Notation and terminology	32
Chapter 2:	Digital Logic Design	33
2.1	Introduction	33
2.2	Boolean Logic	33
2.3	Logic Gates	34
2.3.1	Logic conventions	35
2.3.2	Specific values	36
2.3.3	Notation	36
2.4	Clock Signals	37
2.5	A simple NOT gate	37
2.5.1	Naive Implementation	38
2.5.2	Second Implementation	38
2.5.3	Third Implementation	39
2.6	NOT gate variants	41
2.6.1	Buffer	42
2.6.2	Clocked NOT Gate 1	42
2.6.3	Clocked NOT Gate 2	43
2.7	Other Logic Gates	44
2.7.1	NOR gate	44
2.7.2	NAND gate	44
2.7.3	Clocking and NAND/NOR gates	45
2.7.4	Combinations	46
2.7.5	Loads	46
2.8	Clocks	46
2.9	Programmable Logic Arrays	47
2.9.1	Logical Properties	47
2.9.2	Circuit	48
2.9.3	Read-Only Memory	49
2.10	Storing Values	50
2.10.1	Precharging	50
2.10.2	Shift Registers	50
2.10.3	Recirculating Shift Registers	51
2.11	Representing Numbers	52
2.11.1	Hexadecimal	53
Chapter 3:	TMS 1000 System Architecture	55
3.1	Introduction	55
3.2	What is a computer?	55
3.2.1	Single-Purpose Computers	55
3.2.2	General-Purpose Computers	55
3.2.3	A Minimal Computer	56
3.2.4	Terminology	56
3.3	The TMS 1000	57
3.3.1	Program Storage	58

3.3.2	Working Memory	58
3.3.3	I/O	59
3.3.4	Arithmetic and Logic	61
3.3.5	Programming	62
3.3.6	A better functional diagram	68
3.3.7	Try it for yourself	68
Chapter 4:	TMS 1000 Components	71
4.1	Introduction	71
4.1.1	Notation	71
4.1.2	Ordering	72
4.2	Clock and Timing Signals	72
4.2.1	Purpose	72
4.2.2	Inputs	73
4.2.3	Outputs	73
4.2.4	Function	73
4.3	Power Up and Clear	74
4.3.1	Purpose	74
4.3.2	Inputs	74
4.3.3	Outputs	75
4.3.4	Control Lines	75
4.3.5	Function	75
4.4	Instruction Decoder	76
4.4.1	Purpose	76
4.4.2	Inputs	78
4.4.3	Outputs	78
4.4.4	Function	78
4.5	Adder	79
4.5.1	Purpose	79
4.5.2	Inputs	81
4.5.3	Outputs	81
4.5.4	Function	82
4.6	ROM array	85
4.6.1	Purpose	86
4.6.2	Inputs	86
4.6.3	Outputs	86
4.6.4	Function	86
4.7	RAM array and controls	91
4.7.1	Purpose	91
4.7.2	Inputs	91
4.7.3	Outputs	92
4.7.4	Memory Cell	92
4.7.5	RAM controls	94
4.8	S-Outputs	96
4.8.1	Purpose	96

4.8.2	Inputs	96
4.8.3	Outputs	97
4.8.4	Function	97
4.9	D-outputs	100
4.9.1	Purpose	100
4.9.2	Inputs	100
4.9.3	Outputs	100
4.9.4	Function	100
4.10	X Register	101
4.10.1	Purpose	101
4.10.2	Inputs	102
4.10.3	Outputs	102
4.10.4	Function	102
4.11	CKB logic	104
4.11.1	Purpose	104
4.11.2	Inputs	104
4.11.3	Outputs	104
4.11.4	Function	105
4.12	Status Logic	105
4.12.1	Purpose	105
4.12.2	Inputs	105
4.12.3	Outputs	107
4.12.4	Function	107
4.13	Address Controls	108
4.13.1	Purpose	108
4.13.2	Inputs	108
4.13.3	Outputs	109
4.13.4	Function	109
4.14	Page Address and Page Buffer Registers	111
4.14.1	Purpose	111
4.14.2	Inputs	111
4.14.3	Outputs	113
4.14.4	Function	113
4.15	Program Counter and Subroutine Return	113
4.15.1	Purpose	113
4.15.2	Inputs	115
4.15.3	Outputs	115
4.15.4	Function	116
4.16	RAM/ROM Word Decoder and Data Select	119
4.16.1	Purpose	119
4.16.2	Inputs	119
4.16.3	Outputs	119
4.16.4	Function	119

Conclusion	123
-----------------------------	------------

4.1	What's left to do?	123
4.2	What I learned	123
Appendix A: Other Transistors		125
A.1	Bipolar Junction Transistors	125
A.2	Junction FETs	126
References		127

List of Figures

1	TMS 1000 package	2
2	TMS 1000 toys	6
1.1	Standard circuit diagram symbols for a battery, an earth ground, and a signal ground.	16
1.2	Resistors	16
1.3	Diodes	17
1.4	Capacitors	17
1.5	TMS 1000 package	18
1.6	Silicon Doping	19
1.7	P-N junctions	20
1.8	Transistors	21
1.9	MOSFET inversion channel	24
1.10	MOSFET symbols	26
1.11	MOSFET in cutoff mode	27
1.12	MOSFET in linear mode	28
1.13	MOSFET in saturation mode	28
1.14	MOSFET current vs voltage	29
1.15	Integrated circuit layers	31
2.1	Circuit diagram symbols for logic gates	35
2.2	NOT gate symbol	37
2.3	NOT gate circuit	39
2.4	Depletion-load NOT gate voltage characteristics	40
2.5	Buffer circuit diagram	42
2.6	Clocked NOT gate 1	42
2.7	Clocked NOT gate 2	43
2.8	NOR gate	44
2.9	NAND gate	45
2.10	Clocked NAND gate	45
2.11	Complex Gate	46
2.12	Depletion load	46
2.13	Ring Oscillator	47
2.14	PLA logic	47
2.15	PLA circuit	48

2.16	The metal and silicon layers of a TMS 1000 microcontroller's instruction decode PLA	49
2.17	Basic shift register	50
2.18	Recirculating shift register	51
3.1	TMS 1000 simplified block diagram	57
3.2	TMS 1000 package	59
3.3	TMS 1000 pinout	60
3.4	Seven-segment driver O-output PLA	60
3.5	ALU block diagram	62
3.6	TMS 1000 standard instruction set	63
3.7	Hardware configuration and timing for display routine, for a calculator configuration [29, 13-3]	64
3.8	TMS 1000 display-driving code	65
3.9	TMS 1000 microinstructions	67
3.10	Block diagram	69
4.1	Clock circuit and resultant timing signals	72
4.2	Oscillator pin connection	73
4.3	Power up and clear circuit	74
4.4	Instruction Decoder	77
4.5	Adder	80
4.6	Accumulator and Y-register	84
4.7	ROM array	85
4.8	Basic ROM array	87
4.9	Page address demonstration	88
4.10	ROM array silicon	89
4.11	RAM array and controls	90
4.12	Single RAM cell	92
4.13	RAM array silicon	95
4.14	S-outputs	96
4.15	Output PLA in use as a seven-segment decoder	97
4.16	D-outputs	99
4.17	X-register	101
4.18	The CKB multiplexer	103
4.19	A close-up of the rightmost gate, which outputs <i>CKB8</i>	103
4.20	Status logic	106
4.21	Address Controls	108
4.22	Page address and buffer	112
4.23	Program counter	114
4.24	Program counter feedback circuit	117
4.25	RAM/ROM word decoder	120

Abstract

Computer science is a field susceptible to mystification. It is easy to abstract away and ignore what happens ‘under the hood’ in a computer, and treat everything as ‘black boxes’. In this thesis, we will attempt to reverse this trend by examining how computers are implemented at the lowest level. We will use a simple microcontroller from the 1970s, the TMS 1000, as a case study, and attempt to develop a ‘full stack’ understanding of how it works, starting with the properties of electricity and ending with its system architecture. We will examine how transistors are created, how logic gates are made from transistors, and how complex system components are constructed out of logic gates and transistors. Additionally, we will attempt to reverse engineer the rationale for various design decisions, and explore the idiosyncrasies of a somewhat unusual, historical microcontroller.

Introduction

The first computers were very large and expensive, affordable only by the largest of organizations. As technology progressed, the ‘mainframe’ computers being produced became more powerful and capable. As a powerful computer is not required for many useful tasks, a secondary market opened up: smaller and cheaper computers could be manufactured, not nearly as powerful as the contemporary generation of mainframe computers, but affordable to smaller businesses and organizations. As these were, in a sense, miniature computers, they were designated ‘minicomputers’.

Concurrently and closely related to the development of the computer were advances in products constructed out of semiconductors, such as silicon. An integrated circuit, contemporaneously known as a microcircuit, is an entire circuit created out of a small piece of silicon, known as a ‘chip’.

By the start of the 1970s, semiconductor technology had advanced such that it became possible to include most of the control and data processing logic of a computer on a single chip. Such a circuit became known as a microprocessor, as it was a processor on a microchip. Computers which used such a chip in a primary role became known as microcomputers, and were even smaller, cheaper, and less powerful than the minicomputers, which had at this point become quite powerful and capable [16, Chapter 1].

At the same time as the development of the microcomputer came that of the single-chip microcomputer: if we can fit most of a computer on a single silicon chip, why not go all the way?¹

These ‘single-chip microcomputers’ could be manufactured in bulk, meaning that for the first time, computers could be manufactured cheaply. Single-chip microcomputers eventually became known as ‘microcontrollers’, as they were and are frequently used as control circuitry in other electronic devices (we will use the two terms interchangeably). The vast majority of computers produced are microcontrollers, and most modern electronic devices rely on microcontrollers to function.

0.1 Roadmap

Over the course of this thesis we will reverse-engineer, and explain the function of, an early microcontroller, the TMS 1000, which was brought to market in 1975 and

¹Gary Boone worked on both the TMX 1795, either the first or the second single-chip microprocessor, depending on who we ask, at the same time that he worked on the TMS 0100 [26].

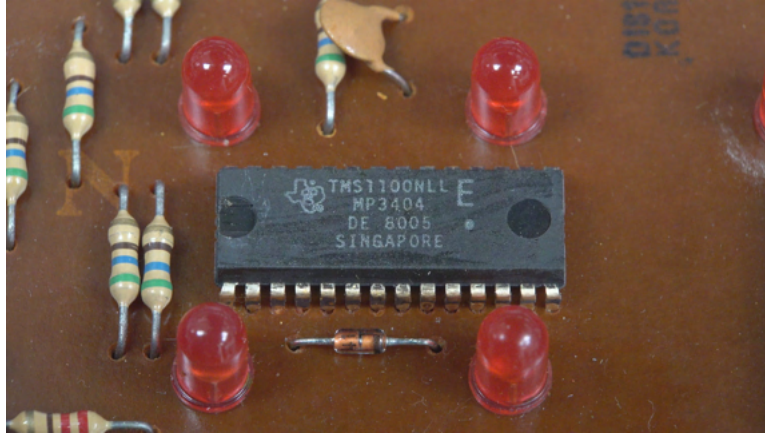


Figure 1: A TMS 1000-family microcontroller [2]

saw widespread use in the late 70s and early 80s.

We will start from scratch. In Chapter 1, we will begin with the first principles of electricity, analyze the properties of semiconductors, and deconstruct how transistors work and how integrated circuits are created. In Chapter 2, we will create the building blocks of digital logic circuits from transistors. In Chapter 3, we will look at the system architecture of the TMS 1000 microcontroller from a high level. In Chapter 4, we will analyze each component in detail, to the effect that, by the end, there should be no mysteries residing in the silicon.

Additionally, included with this thesis is a cycle-stepped, register-level emulator of the TMS 1000, a basic debugging interface, and a basic compiler and decompiler, allowing one to run and closely examine code written for the TMS 1000. It is available at github.com/atomicmoloch/TMS1000.

0.2 Motivation

I have always held the ideal of possessing total knowledge, when dealing with complex systems. That is to say, I believe that there are positive emergent properties of understanding a system thoroughly top to bottom.

We have always lived in a world of complex natural systems. A more recent development is the emergence of pervasive, incredibly complex man-made systems. The typical medieval peasant did not accurately understand much of the natural world, but they would have thoroughly understood the man-made aspects of their environment: where their food came from, how their buildings were constructed, how their tools and possessions were made, and how they function. Few people in the modern day, if anybody, can say the same. Our world is full of ‘black boxes’; we are all a bit alienated from the world.

The desire to understand what makes things tick was a large part of why I wanted to study computer science. But the problem resides in the microcosm as well: the complexity of modern computer systems necessitates specialization and layers and layers of abstraction. Computer science curricula starts on a high level of abstraction

(for instance, at the present time, the introductory computer science classes at Reed teach Python as the first programming language). Sometimes the curricula never end up getting close to the hardware.

I used this thesis as an excuse and an opportunity to study low-level computer architecture. I can finally say that I fully understand how a computer is constructed and how it functions, even if it is a very simple computer. I learned quite a bit: before this year, I had never written an emulator, and didn't even know how transistors worked. I also taught myself Rust along the way.

Additionally, I attempted to write this document as an introduction to how computers work from the 'other direction', starting at the low level and working up. I have tried to make it a real, useful introduction, presupposing little to no knowledge of the field. I have done my best to minimize abstractions, metaphors, and unacknowledged simplifications of concepts in this work, except for the mild and unobjectionable conceit of ignoring most technological developments since the mid 1970s.

0.2.1 Why the TMS 1000?

The TMS 1000 possesses a number of characteristics which caused me to choose it as the subject of my thesis.

It's simple. It comes from the era when microprocessors and computers were designed by individuals, instead of massive teams. Everything is on a human scale, and thus explicable.

It's complete and self-contained. Being a microcontroller, all the components of a complete computer are present. In many applications, the TMS 1000 does everything, with no support chips.

It's well-documented. Texas Instruments obtained a significant amount of revenue from their patented inventions (a business model used by many highly-diversified, research-intensive electronics companies of the 20th century, such as RCA) [26]. Because of this, patents filed by Texas Instruments are unusually detailed and informative. The TMS 1000 patent allowed Texas Instruments to patent the concept of a microcontroller in general.

It was widely used. The TMS 1000 was used in everything from telephone switching to handheld gaming. At one point it was almost certainly the most common microprocessor in the world. For many people, the first digital computer they owned was a toy or appliance with a TMS 1000 in it.

It's unique. Being the first of its kind, and having no direct successors or derivatives, means that its architecture is idiosyncratic. It's not like the other microprocessors.

It's historically notable. The TMS 1000 is one of the first microcontrollers, and is a direct descendant of the very first ever microcontroller.

Finally, it's ignored. Compared to many of its contemporaries, which have often been extensively reverse-engineered, simulated, emulated, reconstructed in FPGAs, and had new software developed for, the TMS 1000 is overlooked. This seemed to me like a region where I could create real added value, instead of treading paths already well-explored. I am hoping that the emulator and utility programs which I wrote will

be a useful resource to others interested in this microcontroller.

0.3 History

Texas Instruments was a large, highly-diversified electronics company in the 1960s, with many substantially independent departments, which almost ran their own businesses [27]. The MOS department of Texas Instruments designed calculators for clients. Using the then-new technology of integrated circuits, TI was able to design calculators smaller and more inexpensive than hitherto was possible [3].

This business was never more lucrative than at the start of the seventies. Texas Instruments had designed and created the first handheld electronic calculator in 1967, which was brought to market by Canon in 1970, initially retailing for slightly under \$400.² A ‘calculator war’ quickly ensued, as other manufacturers tried to undercut each other in selling cheaper and more full-functioned handheld calculators. Naturally, the Texas Instruments MOS department was flooded with orders for calculator designs [23].

The business worked as follows: a manufacturer would present a list of specifications for the calculator design they wanted (for instance, what mathematical functions the calculator should be able to do, the precision of the calculations, and how calculation results should be displayed). Then, Texas Instruments would assign an engineer to the project, who would design a bespoke circuit, tailored to the specification. The resulting designs would all, of course, use chips made by Texas Instruments, generating additional revenue for the company.

The individual specifications of these calculators differed, and required months of designing, but the purposes and functions differed little. Most calculators implement the same few operations. The boredom and inefficiency of constantly coming up with redundant calculator designs spurred a group of TI engineers, headed by Gary Boone and Daniel Baudoin, to attempt to create a standardized but easily-reconfigurable calculator architecture. Furthermore, taking component minimization to an extreme, the new architecture would be implemented on a single silicon chip.

But how can this be done? In order to allow a calculator architecture to easily be reconfigured for the specifications given by different calculator companies, it essentially had to be programmed. With little fanfare, because of the boredom of ‘reinventing the wheel’ and as a functional solution to an operational problem, TI engineers created the smallest computer in the world, and the first to fit on one silicon chip.

The result was announced in September of 1971 as the TMS 0100 (TMS being short for TI MOS, and the number being an arbitrary product identifier. The leading zero was added later, to distinguish the TMS 0100 and TMS 1000). By the end of the year, it had paid back its development costs eighteen times over [3]. With all of the logic of a complex calculator now implementable on a single chip, available from TI for a mere \$20, handheld calculator prices rapidly dropped, becoming the staple of life they are today.

²Due to retailer markups, the price slightly varied between establishments.

From the start, the fact that the TMS 0100 was a full computer was not lost on its designers. In one the original patent applications, non-calculator use was speculated on [3]. However, the TMS 0100's architecture was limited and specialized for calculator applications, and the Texas Instruments MOSFET department was still locked in to their business model of designing calculators in-house, which TI had begun manufacturing and selling on their own as well.³ Thus, it was several more years before this potential was realized.

In early 1975, Texas Instruments introduced the TMS 1000. It was, in essence, a more refined and general-purpose variant of the TMS 0100, and it took the world by storm.⁴

At introduction, the TMS 1000 cost \$10 per chip in bulk orders [10]; a few years later, this was down to \$2 [11]. For the first time ever, it was economically viable to produce consumer electronics with digital computers embedded inside. And Texas Instruments was practically the only supplier of this market. As one journalist put it in 1979, "Previous single chip [microcomputers] were the TMS 1000 from Texas, and, er, and that was it". [15]

Over the course of its production life, approximately 221 million TMS 1000 one-chip microcomputers were sold. In 1980 alone—when production peaked—44 million were sold, capturing 45.8% of the four-bit microcontroller market, and 37.3% of the microcontroller market as a whole [8, Chapter 1]. In all likelihood, for a few years, the TMS 1000 was the most-produced computer in the world.

In a list of applications, published in 1981, Texas Instruments stated that the TMS 1000 had been used in alarms, scales, meters, ovens, printers, terminals, door chimes, gas pumps, jukeboxes, radios, thermostats, microwaves, traffic lights, coin changers, clock systems, fuel meters, taxi meters, televisions, copy machines, fax machines, pinball machines, vending machines, washing machines, accounting machines, annunciators, cash registers, air conditioners, computer displays, electronic games, photo processing, security systems, telephone switching, appliance timers, computer peripherals, automobile electronics, and, of course, lots and lots of calculators [30, 2-16]. Looking through contemporary patents and advertisements of the time confirms this information. The TMS 1000 really was everywhere.

The decline of the TMS 1000 was expected and not particularly dramatic. It was always considered to have a somewhat backwards, primitive system architecture. Programming for the TMS 1000 was abstruse; Texas Instruments had definitely designed it with their previous business model in mind, where companies would tell TI what they wanted, and TI would do the electronics design and programming themselves,

³The architecture of the TMS 0100 is very unusual. Opcodes are 11 bits long, and include completely parallel addition and subtraction operations. Memory consists of three massive 44-bit storage registers, which encode numbers in decimal (using BCD, or binary-coded decimal, which uses four bits to encode a number between 0 and 9, instead of using the full possibility space to encode a number between 0 and 15) so that they could be outputted without conversion, at the cost of wasting space. There are no function or subroutine call instructions, and the ROM can only hold 320 instructions total [4].

⁴The TMS 0100 was the first microcontroller. The TMS 1000 has a fair claim to the title of 'first general purpose microcontroller'. Sometimes this is shortened to 'first microcontroller', which is incorrect.



Figure 2: An assortment of late 1970s electronic games, all of which use TMS 1000-family chips. Note the presence of Milton Bradley’s Simon, and TI’s own Speak-and-spell, still popular to this day. Additionally present is the Milton Bradley Microvision, considered the first handheld game console to use interchangeable cartridges. Some of these used support chips, such as the Speak and Spell (which used an external ROM chip and a speech-synthesizer chip), while many others, such as Parker Brothers’ popular Merlin, only consisted of a TMS 1000, some buttons, and a display [18]

and so had completely deprioritized ease of programming in favor of cost-effectiveness. This is fine for TI employees, who are paid specifically to write programs for TI products, but programmers elsewhere were less willing to invest time and effort learning the necessary idiosyncratic techniques. The use of a mask ROM (read-only memory) had a similar effect: it kept costs down, but hindered rapid prototyping [17].⁵

Thus, the main virtue of the TMS 1000 (once other competitors had arrived on the market) was that it was cheap. But the computer industry in the 70s and 80s was vicious and fast-paced. Price wars occurred in almost every level of every market sector. When the state of the art advanced, and more powerful one-chip microcomputers arrived on the market, with less unusual architectures, at comparable prices, the TMS 1000 was over. Between 1979 and 1984, a space of five years, the TMS 1000’s market share in the microcontroller market went from 45.2% (almost one in every two microcontrollers produced was a TMS 1000!) to barely 2% [8, Chapter 2].

Texas Instruments came out with numerous successors in the TMS 1000 family with additional capabilities, but none of them outlived the original [30, 2-2]. However, many TMS 1000 microcontrollers survive *in situ*, diligently executing code programmed into them almost five decades prior, and they will live on indefinitely in the

⁵The cited article, from the April 5, 1979 issue of EDN magazine, will give you a good feel for how the TMS 1000 was regarded in its day. It is available on the Internet Archive. The fact that the TMS 1000 is hard to program for is reinforced by the fact that even the well-versed author of this article misunderstands how the LDP instruction works within subroutine calls.

hearts of those interested in vintage electronics and computers.⁶

0.3.1 A Note on Nomenclature

Texas Instruments typically referred to the TMS 1000 as the “TMS 1000”, including a space in the middle, and that is the convention this thesis will follow. In component catalogs, and certain later publications, TI omitted the space (perhaps assimilating it into the one-word naming convention of other microprocessors and microcontrollers). A dash is never used in any official Texas Instruments publication to present knowledge.

⁶According to TI, the failure rate of PMOS TMS 1000 microcontrollers is 0.05% per 1000 hours of operation [30, 2-18]. This suggests that on average, a TMS 1000 will last 228 years in continuous operation, and that a TMS 1000 that has been continuously running since 1978 has an 81.6% chance of still working today. This is probably not entirely accurate, since the failure rate is likely not linear on extremely large numbers of hours of operation, and it doesn’t account for age-related effects, like the breakdown of materials, or adverse environmental conditions. However, these chips were well-manufactured for the time and there’s not a lot to go wrong in them, and every one that I’ve encountered has worked.

Chapter 1

Electricity and Semiconductors

1.1 Introduction

In this chapter, we will begin with an abbreviated overview of the fundamental concepts of electricity that underlie the design and function of electric circuits. Then, we will look at the electrical properties of semiconductors, which builds up to an examination of the electrical operation of transistors. The circuit of the TMS 1000 is almost entirely made out of transistors, so we must thoroughly understand how these building blocks work before we can understand how the larger circuits function.

1.2 A Crash Course in Electronics

1.2.1 Electromagnetism and Charge Carriers

Electromagnetism is one of the four fundamental forces in physics, and it affects charged particles, which can be either positively or negatively charged. Particles with the same charge repel; particles with opposite charges attract.

Coulomb's Law

The force of this attraction or repulsion can be calculated with Coulomb's law, which states that the force between two charged particles is proportional to the degree of charge of each particle, and inversely proportional to the square distance between the two particles. In other words, for some constant k , for two charges q_1 and q_2 and distance d between them, $|F| = k \frac{|q_1||q_2|}{d^2}$.

The forces exerted by a charge, or group of charges, on other charged objects is an **electric field**. It is called such because electromagnetic attraction and repulsion is exerted on all sides of a charged particle, and diminishes rapidly with distance, according to Coulomb's law. Thus, a circular 'field' surrounds charged particles, in which that particle can exert a meaningful electromagnetic force.

Charged Particles

The fundamental negatively-charged particle is the electron. The fundamental positively-charged particle is the proton, with a positive charge equivalent to the electron's negative charge. Electrons are held to the nucleus of atoms by the electromagnetic attraction between the protons in the nucleus and the orbiting electrons.

If the number of protons and electrons in a region are equal (such as in a non-ionic atom), the charges in that region are balanced, and every electromagnetic force is cancelled out. Such a region is considered to be **electrically neutral**. If the number of protons and electrons is unbalanced, the charges are unbalanced, and meaningful electromagnetic forces may occur. Discussions of charge usually refer primarily to such a state of unbalanced charge, or net charge.

Note that this concept of electrical neutrality/polarity is relative, and based on scale. For instance, a hydrogen atom is electrically neutral, but the proton alone and the electron alone, which make up its constituent parts, are electrically charged.

Charge Carriers

A particle or quasiparticle which carries a charge and is free to move is called a **charge carrier**, because it has the capability to 'carry' a charge through a substance.

If a charge carrier encounters an electric field of the same polarity (positive or negative) as the charge carrier, electromagnetic force will imbue the charge carrier with motion, away from the electric field. If a charge carrier encounters an electric field of an opposite polarity, an attractive electromagnetic force will pull it towards the field. As a charge carrier is charged, the relative charges of the charge carrier's source and destination are altered. In this manner, a charge carrier carries charge.

Electrons, when in a material which allows them to be free from atomic bonds, are negative charge carriers. Ions (atoms with more or less electrons than they have protons), in a liquid or gaseous environment, can serve as negative or positive charge carriers. In certain materials, protons may be used as charge carriers—but as a raw proton is equivalent to a positive hydrogen ion, they are considered ions. A 'missing' electron in the valence (outermost) shell of an atom, when connected to other atoms through atomic bonds, can function as a positive charge carrier quasiparticle, as atoms will fill the electron hole by taking an electron from a neighbor's valence shell, thereby passing on the hole and allowing it to be mobile.

Notably, opposite-charged charge carriers will recombine if they come into contact with each other, and become electrically neutral. For instance, a positive ion with an electron hole will accept a free electron into its shell, and become a normal, non-ionic, atom.

Additionally, of note, charge can be passed between different charged particles and charge carriers, as charge carriers all generate their own electric fields. Thus, for instance, if two copper wires are connected by an ion solution, charge may pass between them.

1.2.2 Voltage

The electromagnetic force pushes like-charged charge carriers away from each other. Thus, if one region has a greater concentration of like-charged charge carriers, and it is connected to a region with a lesser concentration of the same-charged charge carriers, the electromagnetic force will push charge carriers from the first region to the second region. The concentration of charge in a region is known as its **electrical potential**, as there is a potential for it to exert electrical force if connected to a region with a different electrical potential. The difference in electrical potential between the two regions, caused by the relative concentration of charge, is **voltage**.

Voltage is always relative. If point A is more negatively charged than point B, a negative voltage exists between point A and B, and an equivalent positive voltage exists between point B and A. Even a region filled to the brim with charge will have no voltage, when compared to itself.

Ground

It is immensely useful to have a common reference point to measure voltages against. This is called the **ground**. It must be remembered that the ground is still an arbitrarily chosen point of comparison.

In many applications, it is best to have the ground be a region of relatively stable charge, so that measurements against it will be consistent. In direct-current applications, the ground is usually either more positively charged or more negatively charged than anything else that it will be measured against, or at least close to such an extreme. For instance, in an isolated circuit with a single battery as the only voltage source, either the negative or the positive terminal would make an effective ground, as the negative terminal will have more of a negative charge than anywhere else in the circuit, and vice versa for the positive terminal.¹ By using such a ground, mathematically, every voltage in the circuit will have the same sign (positive or negative).

The subsurface of the Earth has a relatively constant electric potential, and thus is often used as a ground, such as in household electrical circuits.²

Symbols and Nomenclature

The commonly-used symbol, to represent voltage between points A and B, is V_{AB} . V_{AB} is positive if A is more positive than B.

¹This is the difference between positive- and negative-ground automobiles. The choice of a ground is that of an arbitrary reference point, but circuits are designed on the premise that voltages relative to that reference point can be induced in order to create electrical effects. A positive-ground automobile connects the positive battery terminal to the chassis, where it is used by many components, while a negative-ground automobile connects the negative battery terminal to the chassis.

²Additionally, the Earth is close to being electrically neutral, and has an enormous capacity for absorbing charge, without its electrical potential changing. Grounding in household electrical systems is part of a system of grounding and bonding that neutralizes a charge—using a ground—and dissipates charge in the case of a fault, using bonding.

If only one point of reference for a voltage is used, such as would be denoted by V_A or V_B , that voltage is assumed to be measured against a ground.

The most common unit of measurement for voltage is the volt, with a symbol of (you may have guessed it) V .

1.2.3 Current

When two regions with a voltage difference between them are connected with a material containing free charge carriers, charge will travel between them. This is **current**. Current refers to the flow of charge carriers past a point, and the measured rate of this flow.

Unlike voltage, current is instantaneous, or measured at a single point rather than as a relative measure between two points.

Direction

The direction of current, given a voltage difference, depends on the charge carriers being used. In the convention set in the 18th century, current is arbitrarily said to flow from areas of positive voltage to areas of negative voltage. This is the behavior which positive charge carriers exhibit. However, negative charge carriers, such as the ever-present electron, travel from areas of negative voltage to positive voltage. These two current directions are termed conventional current and electron-flow current respectively. Whichever one reflects reality is determined by the charge carrier used in the specific material.³

Symbols and Nomenclature

Current is most commonly referred to using the symbol I . The current at point A , or a specific current value A , may be designated by I_A . If points A and B are in series (the definition of this will be discussed shortly), and have the same current passing through them, that current may be written as I_{AB} , and is equivalent to I_A and I_B .

The most common unit of measurement for current is the ampere, which is often shortened to *Amp* or A .

1.2.4 Conductance and Resistance

Charge can travel through some substances relatively easily, but has difficulty travelling through other substances. The degree to which a unit of a substance prevents charge from flowing through it is its **resistivity**. The degree to which a unit of substance allows charge to flow through it—the inverse of its resistance—is its **conductivity**.

A **conductor** is a substance with high conductivity (low resistivity), such as copper and aluminum. An **insulator** is a substance with low conductivity (high

³In some cases—such as electrolytes containing both positive and negative ions, or semiconductors, as will be discussed shortly—current can be travelling in both directions simultaneously.

resistivity), such as glass. A **semiconductor** is any substance with lower conductivity than a conductor, but higher conductivity than an insulator. These distinctions are relative and not absolute categories, since a myriad of factors can influence whether a current can be induced in a substance, such as temperature, thickness of material, and the voltage being applied.

But what makes a conductor a conductor and an insulator an insulator? The answer is the presence of free charge carriers. But why do some substances have more free charge carriers than others?

Recall that electrons are held to the nucleus of an atom by a perfectly-balanced electromagnetic force. If additional energy is provided to an electron, it may be able to overpower the electromagnetic force binding it to the nucleus, and be ‘freed’. We may recall that a free electron is a charge carrier.

As the electromagnetic force rapidly diminishes with distance, it is easier to free electrons on the outermost, or valence, shell of an atom. That is to say, the electrons that are furthest away from an atom’s nucleus can break free more easily than those which are close to the nucleus, and are subject to stronger forces.

When multiple atoms are together in a material, they form chemical bonds with each other which may make it harder for electrons to break free. Thus, the atomic structure of a material influences how easy it is for electrons to break free, which affects the prevalence of charge carriers, which affects the conductivity.

Metallic conductors, in their atomic structure, possess a small number of loosely-bound electrons in their valence shells, which allows electrons to break free with minimal additional energy, and go where they will. Insulators have filled out, strongly-bound electrons in their valence shells, preventing electrons from escaping and becoming charge carriers. Semiconductors lie in the middle, where valence electrons can break free with relatively small, but nontrivial amounts of energy. This causes the peculiar characteristic that the conductivity of semiconductors increases with temperature, which is not true with conductors [12, 15].

Each electron shell of an atom imparts a degree of energy to the electrons that reside in it, with the valence shell electrons receiving the most. These relative levels of energy are often conceptualized as ranges of energy known as bands. The amount of energy imparted to electrons in the valence shell is called the valence band. The amount of energy required to break free from the proton nucleus’ attractive force is called the conduction band. The difference in energy between the valence band and the conduction band is called the band gap. Conductors have no band gap. Insulators have large band gaps, while semiconductors have small and easily-surmountable band gaps [19, 39-40].

Resistance and Conductance

Resistance and conductance are the applied forms of resistivity and conductivity (sometimes known as specific resistance and specific conductance respectively). For instance, the resistance of a wire can be calculated by multiplying the resistivity of the material the wire is made of times the length of the wire, multiplied by its cross-sectional width.

Resistance is defined relative to voltage and current: the current passing through a region is equivalent to the voltage across the region divided by the resistance. So the resistance is equivalent to the voltage divided by the current. This is **Ohm's law**, and is of extremely great utility.

Note that for many materials, and many electrical components, the resistance will be a constant (a single, unchanging number). We will see how resistance is not constant in semiconductors and transistors later on in this chapter.

Symbols and Nomenclature

The conventional symbol for resistance is R . The standard unit of measurement for resistance is the ohm, symbolized by Ω .

As conductance is the reciprocal of resistance, $\frac{1}{r}$ for some resistance r , it is most commonly stated in terms of resistance.

Ohm's law may be written as $I = \frac{V}{R}$, $R = \frac{V}{I}$, or $V = IR$.

1.2.5 Circuits

A circuit can best be described as a closed loop of electricity. It can be as large as a power grid or as small as a wire connecting two battery terminals. Circuits are fundamentally necessary for doing anything useful with electricity, as without a closed loop, continuous current cannot be induced.

Circuit diagrams are graphic representations of circuits. In them, lines represent wires, and symbols represent electrical devices used in the circuit. We will look at several such devices, and the symbols used to represent them.

Terminology

circuit element A device to accomplish a single electrical task which may be included in a circuit. The building-blocks of circuits. Also may be called an electrical component.

terminal The portions of an electrical component which interface with the rest of the circuit. May also be called pins.

voltage drop A voltage potential, or difference in voltages at two points in a circuit. As can be noted from Ohm's law, if a circuit element has nonzero resistance, if a voltage source is connected through it, there will be a voltage drop across it and current will flow across it.

load The total voltage drop of a circuit or a circuit path.

A positive voltage is often known as 'high' voltage, and a negative voltage is known as a 'low' voltage. This is separate from the common use of the term high voltage, as in a powerful voltage of large magnitude.

A positive voltage source is a negative voltage sink. A negative voltage source is a positive voltage sink.

Two circuit elements are described as being ‘shorted’ if they are connected with low resistance.

Two circuit elements are described as being ‘in series’ if they are connected linearly; charge passing through one must pass through the other in order to reach a voltage sink.

Two circuit elements are described as being ‘in parallel’ if they are connected such that they share a voltage source and a voltage sink, but charge passing through one element does not need to pass through the other in order to reach a voltage sink.

Kirchhoff’s Laws

Two useful circuit properties derive from the conservation of charge.

Kirchhoff’s current law states that the sum of the currents into a point in a circuit equals the sum of the currents leaving that point in the circuit. Thus, if multiple circuit elements are connected in series, the current passing through each element will be the same.

Kirchhoff’s voltage law states that the sum of the voltages or voltage drops in a circuit is zero. This has the import that circuit elements connected in parallel will have the same voltage across them.

Direct and Alternating Current

In a circuit using **direct current** or DC, charge only flows in one direction. In other words, for two connected voltage sources A_+ and B_- , $V_{AB} \geq 0$ at all times, and by the positive convention, current will only flow from A to B .

In a circuit using **alternating current** or AC, voltage sources regularly alternate in polarity. Thus, the current switches directions each of these cycles.

Alternating current is preferred for, and almost universally used in, long-distance power distribution due to the ability to efficiently switch between voltage levels using transformers. Electrical properties peculiar to alternating current circuits are outside the scope of this overview.

Direct current is utilized in circuits powered by batteries, and many household electronics, where alternating current is converted to direct current through a process called rectification.

1.2.6 Electrical Components

We will now briefly discuss some of the most important electrical components, with a focus on functionality. This is not exhaustive; of note, inductive devices will not be discussed. Transistors will be covered later in the chapter.

Example images of actual components are included. But beware, as looks can be deceiving! Electronic components are packaged in standardized ways, so very different components may look very similar on a circuitboard. Also the same component, with the same function, may be manufactured using different technologies and take on radically different forms.

Voltage Source

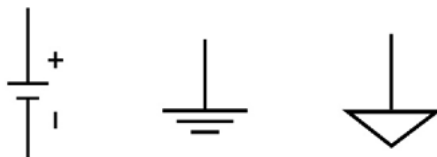


Figure 1.1: Standard circuit diagram symbols for a battery, an earth ground, and a signal ground.

A voltage source provides a continuous supply of some voltage that is not equivalent to the circuit's ground. Batteries are an archetypal example; by means of a stable chemical reaction, a voltage difference is maintained between the battery's terminals. This chemical reaction occurs continuously and is dependent on the current between the terminals, or else the voltage between the terminals would start high and quickly deplete with current.

Resistors



(a) American standard circuit diagram symbol of a resistor



(b) A variety of discrete resistors, including a 'varistor', which has low resistance except at high voltages, and are useful in surge protection. The colorful bands on resistor housings are a code used to indicate the resistance of each resistor.

Figure 1.2: Resistors

A resistor is a small region of relatively high resistance placed in a circuit of conductive material, in order to add resistance to a circuit path.

Resistors are used in order to control the current which travels down a circuit path.⁴ For instance, if a component cannot handle a large amount of current, adding

⁴An important use of this is in impedance matching. For instance, in order to ensure that an equivalent amount of current travels down two paths in parallel, the resistance of both paths must be approximately equal.

a resistor on its path will reduce the current flow, by Ohm's law. Alternatively, a resistor can be used to induce current to flow by creating a voltage drop.

Diodes



(a) Standard circuit diagram symbol of a diode. Current is allowed to pass in the direction that the arrow is pointing, in the positive current convention.



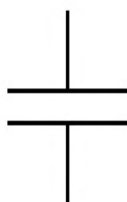
(b) A typical diode [21]

Figure 1.3: Diodes

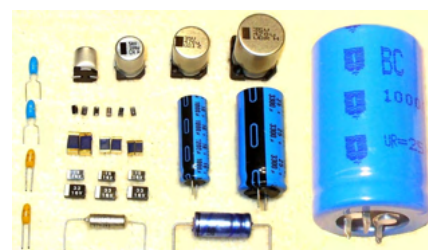
A diode restricts the direction of current to a single direction. It accomplishes this by having very low resistance for one current direction, and very high resistance for the opposing current direction. Thus, very little current can pass one direction, but current may pass freely in the other direction.

Diodes can be used to electrically insulate circuit components, to filter electrical signals, for converting alternating current to direct current, for many signal processing tasks, or, in the special case of light-emitting diodes, to produce light.

Capacitors



(a) Standard circuit diagram symbol of a capacitor. In action, the two 'plates' become polarized against each other.



(b) A variety of capacitors [9]

Figure 1.4: Capacitors

Capacitors are charge-storing devices. By separating two conducting regions of different charges by a thin insulating region, positive and negative charge carriers will exert an attractive force on each other, but will not be able to recombine due to the insulating region. Charge carriers will remain in the capacitor by the attractive force they have on each other.

Capacitors can be used to provide voltage for a short time when primary voltage sources are disconnected, to smooth out unstable voltages and reduce electrical noise (known as a filter cap), and to create a larger voltage output from a small voltage input, among many other uses.⁵

Capacitance, like resistance, is a fundamental electrical property, and occurs in electrical components that are not capacitors as well.

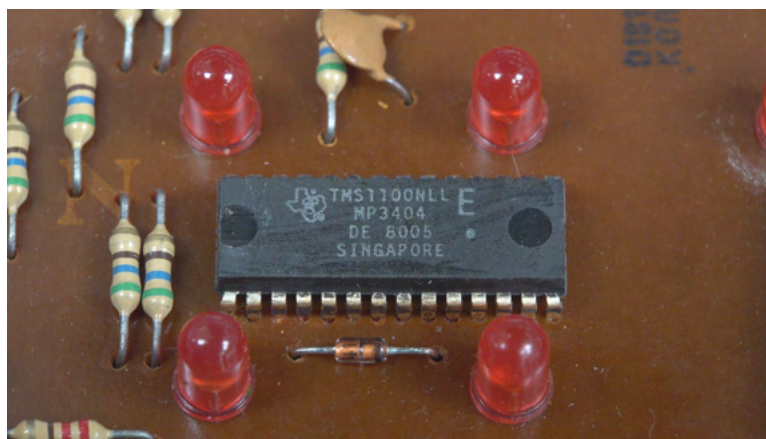


Figure 1.5: Eight resistors and five LEDs are visible alongside the TMS 1100. The circular, ceramic-wrapped element above the chip is a capacitor. The axial-mounted component below the chip is a diode. [2]

1.3 Semiconductor Doping

1.3.1 Introduction

‘Doping’ is the process of introducing impurities into a semiconductor in order to modify its electrical properties. In particular, doping is used to increase the number of charge carriers. This can be done by increasing the number of electrons, known as negative doping (due to the conventional negative charge of electrons), or increasing the number of mobile electron holes, known as positive doping.

The impurity which is introduced into doped silicon (sometimes also known as extrinsic silicon) is called a dopant. A semiconductor that has been negatively-doped is described as an n-type semiconductor; a semiconductor that has been positively-doped is described as a p-type semiconductor.

Silicon is by far the most widely-used semiconductor, and was used in the TMS 1000 microcontroller. As such, we will focus on it.

⁵This doesn’t violate conservation of charge, as the resultant current is correspondingly reduced.

1.3.2 How it works

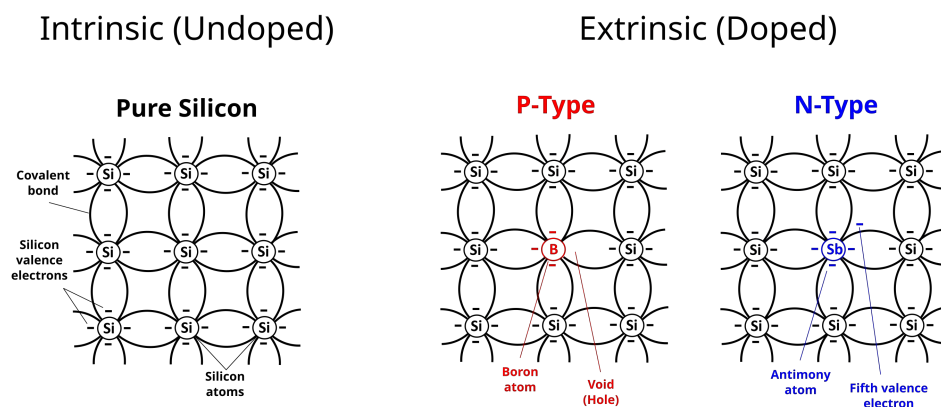


Figure 1.6: Silicon Doping [31]

Undoped (Intrinsic) Silicon

Silicon’s remarkable and useful properties come from its atomic and molecular structure. A silicon atom has four electrons on its valence shell. In pure silicon, neighboring silicon atoms will form strong covalent bonds, sharing electrons in order to fill up each atom’s valence shell, forming a distinctive, very strong ‘cubic lattice’ structure [5, 3-5].

N-type Doping

The most common n-type dopant for silicon is phosphorus, which contains five electrons on its valence shell, while silicon only possesses four electrons on its valence shell. Thus, when a phosphorus atom forms covalent bonds with its silicon neighbors, four of its valence shell electrons will form bonds, and one will not be able to. This spare electron is very weakly bonded to the phosphorus atom, as the shell it resides on is filled up without it, while the covalent bonds of the silicon atoms with each other and with the phosphorus atom’s other valence electrons are very strong.

Because of this, only a very small amount of additional energy is sufficient to break the spare phosphorus electron free. By heating up the doped silicon, enough additional energy is introduced that every “spare” electron breaks free and becomes a free charge carrier, while the rest of the atoms are unaffected. Only very minute quantities of phosphorus are sufficient to achieve this effect. Each free electron leaves behind a positive phosphorus ion.

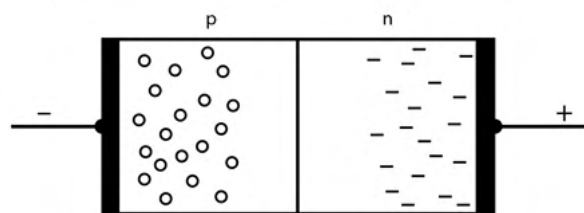
We might ask, if the goal is to introduce additional free electrons to silicon, why not do so directly? However, adding electrons directly to silicon would cause the silicon to be negatively-charged, while adding neutrally-charged phosphorus to neutrally-charged silicon results in a stable, neutrally-charged substance with a surplus of free electrons [5, 6-7].

P-type Doping

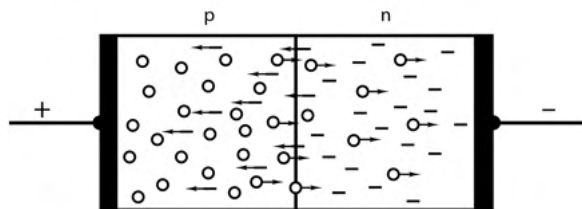
The most common p-type dopant for silicon is boron, which has three electrons on its valence shell (as opposed to silicon's four). Thus, when a boron atom forms covalent bonds with its silicon neighbors, it can only form three. In order to fill this fourth covalent bond, it must 'steal' an electron from one of its neighbors' valence shells, which then passes on the hole, leaving behind a negative boron ion, and creating a positive silicon ion. As this hole is thus mobile, it functions as a charge carrier, despite not being a particle. Like with phosphorus, positive charge carriers are added to silicon without positively charging the silicon [5, 8-9].

Of note, n-type semiconductors contain some holes, and p-type semiconductors contain some free electrons. These charge carriers are called **minority carriers**, and the dominant charge carrier is referred to as the **majority carrier** [5, 10].

1.3.3 P-N Junctions



(a) A p-n junction with reverse bias



(b) A p-n junction with forward bias

Figure 1.7: P-N junctions (Adapted from [12, 16])

Sufficiently-doped p-type silicon and sufficiently doped n-type silicon both readily conduct electricity. On their own, each type of silicon is a slightly inefficient but mostly-ordinary conductor. Together, however, new properties emerge.

Where a P region is joined to an n-region, an insulating layer is created between the two regions. Electrons and electron holes come into contact with each other and recombine, leaving only ions behind. Thus, an area is created that has no charge carriers, and has a layer of positive and a layer of negative ions, which repel further charge carriers from crossing the divide. This layer is called a **depletion region**, because the charge carriers in the region are depleted.

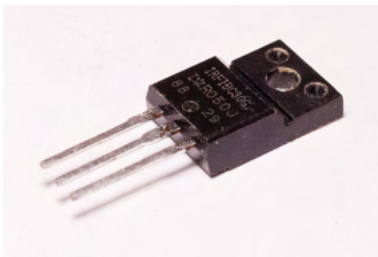
In practice, this creates a diode, since by applying negative voltage to the n-region, and positive voltage to the p-region, charge carriers in each region are pushed together from the repulsion of like charges, and the depletion region is shrunk until charge carriers can freely pass it. Doing the opposite only strengthens the depletion region, as the electrons and holes are drawn towards the poles [12, 15-6].

To put it another way: when voltage is applied of an opposite polarity to the charge carriers in each region, the electric field at each terminal of the p-n junction will attract the charge carriers on its own side. The charge carriers will stay where they are, and current will not flow. When a like-polarity voltage is applied, the electric fields are repulsing the charge carriers on their own sides, and attracting charge carriers from the other side, inducing charge carriers to move, so a current results.

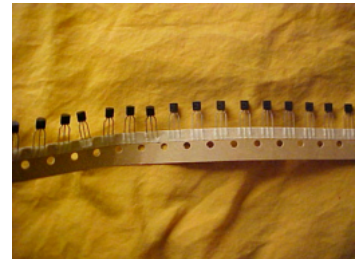
1.3.4 Bias

Forward bias refers to the situation where a voltage differential exists between the p and n-regions of a p-n junction such that the p-region is connected to a positive voltage source, and the n-region is connected to a negative voltage source. Under these circumstances, majority carriers will create a current.⁶

Reverse bias refers to the opposite situation: the n-region is connected to a positive voltage source, and the p-region is connected to a negative voltage source. Under these circumstances, only minority carriers will create a current, and it will be very small [5, 16-9].⁷



(a) A discrete MOSFET transistor [24]



(b) Discrete PNP transistors

Figure 1.8: Transistors

⁶In terms of electron flow, current flows from the n-region to the p-region. In terms of conventional current, current flows from the p-region to the n-region.

⁷A very large reverse-bias voltage differential will, however, cause the p-n junction to break down due to either a very strong electric field (Zener effect breakdown), or due to charge carriers being imbued with so much energy that more charge carriers are knocked free in a cascading effect (avalanche breakdown).

1.4 Transistors

1.4.1 Introduction

Transistors are three-terminaled electric components which allows voltage applied to one terminal to control current between the other two terminals.⁸ They can be used as amplifiers, with the control voltage ‘signal’ being translated into a proportionally increased current between the other two terminals, or as electrically-controlled switches, where the amount of current is not cared about, only its presence or absence.

There are two main varieties of transistors: bipolar junction transistors (BJTs) and field effect transistors (FETs). Bipolar junction transistors are the older type (invented in 1947, starting a revolution in electronics), and are more suited to amplification tasks, while field effect transistors are better suited to digital logic applications. There are two commonly-used types of FETS: Junction FETs (JFETs) and Metal Oxide Silicon FETs (MOSFETs) [13, 131-2]. This chapter will focus on FETs, and specifically the most widely-used kind of FET, the MOSFET. Information on other types of transistors, and how they work, can be found in Appendix A.

All kinds of transistors utilize p-n junctions. By placing two p-n junctions in series (creating either a p-n-p junction or an n-p-n junction), practically no current can pass from one end to the other; the two p-n junctions function analogously to two diodes aligned in opposing directions. By using various means to either create and destroy p-n junctions, or else weaken the depletion regions, transistors can allow and disallow current to flow.

1.4.2 P- and N-types

All transistors come in two flavors: those which sandwich an n-region between two p-regions (pnp bipolar junction transistors, p-channel MOSFETs), and those which sandwich a p-region between two n-regions (nnp bipolar junction transistors, n-channel MOSFETs). The operation of both types is exactly the same, but with the polarities reversed: a negative voltage potential in the control terminal of a p-type transistor does what a positive voltage potential does in an n-type transistor [13, 131-4].

For the sake of brevity, explanations will only refer to p-type transistors, which are what is used in the TMS 1000. In order to convert these explanations to refer to n-type transistors, change all references of “p-type silicon” to “n-type silicon” (and vice versa), change all references of “electron holes” to “electrons” (and vice versa), and change all references of “negative voltage” and “negative charge” to “positive voltage” and “positive charge” (and vice versa).

⁸Some MOSFETs have four terminals. This will be discussed later.

1.5 Introducing: the MOSFET

1.5.1 Field Effect Transistors

FETs are called such because they utilize an electric field to control current. FETs have three terminals: the gate, drain, and source. Voltage applied at the gate terminal controls current between the source and drain terminals. Some FETs have an additional terminal, which is termed the body terminal. We will see what the purpose of this terminal is in a later subsection.

Traditionally, the source is supposed to be more positive than the drain (V_{DS} is negative, V_{SD} is positive). However many FETs are symmetrical, and can function in reverse. Thus these become operational characteristics: the source is whichever terminal is more positive.

There are two configurations of FETs: enhancement-mode and depletion-mode. In enhancement-mode FETs, current is blocked when no voltage is applied to the gate terminal, and applying voltage to the gate terminal allows current to flow. In depletion mode FETs, current already flows when no voltage is applied to the gate terminal, and applying voltage to the gate terminal can partially or fully cut off the current [13, 135].

1.5.2 Metal Oxide Silicon FETs

Metal Oxide Silicon FETs are called such because of the traditional materials used in their construction. The gate terminal of a MOSFET is connected to a conductor (in early applications, aluminium was used), separated from the semiconductor body (usually made of silicon) by an insulator (usually a layer of oxide, created by oxidizing the silicon). Thus, three layers are created: metal, oxide, and silicon.

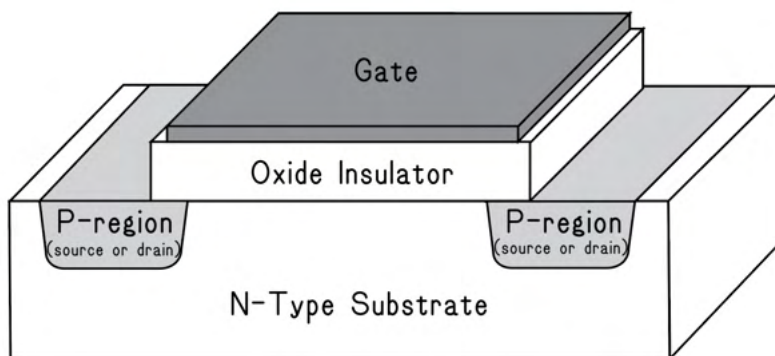
Enhancement-mode MOSFETs

In an enhancement-mode MOSFET, two highly-doped p-regions are embedded in n-type silicon, such that they are entirely separated. The source and drain terminals are each connected to one of the p-regions, while the gate is placed above the separating n-region, with a small amount of overlap with each p-region [5, 2-3][12, 25].⁹

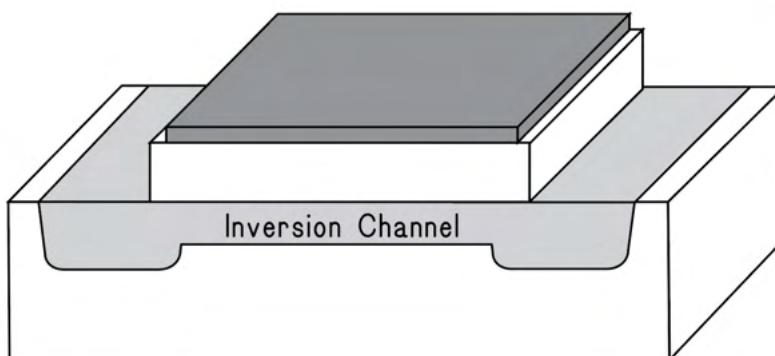
When no voltage is applied to the gate, current is blocked from passing between the source and drain, due to the two oppositional p-n junctions.

When a sufficient voltage negative relative to the n-type silicon substrate is applied at the gate, the majority carrier electrons in the n-type silicon are repelled from the area close to the gate, and minority carrier electron holes are attracted. Under normal circumstances, these electron holes would combine with the electrons in the gate and form a depletion region, but due to the insulating layer of oxide, this cannot happen, and they only collect. We may note that the MOSFET gate essentially forms a capacitor with the body.

⁹This overlap ensures that a continuous inversion layer is achieved, and adds additional capacitance [5, 43].



(a) MOSFET with no inversion channel



(b) MOSFET with inversion channel present

Figure 1.9: An enhancement-mode MOSFET has no inversion channel when the gate-source voltage is 0, while a depletion-mode MOSFET has an inversion channel then.

As this positive charge accumulates, a small layer is created where electrons have been repelled, and electron holes have been concentrated, which essentially amounts to an area of p-type silicon. This is called the **inversion layer**, or inversion channel, as the n-type silicon has become inverted. Current can now freely flow between the p-type source and drain regions, using electron hole charge carriers, through the temporarily p-type inversion channel [19, 291-2].

Depletion-mode MOSFETs

In a depletion-mode MOSFET, source and drain p-regions are embedded in n-type silicon, as in the enhancement-mode type. Additionally, a thin channel of p-type silicon is embedded between the source and the drain, connecting them and allowing current to flow between them. The gate is placed above this channel.

When positive voltage is applied to the gate, electron holes in the channel are repelled, and electrons are attracted, creating an inversion layer of temporarily n-type silicon. When this inversion layer is large enough, it cuts off the channel, blocking current from flowing.

It may alternatively be phrased that in depletion-mode MOSFETs, an inversion layer exists even when there is no electric field, and an electric field of the opposite polarity is required to remove it [19, 301-3].

Threshold Voltage

The smallest quantity of gate voltage which opens an inversion channel connecting the source and drain is known as the **threshold voltage** of a transistor [7, 6].

Though the conduction in a transistor is really dependent on the gate-substrate voltage, threshold voltages are expressed in terms of gate-to-source, to simplify equations [13, 137].

In an ideal transistor, the current passing through a transistor would be zero when the gate-source voltage is just below its threshold voltage, and be significant when the gate-source voltage is just above its threshold voltage. In practice, the transition is much less dramatic [13, 136-7]. We may note that silicon's conductivity increases with temperature; thus, a transistor's threshold voltage is somewhat temperature-dependent, and should be treated as an estimate [12, 73].

In enhancement-mode MOSFETs, the threshold voltage is negative, requiring the application of a negative gate voltage to allow current to pass, while in depletion-mode MOSFETs, the threshold voltage is positive, so current can pass at a voltage of zero [5, 37-8][13, 136-7]. Through the use of the threshold voltage concept, we have now consolidated our two models of MOSFET varieties into one model!

1.5.3 MOSFET body regions

The n-type substrate, in which the p-regions are embedded in, is often termed the bulk, or body. As has been noted, in order for current to flow through a transistor, the gate voltage must be negative compared to the body region.

In order to make circuit design easier, many discrete MOSFETs internally connect the source terminal and the n-type body. By shorting the source terminal and the body, now the gate voltage only has to be negative with respect to the source. This allows transistors to be treated as a discrete and self-contained component, and there isn't an extra base voltage value to keep track of.

However, because of this shorting, these MOSFETs become asymmetrical, since there is no longer an insulating double p-n junction between the source and the drain. Instead, there is a single p-n junction, with the drain on the p-side and the source on the n-side. So if the drain is more positive than the source, current will flow regardless of whether the gate is active or not. Thus, it must be guaranteed that the source is more positive than the drain at all times.

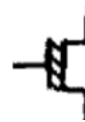
Additionally, shorting the source and drain prevents multiple MOSFETs from being created out of a single substrate of silicon, or else every source terminal would be connected together.

If the body is not shorted to the source, then for current to flow through a MOSFET, the gate voltage must be negative relative to whatever the body is connected to. Additionally, the body must be equally or more positive than the source, or else current would flow through the source-substrate p-n junction. Due to these factors, usually the body will be connected to the most positive voltage source in the circuit. This is the case in integrated circuits, such as the TMS 1000 [13, 134].¹⁰

1.5.4 MOSFET Notation and Convention



(a) A symbol used to represent enhancement-mode MOSFETs



(b) A symbol used to represent depletion-mode MOSFETs

Figure 1.10: MOSFET symbols used in TMS 1000 patent applications. In typical orientation, the top terminal is the drain, the left terminal is the gate, and the bottom terminal is the source.

When discussing electrical properties, in common notation, S refers to the source, D to drain, and G to the gate, and B to the body (e.g. V_{GD} is the voltage between the gate and the drain).

Additionally, V_T represents the threshold of the transistor. Note that this is a relatively fixed value and an abstraction of the electrical properties of a transistor, not a voltage measurement.

When the inversion channel is present, and current can pass through a MOSFET, it is said to be ‘open’. When it is not, the MOSFET is said to be ‘closed’. When a MOSFET is in a conducting state different from that which it would be at with $V_G = 0$, it is said to be ‘active’ or ‘on’; otherwise, it is said to be ‘inactive’ or ‘off’.

¹⁰The drain also must be less positive than the body, but in p-channel MOSFETs, the drain is already defined as being less positive than the source.

The ‘switching speed’ of a MOSFET is the time it takes for it to change states.

A Choice of Ground

For voltages, by convention, V_S is treated as a ground [14, 56]. Thus, for example, V_G is equivalent to V_{GS} .

On a physical level this seems counterintuitive, or even just plain wrong. After all, in order to create an inversion layer, the gate must be negatively-charged compared to the silicon substrate, not to the source. In truth, the reason behind this convention is that it makes voltage and current calculations much easier.

A consequence of this convention is that V_T is dependent on the source voltage. This can be noted by observing that by the definition of the threshold voltage a transistor is always supposed to be open if $-V_{GS} > -V_T$, that the state of a transistor’s inversion channel is dependent on V_{GB} and not V_{GS} , and that V_{SB} may not be equal to 0. The upside is that this allows us to roll up a lot of complexity into one variable.

The effective threshold value of a transistor, for some V_{SB} is calculated by adding the threshold voltage if $V_{SB} = 0$ to a complex term measuring the influence of V_{SB} on the threshold voltage, which is influenced by a myriad of factors such as the relative concentration of charge across the transistor and the material properties of transistor components. We will only note that this term is proportional to the square root of V_{SB} , allowing it to oftentimes be ignored without great hazard [14, 60].

1.5.5 MOSFET modes of operation

MOSFETs have three primary modes of operation. The **cutoff mode** is when the transistor is ‘off’, and current is blocked between the source and drain. The **linear mode** is when a channel is created, and some current is passing. The **saturation mode** is when the greatest possible amount of current is passing through the transistor, for a given gate-source voltage.

Cutoff Mode

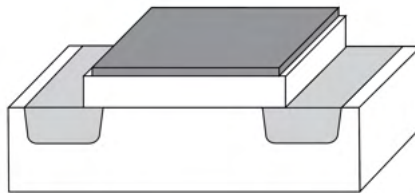


Figure 1.11: MOSFET in cutoff mode

In the cutoff mode (or cutoff region of gate voltages), the gate voltage is more positive than the threshold voltage ($V_{GS} - V_T > 0$) and majority carrier current may not pass.

However, we may recall, that minority charge carriers can still carry a small current. Thus, even in the cutoff mode, a small current passes through the transistor. This is called the subthreshold, weak inversion, or leakage current.

The amperage of a leakage current is not significant enough to influence most circuit components, and can for the most part be ignored. But it has a very important side effect: a MOSFET can never totally electrically disconnect the source and drain. It may take a very long time, but eventually the levels of charge of the source and drain will equalize, if one is not connected to a voltage source [13, 138].

Linear Mode

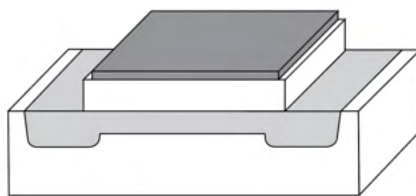


Figure 1.12: MOSFET in linear mode

In the linear mode (or linear region), the gate voltage is more negative than the threshold voltage ($V_{GS} - V_T < 0$), and the drain voltage is less negative than the difference between the gate-source voltage and the threshold voltage ($V_{DS} > V_{GS} - V_T$, alternatively $|V_{DS}| < |V_{GS} - V_T|$). In this mode, drain-source current is proportional both to the gate voltage and to the drain-source voltage; increasing either will increase the drain-source current. The linear mode is called such because current increases roughly linearly with the drain current, with a slope of approximately $V_{GS} - V_T$. It is also sometimes called the ohmic mode, because the transistor behaves like a variable resistor, or the triode mode, because the transistor behaves like a vacuum-tube triode [5, 42-5][13, 136-8].

Saturation mode

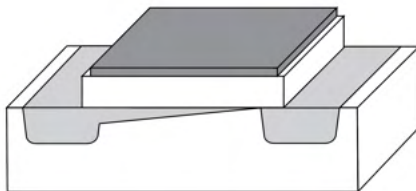


Figure 1.13: MOSFET in saturation mode

A MOSFET enters the saturation mode when the gate voltage is more negative than the threshold voltage, and the drain voltage is equal to the difference between the gate voltage and the threshold voltage ($V_{DS} = V_{GS} - V_T$). As the voltages are equal, the electric fields of the drain and the gate are of equal strength, and the drain attracts electron holes with the same force that the gate is attracting electron holes. Thus, any electron hole that is closer to the drain than the gate, however marginally, will be sucked into the gate. This makes the channel very narrow, and creates a depletion region around the drain that blocks off most of the channel, as electron holes are pulled into the drain, and electrons are repelled by both the gate and drain, leaving no charge carriers. This is known as **pinch-off**, as the channel is ‘pinched’ near the drain, and the current at this point is the maximum possible for the given transistor at that V_{GS} .

Increasing the magnitude of V_{DS} ($|V_{DS}|$) above $|V_{GS} - V_T|$ will fully cut off the channel and create a depletion region, with no charge carriers. However, as this depletion region is exactly as large and as strong as the drain’s electric field, since it was created by the drain pulling charge carriers into itself, the insulating ability of the depletion field and the attracting ability of the drain’s electric field will perfectly cancel each other out. Thus, the effective drain-source current will remain *exactly the same*, at the $V_{DS} = V_{GS} - V_T$ level [5, 45-7][13, 136-8].

In conclusion, when a MOSFET is activated with $V_{GS} - V_T < 0$, increasing the size of the source-drain voltage will linearly increase the current, until the MOSFET is saturated, at which point the current will become constant. We may say that the maximum effective drain-source voltage drop is equivalent to $V_{GS} - V_T$.¹¹

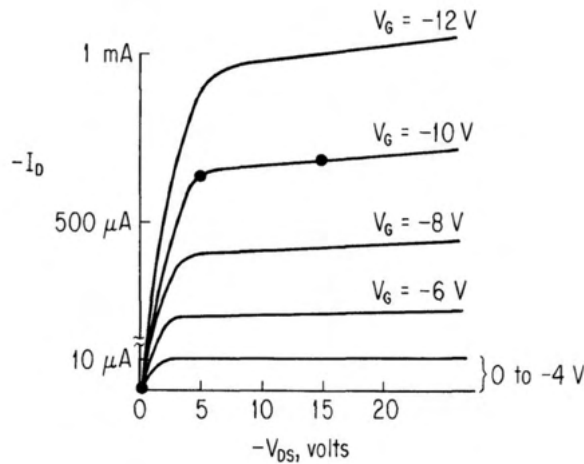


Figure 1.14: A graph of current passing through a MOSFET, relative to V_G and V_{DS} [7, 6]

¹¹There are more complicated effects that occur if the size of the depletion region becomes large enough to significantly shorten the length of the channel [5, 47].

1.5.6 MOSFET current equations

The current which passes through an open MOSFET is dependent on many variables. However, many of them are constant physical properties of the transistor materials, or else tend to vary relatively little (namely charge carrier mobility). These constants and almost-constants are thus reduced to a single variable, β , so that current can be expressed purely as a function of voltage, in the following equation:

$$I_{DS} = \beta[(V_{GS} - V_T)V_{DS} - \frac{V_{DS}^2}{2}]^{12}$$

Note that I_{DS} does not express the current between the drain and source, but the current at the drain and source, which are equivalent [7, 29].

The MOSFET enters the saturation mode when, $V_{DS} = V_{GS} - V_T$. We can thus derive that when entering the saturation mode:

$$\begin{aligned} I_{DS} &= \beta[(V_{GS} - V_T)(V_{GS} - V_T) - \frac{(V_{GS} - V_T)(V_{GS} - V_T)}{2}] \\ &= \beta(V_{GS} - V_T)^2 - \frac{\beta(V_{GS} - V_T)^2}{2} \\ &= \frac{\beta}{2}(V_{GS} - V_T)^2 \end{aligned}$$

This make sense: I_{DS} can only increase when $|V_{GS} - V_T|$ increases, due to pinch-off, and so is no longer proportional to V_{DS} . Additionally, we can note that the derivative of the current equation, $\frac{dI}{dV}$, is equivalent to zero at $V_{DS} = V_{GS} - V_T$.

1.5.7 MOSFET capacitors

When a negative voltage is applied at the gate of a MOSFET, electron holes are pulled to the surface of the silicon near the gate by the attractive force of the opposite-charged electrons. This force works both ways, however: the electron holes in the silicon will keep the electrons in the gate from diffusing away, and even if the external voltage source is removed, the MOSFET will hold itself on for some time. Connecting the gate to a voltage sink will quickly exhaust the charge stored in it [14, 48].

1.5.8 MOSFET resistors

In their linear mode of operation, MOSFETs function similarly to voltage-controlled resistors.

By Ohm's law, $R = \frac{V}{I}$. Thus, the resistance across the source and drain of a transistor in linear mode is $\frac{V_{DS}}{k[(V_{GS} - V_T)V_{DS} - \frac{V_{DS}^2}{2}]}$, which simplifies to $\frac{1}{k[(V_{GS} - V_T) - \frac{V_{DS}}{2}]}$.

Note that for a fixed $V_{GS} - V_T$, this behaves like a varistor. When V_{DS} is small compared to $V_{GS} - V_T$, the resistance is relatively constant, and can be 'set' by altering

¹²Full equation: $I_{DS} = \frac{\mu\epsilon_{ox}\epsilon_o W}{t_{ox} L}[(V_{GS} - V_T)V_{DS} - \frac{V_{DS}^2}{2}]$, where μ is the charge carrier mobility, ϵ_{ox} is the permittivity of the oxide layer, ϵ_o is the permittivity of the free space, t_{ox} is the oxide thickness, W is the channel width, and L is the channel length. This can also be written as $\mu C_{ox} \frac{W}{L}$, where C_{ox} is the specific capacitance of the gate oxide [5, 42-5].

V_{GS} . When V_{DS} approaches $V_{GS} - V_T$, $k[(V_{GS} - V_T) - \frac{V_{DS}}{2}]$ gets very small, and the resistance correspondingly quickly increases.

In the saturation region, the resistance is theoretically infinite, since increasing the voltage doesn't increase the current at all. This is an instance where Ohm's law does not properly function (but does remain a useful tool).

1.6 Integrated Circuits

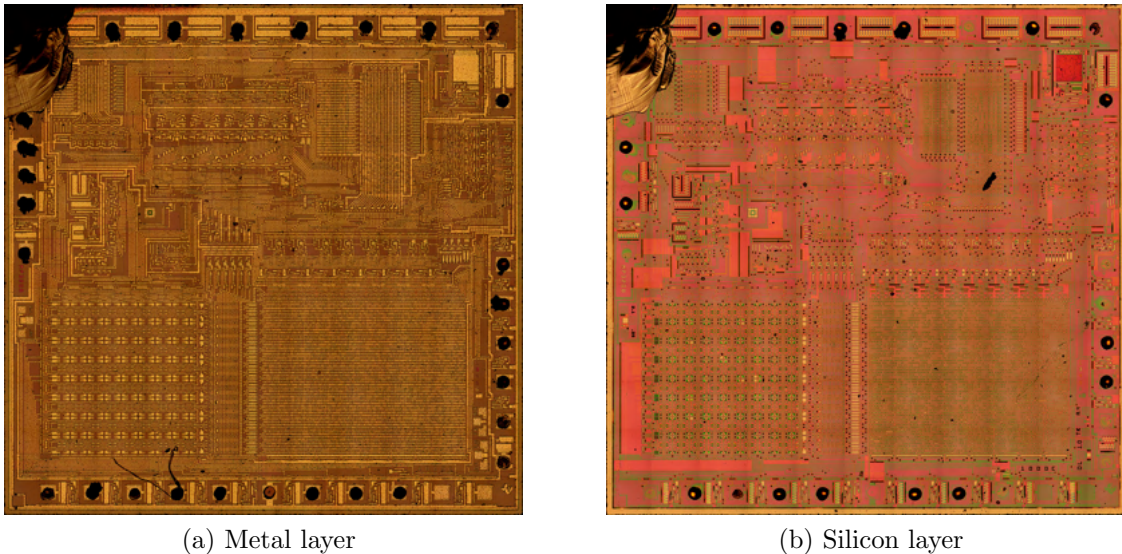


Figure 1.15: The two layers of a TMS 1000 microcontroller integrated circuit [25]

MOSFETs are made by implanting small regions of p-type silicon into a larger n-type silicon bulk. There is nothing preventing us from embedding multiple MOSFETs in the same piece of n-type silicon, as they are self-isolating—the p-n junctions separating source and drain regions from the n-type bulk also separate them from other source and drain regions. Furthermore, MOSFETs are very small, consume very little power, and can serve as switches, amplifiers, capacitors and resistors. The multifaceted functionality of MOSFETs means that we can make many complex and useful circuits using no other components.

These factors spurred the creation of the MOSFET **integrated circuit**. An integrated circuit is a circuit created out of a single piece of silicon. This is what is meant by the term ‘chip’ or ‘microchip’. Most, but not all, integrated circuits are created using MOSFETs.

It is integrated circuits, made possible by the humble MOSFET, that has made silicon a metonym for computers, and has spurred a half-century of electronic miniaturization.

1.6.1 Layers and Traces

Integrated circuits are naturally layered circuits: after all, each MOSFET consists of a metal gate layer and a silicon layer.¹³ Greater circuit density is achieved by creating circuits on both layers: one out of metal (the ‘gate layer’), and one out of lines of p-type silicon (the ‘channel layer’). Components called conduits or interconnects allow charge to travel from one layer to the other, and may be created simply by laying a metal region on top of a silicon region without an insulating oxide layer between.

1.6.2 Notation and terminology

Integrated circuits take MOSFET notation and apply it to larger structures.

The positive and negative terminals on a MOSFET are symbolized by S and D respectively. Thus, the most positive voltage source in an integrated circuit is labeled SS (the source of all sources), and the most negative voltage source in an integrated circuit is DD (the drain of all drains).¹⁴

Just as V_S is used as a ground in MOSFETs, V_{SS} is used as a ground in integrated circuits. Thus, V_{DD} is the largest-magnitude voltage available in an integrated circuit.

The ‘wires’ of an integrated circuit, made out of metal on the gate layer and p-type silicon on the channel layer, are called lines.

Of note, electrical components that are not part of an integrated circuit chip are often termed ‘discrete’ components, in contrast to ‘integrated’ components on a chip.

¹³Modern ICs can contain more layers, and do not use metal gates, but this is out of scope.

¹⁴Often V_{DD} is used to indicate both lines carrying the most negative voltage source, and the value of the voltage provided by them. We will use a less ambiguous convention where possible. Additionally, V_{DD} is often always the most positive voltage source, even in PMOS circuit diagrams, where it would in fact be the source voltage. Again, we will not use this convention for the sake of clarity.

Chapter 2

Digital Logic Design

2.1 Introduction

Digital logic circuits are a specific subset of circuits which are utilized in basically all general-purpose computers. They are called such because they implement a system of logic, using digital signals.

There are many different techniques for the creation of digital logic circuits, and the same circuit structures can be created many different ways. For instance, many early computers used vacuum tubes or relays to accomplish the same tasks that are handled by transistors today. We will focus on digital circuits created specifically out of p-channel MOSFET transistors, as are used in the TMS 1000 microcontroller.

Additionally, we will use the notation of integrated circuitry, such as V_{DD} and V_{SS} . If a circuit element is described as ‘on’, ‘active’, ‘charged’, or ‘powered’, this means that the voltage between that element and ground (SS) is V_{DD} , or close to V_{DD} . If a circuit element is described as ‘off’, ‘inactive’, ‘discharged’, or ‘unpowered’, this means that the voltage between that element and ground (SS) is 0, or close to 0.

2.2 Boolean Logic

Digital logic circuits implement a specific system of formal logic known as boolean logic. Essentially, it is a self-contained system that allows for many useful things to be done, similar to how numbers and mathematical operations are an abstract concept with many practical uses.

A boolean value, corresponding to the role that a ‘number’ plays in basic mathematics, can either be *True* or *False*, often represented as 1 and 0. We will adopt the more verbose convention for now. The concept of a variable is used in boolean logic as in algebra.

A boolean function takes as inputs one or more boolean values, or operands, and outputs a boolean value that is deterministically related to the input values. This may be equated to the role which functions such as ‘addition’ play in basic mathematics.

There are three basic boolean functions: NOT, AND, and OR.

Not, or the negation, is represented by $\neg$, and outputs the opposite value of its input.

AND is represented by $\wedge$. AND outputs *True* if all of its inputs are *True*, and *False* otherwise. Perhaps this may be thought of as the answer to the question “are all of the inputs *True*?”.

OR is represented by $\vee$, and outputs *True* if at least one of its inputs are *True*, and otherwise outputs *False*. It may be thought of as the answer to the question “is at least one of the inputs *True*?”.

NOT can only take one input, while all others can take any number of inputs.

These boolean functions can be combined in any manner, to create complex boolean expressions, or equations. Some of these combinations are common enough to warrant their own names. For example, NAND and NOR functions are negated AND and OR functions (for two operands, $\neg(A \wedge B)$ and $\neg(A \vee B)$). An XOR, or exclusive-OR gate, only outputs *True* if exactly one of its inputs are *True*; it is constructed by ANDing together an OR expression and the negation of an AND expression (for two operands, $(A \vee B) \wedge \neg(A \wedge B)$).

There are a number of different conventions for representing boolean expressions. The only variant that will be used in this thesis is that of representing negation by placing a line over the negated variable (so that $\overline{A}$ is equivalent to $\neg A$).

Function	Value of A	Value of B	Output
$\neg A$	<i>True</i>		<i>False</i>
$\neg A$	<i>False</i>		<i>True</i>
$A \wedge B$	<i>True</i>	<i>True</i>	<i>True</i>
$A \wedge B$	<i>True</i>	<i>False</i>	<i>False</i>
$A \wedge B$	<i>False</i>	<i>True</i>	<i>False</i>
$A \wedge B$	<i>False</i>	<i>False</i>	<i>False</i>
$A \vee B$	<i>True</i>	<i>True</i>	<i>True</i>
$A \vee B$	<i>True</i>	<i>False</i>	<i>True</i>
$A \vee B$	<i>False</i>	<i>True</i>	<i>True</i>
$A \vee B$	<i>False</i>	<i>False</i>	<i>False</i>

Table 2.1: Symbolic representation of the operation of the basic boolean functions

2.3 Logic Gates

The fundamental building block of a logic circuit is the **logic gate**. This is a physical component that represents a boolean logic function.

However, a fundamental problem of physical representation comes up. How can boolean values, of true and false, be represented electronically?

The physical world is **analog**, or continuous. Values in a circuit, such as voltage and current, exist on a continuum. However, logical operands must be **digital**, or discrete. We cannot have a value that’s 25% true and 75% false. Great care must

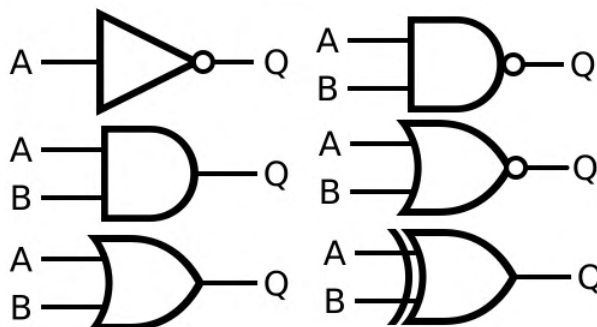


Figure 2.1: Circuit diagram symbols for logic gates. Left column, from the top: NOT, AND, and OR. Right column, from the top: NAND, NOR, and XOR. Note that the NAND and NOR gates have ‘inversion bubbles’ on their outputs, and are otherwise identical to the AND and OR gates

be taken to ensure that the electrical representations of boolean values never fall into these ambiguous, intermediate values, or all correctness is gone and random behavior may occur.

As MOSFETs are voltage-controlled devices, MOSFET logic gates use voltage to represent boolean values. The only way we can distinguish between voltages is whether a transistor activates with some V applied to the gate, or if it does not activate. This creates two voltage ranges, which can be assigned to logical values.

2.3.1 Logic conventions

The choice of which voltage range represents a logical True, and which represents False, is arbitrary. There are two natural conventions: the **positive logic convention**, where True is represented by a more positive voltage range than False (such that in PMOS circuits, *False* outputs activate transistors, and *True* outputs do not), and the **negative logic convention**, where True is represented by more negative voltages than False (such that in PMOS circuits, *True* outputs activate transistors, and *False* outputs do not).

The positive logic convention is far more widely used. However, it is generally desirable that the voltage representing *True* activates transistors, and PMOS circuits have a positive ground, making the negative logic convention more natural for PMOS logic circuits [14, 141-2].

For the TMS 1000, this is somewhat of an unsolvable dilemma. Being a PMOS circuit, the negative logic convention makes more sense for control circuitry. However, in order to properly interface with other digital logic components, which more commonly used the positive logic convention, the microcontroller’s outputs, and data lines which lead to outputs, had to use the positive logic convention. No matter which convention is adopted, some components will function counterintuitively.

Texas Instruments prioritized making data paths intuitive, so documents concerning the TMS 1000 uses a positive logic convention. As such, this and following chapters will adopt it as well.

This will make things confusing! Everything is backwards! Just remember that *False* activates transistors, and *True* does not. Additionally, the ground voltage, V_{SS} , is the most positive (‘high’) voltage in the circuit, despite it being 0—this is because all other voltages are negative.

2.3.2 Specific values

We want a *False* voltage signal to always activate transistors, and a *True* voltage signal to never activate transistors. However, as discussed in Chapter 1, MOSFET threshold voltages are not precise; additionally, MOSFETs with differing threshold voltages are often used on the same integrated circuit. Thus, in practice, the voltage levels of *True* and *False* are defined at the circuit element level, by four critical voltage thresholds:

V_{OL} : The voltage outputted by a gate, representing a *False* value.

V_{IL} : The most positive input voltage which is interpreted by a transistor or logic gate as a boolean *False* value.

V_{IH} : The most negative voltage which is interpreted by a transistor or logic gate as a boolean *True* value.

V_{OH} : The voltage outputted by a gate, representing a *True* value.

Voltages between V_{DD} and V_{IL} are interpreted as *False*, and voltages between V_{IH} and 0 are interpreted as *True*.

In order to have proper functionality, a number of properties must be true.

- Outputted voltage signals must be read correctly, so V_{OL} must always be less than or equal to V_{IL} , and V_{OH} must always be greater than or equal to V_{IH} .
- Steady voltage signals between V_{IL} and V_{IH} must never occur, as they cannot be properly interpreted as *True* or *False*.
- Every logic element connected together must have similar critical voltage thresholds.
- We must maintain a healthy margin of error between the input and output values, so that temperature, electrical noise, and slightly off-standard voltages don’t cause malfunctions.

In order to ensure all of these properties, it is best practice to keep V_{OL} as close to V_{DD} as possible, and V_{OH} as close to V_{SS} as possible. This leaves a certain necessary ‘room for error’.

A common working definition is to say that any voltage between V_{DD} and $\frac{V_{DD}}{2}$ represents *False*, and any voltage between V_{SS} and $\frac{V_{DD}}{2}$ represents *True* [14, 141-6].

2.3.3 Notation

In order to eliminate ambiguity between logical 0 and a voltage of 0V, we will refer to voltages of 0V as V_{SS} , or ground, as much as possible. We will also frequently refer to the logical 0 as ‘logical 0’, to further remove ambiguity.

2.4 Clock Signals

We have reduced the analog spectrum of voltage values to a set of digital values. However, there is another analog component of circuits: timing. Often it is desirable to synchronize different circuit components, so that they activate simultaneously, but designing circuits to guarantee this is difficult. The time it takes for a line to become charged, or for a transistor to activate or deactivate, is very small but is variable, and small timing differences add up.

The solution is to use a clock generator. A clock generator is an electrical component that alternately outputs V_{DD} and V_{SS} at a fixed, consistent rate. Each of these cyclical outputs is called a **clock pulse** or clock tick. The rate at which the clock repeats its cycle is known as the **clock speed** or clock frequency. The duty cycle, or the part of the cycle where the clock outputs V_{DD} out of the total length of the cycle, is 50% in most digital circuits.

We will refer to any voltage signal which is directly or indirectly created by a clock as a timing signal. Clock pulses can be used as timing signals directly. However, more complex timing arrangements can also be made; for instance, clock pulses could be ‘counted’, so that one timing line is powered on even pulses, and the other on odd pulses, for example. We will designate timing signals, and the corresponding lines which carry the timing signals, with the symbol ϕ . The time it takes for a timing signal cycle to repeat may be termed the **clock cycle** or **machine cycle**, although the latter term may be used to refer to other operating cycles as well.

By placing an enhancement-mode transistor controlled by a timing signal in the middle of a line, we can guarantee that charge (and thus a signal) can only pass through the line at specific times. When a circuit element is arranged as such, it is said to be ‘clocked’ to the timing line. By determining the timing when voltage signals pass through lines, we can determine the order when components are activated, or ensure that components are activated simultaneously. Thus, the continuous spectrum of timing values is reduced to a set of digital values [12, 114-5].

2.5 A simple NOT gate

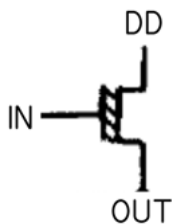


Figure 2.2: NOT gate symbol

The simplest logic gate is the NOT gate, which implements the NOT expression. It takes an input on one line, and outputs the opposite signal on another. As converting one signal to a different signal is fundamental to the operation of all logic gates, similar structures will appear in all of them.

We will label the input line of the gate IN and the output line OUT. We will assume that the OUT line terminates in the gate of a MOSFET.

2.5.1 Naive Implementation



Perhaps the simplest solution that may come to mind is to use a single depletion-mode MOSFET. When IN is unpowered, *DD* is connected to OUT, sending *False* to OUT. When IN is powered with a positive voltage relative to the body, *DD* is disconnected from OUT.

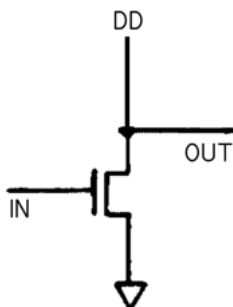
There are two major problems with this.

Firstly, disconnecting OUT and *DD* does not immediately discharge OUT. Since MOSFET gates are capacitors, this means that the transistor which the OUT line is connected to will remain open for an indeterminate time. OUT is left electrically isolated, rather than connected to ground.

Secondly, as the ground in a PMOS circuit is the most positive voltage source in the circuit, it is impossible to power IN with a voltage more positive than 0. In order for a transistor to be cutoff, or closed, it must be that $V_{GS} - V_T > 0$, which in this case means that $0 - V_T > 0$. As a depletion-mode MOSFET's threshold voltage is positive by definition, this is impossible.

Thus, two further constraints are introduced: the NOT gate must be activated with an enhancement-mode transistor, and OUT must never be fully disconnected from the rest of the circuit.

2.5.2 Second Implementation



Instead of disconnecting OUT and *DD*, we can try setting it up so that applying voltage to IN connects OUT and ground, using an enhancement-mode resistor.

However, there is no way to disconnect OUT and *DD*, and as they are shorted (directly connected with a low-resistance conductor), V_{out} will always equal V_{DD} . We

can verify this by noting that by Ohm's law, the voltage difference between two regions is equivalent to the current times the resistance. As there is no resistance between DD and OUT, this term is equal to 0, and there can be no difference between V_{DD} and V_{out} .

2.5.3 Third Implementation

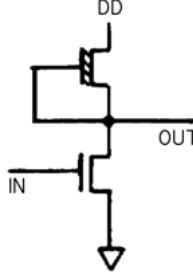


Figure 2.3: NOT gate circuit

Recall that MOSFETs can behave similarly to resistors. By placing a MOSFET in between OUT and V_{DD} , a voltage between DD and OUT can be created. Finally, we have a working NOT gate design. In short order, we will attempt to calculate what the value of the output voltages will be.

This form of gate is called a depletion-load logic gate, as a depletion-mode MOSFET is used to create a load. As such, the depletion-mode MOSFET between OUT and DD is called the load transistor, and will be represented by the symbol l (e.g. V_{Tl} is the threshold voltage of l). The enhancement-mode MOSFET connected to IN is called the driver, and is called such because it controls the circuit's connection to ground, and thus its output voltage. The driver transistor will be represented by the symbol d [5, 108].

As noted, in an integrated PMOS circuit like this one, no depletion-mode MOSFET can be closed. Therefore, the load transistor is always open.

Operating Characteristics

Suppose that $V_{in} = 0$. That is to say, $V_{Gd} = 0$, and the input to the gate is a logical *True*. In this case, as the driver transistor is an enhancement-mode transistor, we know that it is in the cutoff region, and negligible current flows through it. As it is in series with the load transistor, we know by Kirchhoff's current law that the current flowing through the load transistor is also 0. Then, from the MOSFET current equation, we can calculate the following for the load transistor:

$$0 = \beta[(V_{GS} - V_{Tl})V_{DS} - \frac{V_{DS}^2}{2}]$$

We know that β is a nonzero constant, and that $(V_{GS} - V_{Tl})$ cannot be zero in a depletion load transistor. From this we know $(V_{GS} - V_{Tl})V_{DS} - \frac{V_{DS}^2}{2}$ isn't zero, as that would require $V_{DS} \geq (V_{GS} - V_{Tl})$, the load transistor would be in saturation mode,

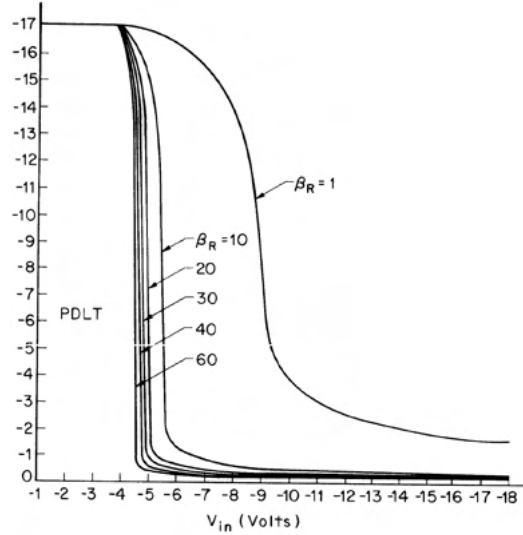


Figure 2.4: Depletion-load NOT gate voltage characteristics [5, 112]

and current would be $(V_{GS} - V_T)^2$, which is nonzero. Therefore, the only solution is that V_{DS} must be zero.

As the drain is connected to DD , and the source is connected to V_{out} , $V_{DS} = V_{DD} - V_{out}$, and as $V_{DS} = 0$, thereby $V_{DD} = V_{out}$. Thus, when the driver transistor is in the cutoff mode, V_{out} is V_{DD} .

We have now shown that the NOT gate will output a *False* voltage signal if the input is a *True* voltage signal, that $V_{OL} = V_{DD}$, and that V_{IH} is at least as negative as the threshold voltage of the driver transistor.

Proving that our NOT gate properly inverts *False* signals is more involved. We will first set up some tools to help us.

If $V_{in} = 0$ (the input is a logical *False*), the driver transistor will be open, and by Kirchhoff's current law, the same amount of current is flowing through the load and driver transistors. This means we can set up an equality:

$$I_d = I_l$$

This simple equality will be our main tool. Additionally, we will find of use a number of known quantities and equalities: we know that the load transistor's drain is DD , the driver transistor's source is ground, and that the load transistor's gate and source, and the driver transistor's drain, are all V_{out} . The load transistor's gate and source are the same voltage; so $V_{GS} = 0$.

Additionally, we can assume that the driver transistor is active, with a *False* voltage at its gate, and that current may pass through it.

Let's start by looking at what would occur if the input quickly switched from *True* to *False*, and V_{out} was equal to V_{DD} , or a value close to it. In that case, V_{out} will be more negative than $V_{in} - V_{Td}$, and the driver transistor will be in the saturation region, with a current of $\frac{\beta_d}{2}(V_{in} - V_{Td})^2$. An equivalent current must pass through the load transistor. By Ohm's law, the voltage drop across the load

transistor is equivalent to the current passing through it times the resistance, so $V_{DD} - V_{out} = \frac{\beta_d}{2}(V_{in} - V_{Td})^2 \cdot \frac{1}{k[(-V_{Tl}) - \frac{V_{DD} - V_{out}}{2}]}$. Thus, unless V_{Tl} is extraordinarily large (and we're designing the circuit, so we can say it's not), V_{out} will quickly become more positive, and the load transistor will be pushed towards the saturation mode.

We can choose threshold voltages so that around the time when the load transistor enters the saturation mode (when $V_{DD} - V_{out}$ becomes more negative than $-V_{Tl}$), the driver transistor will leave the saturation mode and enter the linear mode. Now we can again use our current equality to set up the equation:

$$\beta_d[(V_{in} - V_{Td})(V_{out}) - \frac{(V_{out})^2}{2}] = \frac{\beta_l}{2}(-V_{Tl})^2$$

This is a quadratic equation, and can be solved with:

$$V_{out} = V_{in} - V_{Td} \pm \sqrt{(V_{in} - V_{Td})^2 - \frac{\beta_l}{\beta_d}(V_{Tl})^2}$$

The positive solution is spurious, as it would result in V_{out} being more negative than $V_{in} - V_{Td}$, putting the driver transistor in the saturation mode. This leaves one possible value for V_{out} . The terms $V_{in} - V_{Td}$ and $\sqrt{(V_{in} - V_{Td})^2 - \frac{\beta_l}{\beta_d}(V_{Tl})^2}$ almost but don't quite cancel each other out, meaning that V_{out} (and following from that V_{OH}) is almost but not quite zero volts.¹

The exact values of V_{IL} and V_{IH} are calculated according to the derivative of the voltage transfer characteristics plot. As such, they are related to derivatives of MOSFET current equations [14, 161-7].

$$V_{IL} = V_{Td} + \frac{\beta_l}{\beta_d}(V_{out} - V_{DD} + V_{Tl})$$

$$V_{IH} = V_{Td} + 2V_{out} + \frac{\beta_l}{\beta_d}[-V_{Tl} \cdot (\frac{dV_{Tl}}{dV_{out}})]$$

As noted, however, circuits should not be designed to be reliant on specific V_{IL} and V_{IH} values, so that there is a margin of safety in operation.

2.6 NOT gate variants

Several minor variants of the NOT gate are used in the TMS 1000.

¹Of note, we have ignored the effect that a nonzero source voltage has on the threshold voltages of the transistors. In practice, to account for that, V_{out} is solved for iteratively: first we find the V_{out} value, pretending that the source voltage is zero. Then we go back and solve for the effective threshold voltage resulting from that V_{out} value. With the new threshold value, we re-solve for V_{out} and go back and forth until the values converge. The result will be close to but not quite the same value as our original result [14, 163]. For good reason, the output-low voltage has been described as the "most significant design constraint" of depletion-load logic gates [14, 166].

2.6.1 Buffer

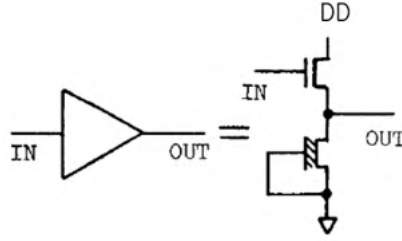


Figure 2.5: Buffer circuit diagram

In the buffer, or reverse NOT gate, the positions of the load transistor and the driver transistor are switched. This flips the operational characteristics. When V_{in} is *False* (between V_{IL} and V_{DD}), V_{out} is equal to V_{SS} , or 0. When V_{in} is *True* (between 0 and V_{IH}), V_{out} is equal to $V_{DD} - V_{OH}$, which is slightly below V_{DD} , as V_{OH} is small but not 0.

Buffer gates allow circuits to utilize secondary characteristics of logic gates, such as input-output isolation, voltage amplification, and gate capacitance, without changing the logic level.

2.6.2 Clocked NOT Gate 1

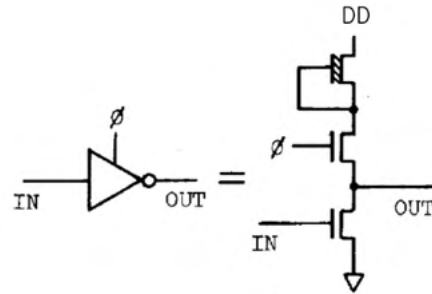


Figure 2.6: Clocked NOT gate 1

An additional transistor is placed between the output line and the load transistor, connected to a timing signal line ϕ . When ϕ is powered with V_{DD} , the transistor behaves very similarly to a normal NOT gate.² When ϕ is unpowered (with a voltage close to zero, in the *True* range), OUT is disconnected from DD . If V_{in} is *False*, $V_{out} = 0$, but if V_{in} is *True*, OUT is entirely disconnected from any voltage source.³

²When V_{in} is *True*, $V_{out} = V_{DD}$ by the same calculations as before. When V_{in} is *False*, $|V_{out}|$ will be as small or smaller than in a normal NOT gate, as the timing transistor will only add additional resistance between OUT and V_{DD} .

³Similarly to the normal NOT gate V_{DD} calculation, the current going through the driver transistor must be zero since there is no way for it to complete a circuit. Therefore, $0 = \beta[(V_{in} - V_{Td})V_{out} - \frac{V_{out}^2}{2}]$, and $V_{out} = 0$.

Because OUT is connected to a MOS capacitor, for a time it will hold its previous charge, from when it was connected to a voltage source. Thus, if V_{in} became *True* before ϕ became low, V_{out} will be equal to 0, and if the reverse, it will be equal to V_{DD} .

There are several benefits to this behavior. Of note, V_{out} can reach the ideal *True* value of 0, which is not possible in a normal NOT gate. Additionally, power consumption is lower; in a normal NOT gate, whenever V_{in} is *False*, a substantial current flows from DD to the ground, wasting power, while in the clocked NOT gate, this current only flows the portion of the time that ϕ is active.

However, timing constraints are introduced. Due to the MOSFET subthreshold current, V_{out} will slowly discharge when it is disconnected. Thus, if ϕ spends too long unpowered, V_{out} will become *True*, even if the input value is also *True*. This means that circuits using clocked depletion-load logic gates have a minimum clock speed! Additionally, if IN is *False* and switches to *True* while ϕ is unpowered, the correct output value of *False* will not be available until ϕ is powered again, creating another timing constraint.

2.6.3 Clocked NOT Gate 2

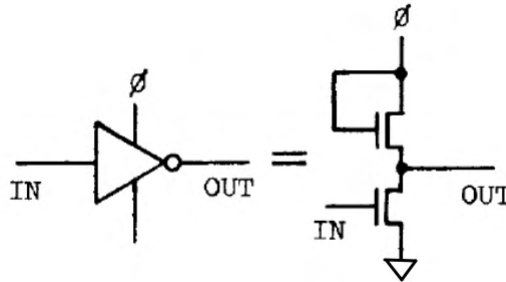


Figure 2.7: Clocked NOT gate 2

By using a timing line to deliver voltage, instead of connecting to DD directly, the NOT gate can be clocked with one less transistor.

Unlike previous inverters, the second clocked NOT gate uses an enhancement-mode transistor as the load (or an enhancement-load). Enhancement-loads require a voltage drop, have higher effective resistance than depletion-loads, and are for this reason dispreferred. In this instance, the reason for using an enhancement is so that when ϕ is unpowered, the load transistor closes, and does not drain charge from OUT.

The second clocked NOT gate shares all of its logical characteristics, and almost all of its functional characteristics with the first clocked NOT gate. When IN is *False* and ϕ is V_{DD} , OUT is connected with the ground, and it operates as a normal NOT gate. When ϕ is unpowered, OUT is connected to ground if V_{in} is *False*, or is entirely disconnected if V_{in} is *True*. The difference is that when IN is *True*, and ϕ is V_{DD} ,

then $V_{out} = V_{DD} - V_{Tl}$ instead of V_{DD} , as the enhancement load becomes saturated when $V_{DS} = V_{GS} - V_T$. It will accordingly also take longer to charge the OUT line, when the input switches from *False* to *True* [5, 110-3].

2.7 Other Logic Gates

Other logic gates can be constructed through modifying the driver portion of the NOT gate. While it may be more straightforward to construct them out of buffer-derived gates, and create AND and OR gates instead of NAND and NOR gates, this is power-inefficient, due to the voltage drop of buffers. Additionally, NAND and NOR gates are functionally complete: any possible logic function can be constructed out of NAND or NOR gates, which is not true for AND and OR gates.

2.7.1 NOR gate

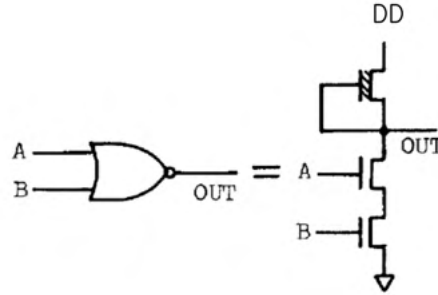


Figure 2.8: NOR gate

The NOR gate works as a NOT gate with the driver transistor replaced by a single path to ground and two driver transistors. When either or both of the *A* or the *B* input lines are *True*, the circuit reduces to a NOT gate with a *True* input, as there is no path to ground. When both the *A* and *B* input lines are *False*, the circuit functions similarly to a NOT gate with *False* input. Using *A* and *B* as identifiers for their respective driver transistors, the current equation is:

$$\beta_A[(V_{in_A} - V_{T_A})V_{out} - \frac{V_{out}^2}{2}] = \beta_B[(V_{in_B} - V_{T_B})V_{out} - \frac{V_{out}^2}{2}] = \frac{\beta_l}{2}(-V_{Tl})^2$$

If β_A is equivalent to β_B , V_{in_A} is equivalent to V_{in_B} , and V_{T_A} is equivalent to V_{T_B} , it completely reduces to a NOT gate with *False* input.

Thus, the NOR gate outputs *False* if either *A* or *B* are *True*, and *True* if both are *False*.

Of note, the positive-logic convention NOR gate is the negative-logic convention NAND gate (and vice versa).

2.7.2 NAND gate

The NOR gate works as a NOT gate with the driver transistor replaced by two paths to ground and two driver transistors. When the *A* and *B* input lines are both *True*,

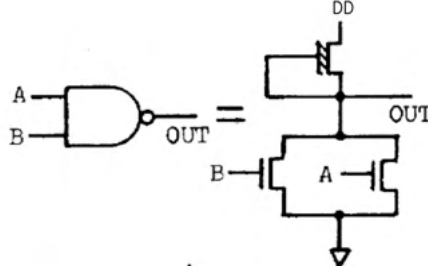


Figure 2.9: NAND gate

the circuit reduces to a NOT gate with a *True* input. When one of the input lines are *False*, the circuit reduces to an NOT gate with a *False* input. When both input lines are *True*, the circuit functions similarly to a NOT gate with a *True* input, with a current equation of:

$$\beta_A[(V_{in_A} - V_{T,A})V_{out} - \frac{V_{out}^2}{2}] + \beta_B[(V_{in_B} - V_{T,B})V_{out} - \frac{V_{out}^2}{2}] = \frac{\beta_I}{2}(-V_{Tl})^2$$

If V_{in_A} is equivalent to V_{in_B} , and $V_{T,A}$ is equivalent to $V_{T,B}$, it completely reduces to a NOT gate with *False* input, and a driver transistor constant of $\beta_d = \beta_A + \beta_B$.

Thus, the NAND gate outputs *False* if both *A* and *B* are *True*, and *True* if either or both *A* and *B* are *False*.

Of note, the positive-logic convention NAND gate is the negative-logic convention NOR gate (and vice versa).

2.7.3 Clocking and NAND/NOR gates

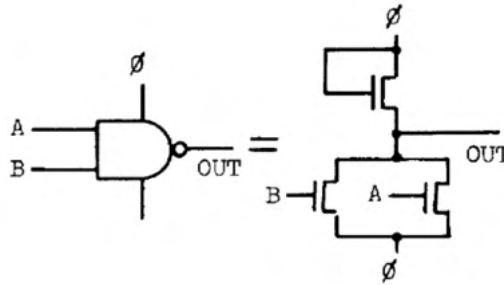


Figure 2.10: Clocked NAND gate

The NOT gate variants we discussed can also be modified into NAND and NOR gates. For instance, by replacing the driver transistor of a clocked NOT gate with two lines to ground with two driver transistors, a clocked NOR gate can be created, as seen above.

By replacing the driver transistor in the buffer gate, non-negated gates, such as AND and OR gates, may be created. However, as noted earlier, these result in a voltage drop on *False* outputs, and NAND and NOR gates are more capable, and thus are usually preferred.

2.7.4 Combinations

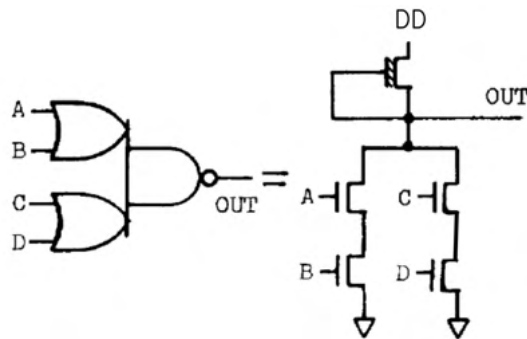


Figure 2.11: Complex Gate

NAND structures and NOR structures can be combined to form more complex gates, in a smaller footprint and at a greater power efficiency than constructing them out of discrete gates. In this construct, the boolean expression $\overline{(A \vee B) \wedge (B \vee D)}$ is implemented.

2.7.5 Loads

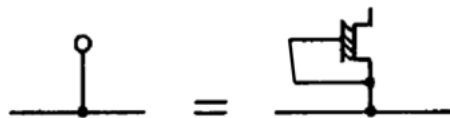


Figure 2.12: Depletion load

By connecting a line to DD through a load resistor, it can be ‘pulled down’. Like in a NOT gate, the line will be charged to V_{DD} if no other voltage source is connected, but will be discharged to V_{OH} if a connection to ground is made.

2.8 Clocks

How can we generate a clock signal? The simplest, most naive solution might be to take a NOT gate and connect its output back to its input, so that it will be constantly forced to switch the signal that it is outputting.

This actually works! However, using just a single NOT gate will result in an unstable signal, as the switching speed of the NOT gate will not be perfectly regular. Even worse, the inputs and outputs lines may balance out in voltage, so that the

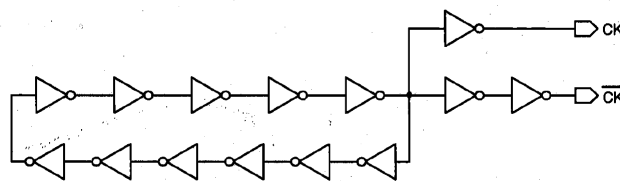


Figure 2.13: Ring Oscillator [14, 550]

NOT gate outputs a consistent, invalid signal between V_{IL} and V_{IH} , instead of an alternating system.

These problems can be mitigated by simply putting more NOT gates in a loop. Each NOT gate does not generate its own input voltage, so it cannot stabilize itself. As the switching speed of the clock is now dependent on the switching speeds of many NOT gates, it is slower but less variable. Such a design is called a ring oscillator.

The limitations of the ring oscillator is that there is still a good degree of variability in the clock speed, and that increasing the speed by using fewer NOT gates also increases the variability. For this reason, high performance digital circuits usually use specialized external piezoelectric components to generate the signal. For low-end digital circuits, such as most applications of the TMS 1000, the economy of using an on-chip ring oscillator outweighs the downsides [14, 549-50].

2.9 Programmable Logic Arrays

2.9.1 Logical Properties

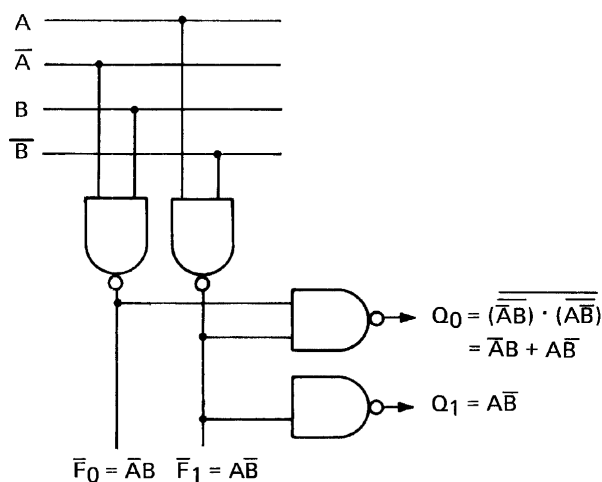


Figure 2.14: PLA logic [29, 2-18]

Suppose we wanted to make a really, really big logic gate, with many inputs and outputs. This could be done using the gate structures we've discussed, but it may

be done in a much more compact manner using a structure called a **programmable logic array**, commonly abbreviated to PLA.

Logically, a programmable logic array consists of two matrices of NAND gates. Each input is split into two lines, consisting of one ‘true’ and one ‘inverted’ line. Any selection of the input lines can be connected together in a NAND expression. The outputs of each of these NAND gates become the inputs for the second matrix. Any selection of these second-matrix input lines may be NANDed together as well. The outputs of these NAND gates become those of the programmable logic array.

Although only NAND gates are used, the programmable logic array behaves like it was constructed out of AND and OR gates. This is because the NAND gates ‘cancel out’. For example, the expression implemented in the figure, $\overline{(A \wedge B)} \wedge \overline{(C \wedge D)}$ is true if either $\overline{A \wedge B}$ is false, or $\overline{C \wedge D}$ is false. So it is true, effectively, if either the boolean expression $A \wedge B$ or the expression $C \wedge D$ is *true* ($(A \wedge B) \vee (C \wedge D)$). Because of this, the first matrix’s terms are in practice ANDed together, and the second matrix’s terms are ORed together. As such, they are termed the ‘AND matrix’ and the ‘OR matrix’ respectively [29, 2-17, 2-18].

This setup allows for great versatility. For instance, if some of the input lines don’t matter for a specific output, they may just be excluded from being NANDed together on the AND matrix, creating a minimal expression. As an alternative metric, the expressions depicted on the diagram would require many gates to implement the logic discretely, but can be implemented with just one PLA.

2.9.2 Circuit

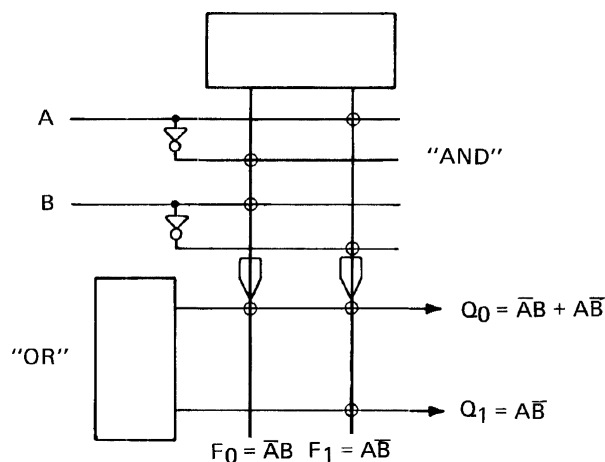


Figure 2.15: PLA circuit [29, 2-18]

Both PLA matrices are constructed the same way, with a grid (matrix) of input lines and output lines. The input lines are constructed out of metal. The output lines are lines of p-type silicon, and are connected on one side to a V_{DD} voltage source, through a load transistor, as is the case in logic gates. Between each of the output lines is an additional line of p-type silicon, connected to a V_{SS} voltage source.

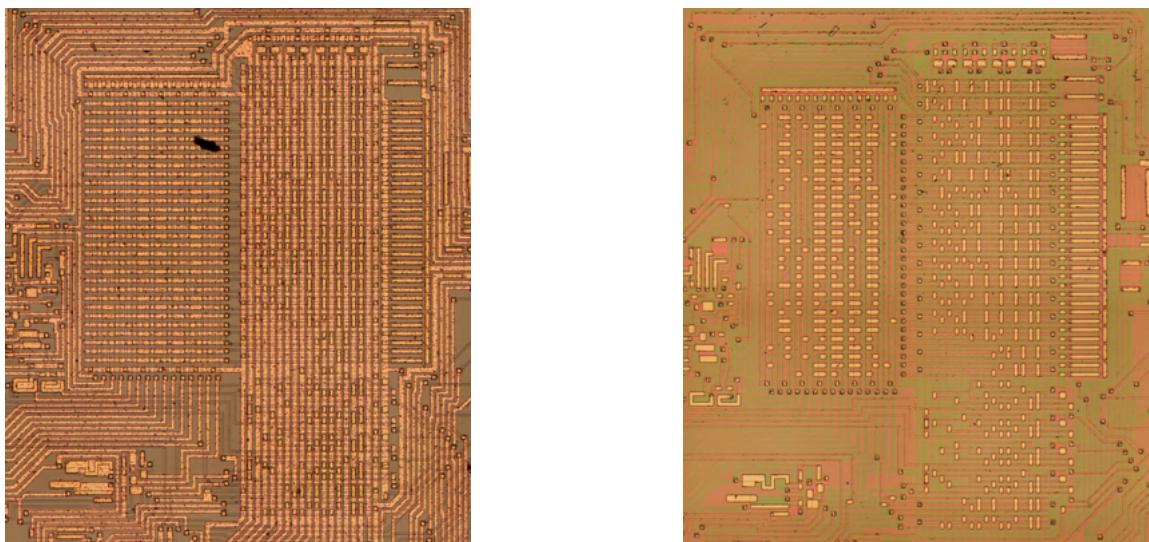


Figure 2.16: The metal and silicon layers of a TMS 1000 microcontroller's instruction decode PLA [25]

On most of the matrix, a thick oxide layer separates the metal input lines from the silicon, so they do not behave as MOSFET gates. However, where a NAND term is to be created, a thin oxide layer is laid down instead, in between an output line and the adjacent V_{SS} line. In the above diagram, the circles represent such thin oxide layers. This creates a MOSFET transistor between the output line and the V_{SS} line, with the gate driven by the input line.

If even one of these thin oxide areas are activated, the output line connected by it goes to V_{SS} . Otherwise, the input line remains at V_{DD} . As *False* voltages activate transistors, this means that the input line will be $V_{DD}/False$ if all of its connected input lines are *True*, and $V_{SS}/True$ otherwise. This is a NAND expression!

In between the AND and OR arrays, a conduit spans the layers and connects the p-channel output lines of the first array with the metal input lines of the OR array. A buffer is used to ensure that the voltage remains high enough to actuate the OR array logic.

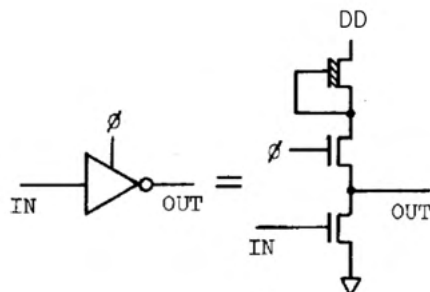
2.9.3 Read-Only Memory

The basic structure of a PLA can also be used as read-only memory. If every input bit is used in every output line of a PLA's AND matrix, it essentially functions as a dictionary, or lookup table. For every possible combination of values on the input line, exactly one AND matrix output line is activated, which outputs some combination of values on the output line. In this use-case, the values on the input line are the 'address', and the values outputted by the PLA with a specific address inputted is referred to as the data stored at that address.

As the values stored in each address can only be read, and cannot be changed after the array is manufactured, this type of data storage is called read-only memory, or ROM. The specific PLA-like structure described is called a masked ROM.

2.10 Storing Values

2.10.1 Precharging



A common integrated circuit technique is precharging. We discussed this concept, but did not name it, in the sections relating to clocked NOT gates. We will bring back the clocked NOT gate for a demonstration.

Suppose that there are two timing signals, $\phi 1$ and $\phi 2$, which do not overlap, and we know that $\phi 2$ occurs after $\phi 1$. The clocked NOT gate is clocked on $\phi 1$.

Now suppose that we know that when $\phi 1$ is powered, IN is unpowered (V_{SS} or *True*). Perhaps the circuit generating IN is clocked so it is only active after $\phi 1$. Then when $\phi 1$ is powered, OUT will be charged to V_{DD} . At the end of $\phi 1$, $\phi 1$ turns off; V_{DD} is disconnected from OUT; OUT remains charged at V_{DD} .

We can now say that OUT is **precharged**. At this point, if IN becomes powered, OUT will be discharged, but if this does not happen, OUT will stay at V_{DD} .

We can only do this if the value of OUT is not important on $\phi 1$, since otherwise, our gate would cause problems by always outputting 0 on $\phi 1$.

Such a setup allows for savings in power, as no current is drawn by an output of *True*. It additionally allows us to guarantee that OUT will be fully charged or discharged by $\phi 2$, even if there is resistance on OUT; allows us to set up complex timing relationships; and finally, it is the base of MOSFET memory storage.

2.10.2 Shift Registers

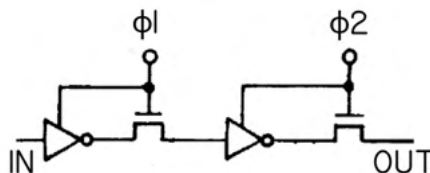


Figure 2.17: Basic shift register

A shift register allows a value to be stored for one clock cycle. It is called such because it ‘shifts’ the value on the input line to the output line with a time delay.

Lines $\phi 1$ and $\phi 2$ are timing lines, set up such that they never are both powered at the same time. On $\phi 1$, the first NOT gate acts like a normal NOT gate, as discussed in the relevant section. Additionally, the transistor following the first NOT gate is opened, connecting the output of the first NOT gate to the input of the second NOT gate. The input of the second NOT gate is thus charged with the inverse of the voltage signal received from IN. To put it another way: the gate is precharged.

When $\phi 1$ turns off, the second NOT gate's gate is electrically disconnected from the first NOT gate's output. The voltage signal at IN may now change, with no effect on the charge stored in the gate of the second NOT gate.

When $\phi 2$ turns on, the output of the second NOT gate is connected with OUT. This will be the inverse of the inverse of the voltage signal received on IN during $\phi 1$, and thus will be the original voltage signal. The boolean value represented by the inputted voltage signal is thereby stored and outputted.

But what if we wanted to store a value indefinitely? The naive solution is to just connect OUT back to IN by a feedback line. This completely works.

2.10.3 Recirculating Shift Registers

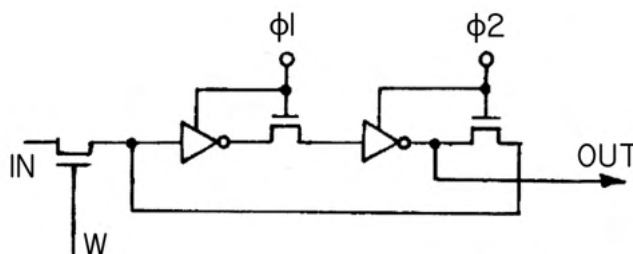


Figure 2.18: Recirculating shift register

By adding a feedback line, a voltage signal (and the corresponding boolean value) can be stored indefinitely. This is called a recirculating shift register, or a 'latch'.

The basic operation is the same: on $\phi 1$, the input NOT gate charges the output NOT gate's latch, setting the output value. On $\phi 2$, the output voltage signal can charge the input NOT gate, setting the input value to itself. In this manner, the value inputted to the latch is stored indefinitely.

Recirculation creates a new timing constraint, not present in the basic shift register. If we want to store a new value in the latch, that is different from the value already stored, we can connect the IN line to a *True* or *False* voltage source, setting the stored value to true or false.

If we set the IN line when $\phi 2$ is active, IN is connected to two differing voltage sources, and what voltage it ends up being charged to is anybody's game. Additionally, the IN voltage might charge or discharge the OUT line if this were to happen.

If we set the IN line between $\phi 1$ and $\phi 2$, precharging the input NOT gate, nothing so catastrophic happens, but when $\phi 2$ turns on, voltage from the output NOT gate

will quickly displace the stored charge. In other words, the old value will still be stored in the latch. In computer science parlance, the write will fail.

Therefore, in order to store a new value in the latch, we must charge or precharge line IN after $\phi 2$'s activation and before or during $\phi 1$'s activation. During that time window, a voltage source applied to IN will overpower the charge stored in the input NOT gate's input by the feedback line.

An additional transistor, W, may facilitate timing. By only activating W after $\phi 2$'s activation, and by only activating W when we wish to store a new value in the latch, we can ensure that invalid timing does not occur, and we can control when and under what circumstances we want to overwrite the value stored in the latch.

The feedback circuit may be called a 'refresh' circuit, as it refreshes the charge stored in the latch.

2.11 Representing Numbers

Now we have found ways to read, store, and perform logical operations on *True* and *False* boolean values. But how can we represent numbers, and perform arithmetic operations on them?

As it turns out, we already have the solution. By using the boolean *True* value to represent the number 1, and *False* to represent the number 0, any number may be represented using a series of boolean values. However, as we only have two numbers to work with, we must represent numbers in **binary**, or base two.

In the base 10, or decimal system, there are ten numbers, 0 through 9. For numbers larger than 9, an extra digit is added. Such is the case with binary, except we only have 0 through 1! Just as every digit in a 'normal' base 10 number represents a power of 10 (with the ones, tens, hundreds, and thousands places), every digit in a binary number represents a power of two (with the ones, twos, fours, eights, and sixteens places).

Suppose we have four lines, which can be *True* and *False*. Using binary, we can represent any number up to 15 on them. The series of voltages *False*, *True*, *True*, *False* can be interpreted as the binary number 0110, equivalent to the decimal number 6. Using the logic gates we have built up, we can 'decode' binary numbers. For instance, a common use for programmable logic arrays is to display numbers on a seven-segment display, such as are found on calculators. The bottommost line on such a display should be illuminated for the numbers 0, 2, 3, 5, and 8, but not for other numbers. A logical expression can be created on a PLA to recognize such numbers and only power the bottom display line in such cases.

For this reason, *True* and *False* are often referred to and represented as a logical 1 and a logical 0. This is the convention that we will adopt after this chapter.

An individual boolean/binary voltage signal, representing a 0 or 1, is what is meant by the term 'bit'. We may say that in our example, 0110 is a four-bit number, as it is represented using four bits.

2.11.1 Hexadecimal

Binary numbers get long quickly. For instance, 65, which requires two digits in decimal, is written in binary as 1000001—seven digits (bits) long! Binary is well-suited for use by computers, but is rather difficult to work with as a human being.

However, decimal does not convert to binary easily. It requires some calculations to even just figure out how long a given number would be in binary.

A solution to this is hexadecimal: base 16. In hexadecimal, we have all of the decimal numbers, and in addition A through F, which represent the decimal values 10 through 15. Binary maps cleanly onto hexadecimal: one hexadecimal digit represents four binary digits (bits).

To go back to our example, 65 is represented in hexadecimal as 41. Because this is in hexadecimal, we know that this translates to the binary representation of 4 (0010) followed by the binary representation of 1 (0001), converting to 1000001.

Hex numbers are often prefixed by the characters ‘0x’, so that they can be clearly differentiated from decimal.

Chapter 3

TMS 1000 System Architecture

3.1 Introduction

We have so far been building up the TMS 1000 ‘from the ground up’, starting from how electricity works. In the next chapter, we will be reverse-engineering various components of the TMS 1000. However, first we will take a ‘birds-eye view’ of the TMS 1000 and examine what the components are supposed to do and how they connect with each other, so that we know what to expect.

3.2 What is a computer?

3.2.1 Single-Purpose Computers

Over the last two centuries, machines have been constructed to perform an enormous variety of tasks at great efficiency. We have created goods-producing machines, transportation machines, machines for creating sound and light, machines for performing household tasks, and more. A computer is simply one variety: a machine created to perform a mathematical or logical task.

Early computers were single-purpose devices. An archetypal example of a very simple computer is the slide rule, which automates the task of performing a number of mathematical operations. More complicated machines were used to do tasks such as solving differential equations, predicting low and high tides, or calculating the trajectory of a missile.

Notice that all of these use-cases are of a form: they are problems that have deterministic, correct solutions. By the 1930s, engineering had progressed to such a point that it was feasible to create a single machine which could be used to solve all problems of this type. This is known as a general-purpose computer.

3.2.2 General-Purpose Computers

The credit of who created the first general-purpose computer is disputed by many. The concept was developed independently several times. The first commercially-available

general-purpose computer was the UNIVAC, first brought to market in 1951.

To reiterate, general-purpose computers are designed to be able to solve any problem with solution which can be arrived at deterministically. This property is formalized in the Church-Turing thesis, which defines the set of problems that can be solved by a general-purpose computer [20, 3-19][22, 21-35].¹

From this definition, it must be possible to specify to the computer the steps required to solve any such problem. This is, at base, the concept of programming. The programmer enters **instructions**, which specify the actions that the computer must perform, with strictly no ambiguity. Then, the computer deterministically executes these instructions. A sequence of instructions to is a ‘program’. The set of instructions which can be used to program a computer is called its instruction set.

3.2.3 A Minimal Computer

A number of functional characteristics are so useful and have been so widely adopted that we may consider them essential parts of a computer.

First is program storage. When the computer is programmed, the instructions for it to execute must be stored somewhere, where they can be executed any number of times.

The next is that the computer must have a ‘working memory’. When a program is executing, it must be able to store and later retrieve data that it generated in the course of the program.

The third is I/O, or input/output. There must be some way to enter data into the computer. Additionally, there must be some way to access and read data from a computer.

3.2.4 Terminology

Address The location at which data should be read from or stored in a memory array.

Array A grid of smaller components.

Opcode The sequence of bits that represents an instruction. Often written in hexadecimal.

Read/Write The terms ‘read’ and ‘write’ are frequently used to refer to the abstractions of accessing and storing data.

Register A type of working memory component, that can be used to store data. Usually small and for a specific purpose.

Word A unit of data. When a component is designed specifically to handle a specific-size unit of data (such as a register that holds four bits), that is said to be that component’s word size or word length.

¹This is a simplification, but a more full explanation would require a significant divergence that would be out of scope of the aims of this thesis.

excessive, and not provide much utility.

3.3.1 Program Storage

The program is stored in a masked ROM, or Read-Only Memory. Masked ROMs were discussed briefly last chapter: they use a PLA-like structure to decode an inputted ‘address’ into a predefined value. This means that a TMS 1000 can only be programmed once, at the factory. After that, the program is unchangeable.

In order to execute a program, a computer must retrieve the instructions which constitute the program one at a time, in the correct order. Thus, the computer must be able to keep track of where it is in the program. The TMS 1000 does this with the **program counter** and the **page address register**. These are two ‘working memory’ registers, the contents of which together make up a complete address, allowing the ROM array to output the preprogrammed instruction at that address. When an instruction is executed, the program counter increments, so that the next instruction will be outputted on the next cycle.

Each TMS 1000 instruction is 8 bits long. As this is the smallest unit of data in the ROM array, 8-bits is considered its ‘word size’. The ROM array’s output is always an 8-bit instruction ‘word’.

The ROM array holds 1024 words, and is divided up into 16 ‘pages’ of 64 words each. The page is selected by the value in the page address register; the word that is outputted, on the current page, is selected by the program counter.

3.3.2 Working Memory

The primary location of working memory is the RAM array. The latch which we examined last chapter stores a bit by constantly recirculating or ‘refreshing’ it. The RAM array is essentially a massive number of recirculating storage registers, called cells, constructed with as few components as possible and in the smallest area possible. The RAM array contains 256 cells, which are accessed in words of 4 bits.

RAM stands for random-access memory. This is a concept best demonstrated by counterexample. An example of non-random access memory would be paper or magnetic tape: data that is stored at the start of the tape can be accessed faster than data at the end of the tape, because we have to go through the entire tape to get to it. In contrast, any data stored in a RAM array can be accessed at the same speed, without going past other data first; any ‘random’ part of the data can be accessed immediately.

As such, there must be some way of locating, or ‘addressing’ the right part of memory. This is done with the 2-bit **X-register** and the 4-bit **Y-register**. Thinking of the RAM array as a grid of words with 4 columns and 16 rows, the currently addressed cell is the cell on the *X*th column, and *Y*th row. When a value is read from the RAM, it is the addressed cell’s value that is outputted. When a value is written to the RAM, it is the addressed cell that is written to.

Additionally, the Y-register and 4-bit **accumulator register** provide unaddressed working memory. When we store a value in the RAM array, we have to keep track of

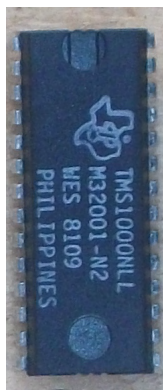


Figure 3.2: TMS 1000 package [25]

its X and Y-register address, and load that address back into the relevant registers when we want to retrieve the value. In contrast, we can access the single 4-bit numbers in the Y-register and the accumulator register at any time. We may think of this as ‘scratchpad’ memory, to be used to store data to be used immediately, or to temporarily store data before moving it to RAM or outputting it.

The **status register** and **status latch** (not shown on the diagram) store the result of logical operation instructions, such as comparing if two values are equal. If the result of such an operation is false, status is set to 0, and otherwise it is set to 1. The status latch is used to store a status value for later use.

Several other registers are used for ROM (instruction) addressing: the program counter, **subroutine return register**, page address register, **page buffer register**, and **call latch** (not shown). The subroutine return register and the page buffer register respectively hold stored program counter and page address values, which may later be loaded into the program counter and page address registers correspondingly. The call latch holds a single bit, which tells whether a subroutine call is currently active.

Finally, the **O-output latches** and **R-output latches** store data to be immediately outputted.

3.3.3 I/O

A microchip’s only connections to external circuits are the spidery metal legs which protrude from it. These are called its **pins**.

Two of the TMS 1000’s pins is used to power the system, delivering V_{DD} and V_{SS} . An additional two are connected to the oscillator used as the system’s clock. The remainder are I/O pins: 5 input pins and 17 output pins.

Outputs

The 17 output pins are divided into 8 **O-outputs** and 11 (or 13, depending on package) **R-outputs**. This is a distinction peculiar to the TMS 1000.

The R-outputs are addressed directly. That is to say, there are instructions **SETR**

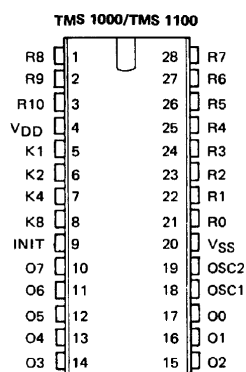


Figure 3.3: TMS 1000 pinout [29, iii]

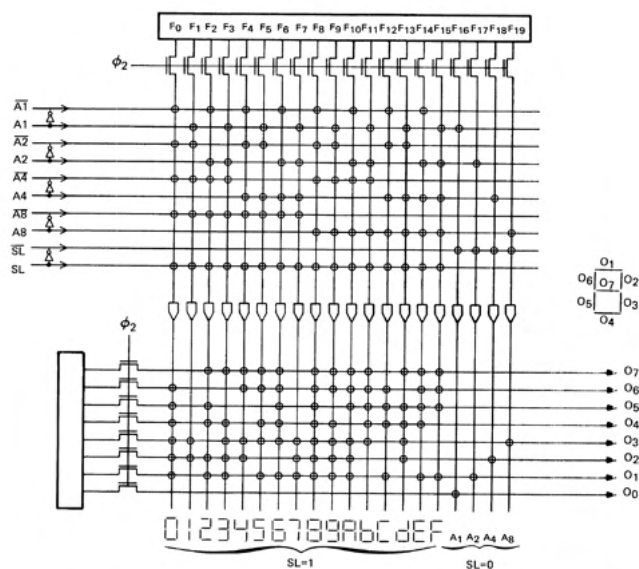


Figure 3.4: Seven-segment driver O-output PLA [29, 2-21]

and **RSTR**, which set a single chosen R-output to a logical 1 and 0 respectively. The R-output that is chosen is selected by the Y-register's value.

The O-outputs are set up primarily as data outputs. Data from the accumulator and status latch are transferred to the O-output latches all at once using a **TDO** instruction. A programmable logic array is between the O-output latches and the output pins themselves, which decodes the five bits stored in the O-output latch into bits outputted on the eight O-output pins.

This makes more sense if we remember that the TMS 1000 was designed with use in scientific calculators in mind. The O-output PLA allows it to decode numbers into seven-segment display signals without any external hardware, as is done in the PLA in the image. As can be seen, if all of the accumulator (A) inputs are 0, the output PLA will output signals to light up segments O1, O2, O3, O4, O5, and O6, creating a 0. In early TMS 1000 documents, the O-outputs are referred to as S-outputs, short for 'segment driver', for this reason.

Inputs

There are five inputs: the four K-inputs, and INIT.

INIT, short for initialize, is unique; when an INIT input of 1 is detected, the microcontroller resets itself, clears most of its registers, and starts executing its program from the beginning again. It exists so that the microcontroller can be reset with a 'reset' or 'clear' key, instead of being turned off and on again. In early TMS 1000 documents, it is labeled as KC, or keyboard clear.

The four K-inputs are K1, K2, K4, and K8. This numbering scheme is commonly used, as it represents the numbers to multiply each bit by, if the four inputs are interpreted as a binary number.

The K stands for 'keyboard'. It may seem that four inputs are not enough to represent all of the possible key-presses of a keyboard. However, the idea that Texas Instruments had was that a key matrix (a grid of buttons) could be created between the R-outputs and K-inputs. If only one R-output line is 1, and one K-input is 1, then we know that the key at the intersection of that R-output and K-input is pressed. We then can distinguish between at most $11 \cdot 4 = 44$ different keys.

In order to read the value off of such a keyboard, the R-outputs are 'strobed'. In other words, one R-input is powered, and the K-inputs are checked; if no key press is detected, the next R-input is powered, and so forth.

In the diagram, an R-K key matrix is created; additionally, the R-outputs are powering a seven-segment display, with data from the O-outputs.

3.3.4 Arithmetic and Logic

The Arithmetic and Logic Unit, or ALU, is a component that does a lot of things by doing very little: all it can do is either add two numbers together, or compare if two numbers are equal. It has two inputs, P and N (short for Positive and Negative). However, almost any value can be loaded into these inputs, and the addition outputs can be stored almost anywhere.

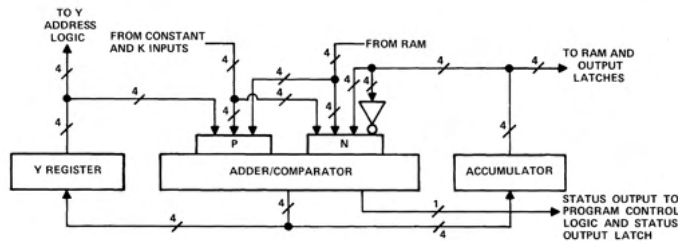


Figure 3.5: ALU block diagram [28, 7]

For instance, suppose that we wanted to subtract two values in memory, $A - B$. First we load B into the P-input. As nothing is loaded into the N-input, it will be 0. Now we store the adder result of $B + 0$ in the accumulator. Now we transfer A to the P-input, and the reversed accumulator bits ('NOT-accumulator') to the N-input, and carry in 1 (the carry input is not shown on the block diagram). Now the adder result is equivalent to adding A to negative B , and we have done subtraction. This exact routine is carried out by executing the instruction **TMA** followed by the instruction **SAMAN**.

Alternatively, say we want to see if a key press has been made. Then we can transfer the K-input value to the P-input of the adder, and compare it to 0. If it is not equal to 0, a key input has been made. This routine is carried out by the instruction **KNEZ**.

3.3.5 Programming

Every instruction for the TMS 1000 consists of an eight-bit-long binary number. However, for human-readability, Texas Instruments used mnemonics. For instance, the instruction 00111100 loads 0 into the X-register. The mnemonic for this instruction is **LDX 0**. A compiler program can convert code written in mnemonic form into the machine code that a TMS 1000 can execute. One such program is included in the TMS 1000 utilities attached to this thesis. This may be considered a basic form of an assembly programming language.

As every instruction in the TMS 1000 is eight bits long, there are 256 possible unique instruction opcodes. However, many of these instructions have operands: for instance, the **LDX** instruction can load any value between 0 and 4 into the X-register, so there are effectively 4 opcodes which decode to **LDX** instructions.

Branch and Call

One feature that is a complete necessity in a computer is conditional nonlinear programming. That is to say, there must be a way to go to a different part of the program, without executing all the code in between. Additionally, it must be possible to go to different parts of the program, depending on if a logical operation evaluates to *True* or *False*.

For instance, suppose that we have a program that either adds or multiplies two

TMS 1000/1200 AND TMS 1070/1270 STANDARD INSTRUCTION SET

FUNCTION	MNEMONIC	STATUS EFFECTS		DESCRIPTION	SYMBOLIC DESCRIPTION	FOR-MAT	OPE-RAND	HEX CODE
		C	N					
Register to Register	TAY			Transfer accumulator to Y register.	$A \rightarrow Y$	4		24
	TYA			Transfer Y register to accumulator.	$Y \rightarrow A$	4		23
	CLA			Clear accumulator.	$0 \rightarrow A$	4		2F
Transfer Register to Memory	TAM			Transfer accumulator to memory.	$A \rightarrow M(X,Y)$	4		03
	TAMIY			Transfer accumulator to memory and increment Y register.	$A \rightarrow M(X,Y); Y+1 \rightarrow Y$	4		20
	TAMZA			Transfer accumulator to memory and zero accumulator.	$A \rightarrow M(X,Y); 0 \rightarrow A$	4		04
Memory to Register	TMY			Transfer memory to Y register.	$M(X,Y) \rightarrow Y$	4		22
	TMA			Transfer memory to accumulator.	$M(X,Y) \rightarrow A$	4		21
	XMA			Exchange memory and accumulator.	$M(X,Y) \leftrightarrow A$	4		2E
Arithmetic	AMAA	Y		Add memory to accumulator, results to accumulator. If carry, one to status.	$M(X,Y) + A \rightarrow A$	4		25
	SAMAN	Y		Subtract accumulator from memory, results to accumulator. If no borrow, one to status.	$M(X,Y) - A \rightarrow A$	4		27
	IMAC	Y		Increment memory and load into accumulator. If carry, one to status.	$M(X,Y) + 1 \rightarrow A$	4		28
	DMAN	Y		Decrement memory and load into accumulator. If no borrow, one to status.	$M(X,Y) - 1 \rightarrow A$	4		2A
	IA			Increment accumulator, no status effect.	$A + 1 \rightarrow A$	4		0E
	IYC	Y		Increment Y register. If carry, one to status.	$Y + 1 \rightarrow Y$	4		2B
	DAN	Y		Decrement accumulator. If no borrow, one to status.	$A - 1 \rightarrow A$	4		07
	DYN	Y		Decrement Y register. If no borrow, one to status.	$Y - 1 \rightarrow Y$	4		2C
	ABAAC	Y		Add 6 to accumulator, results to accumulator. If carry, one to status.	$A + 6 \rightarrow A$	4		06
	ABAAC	Y		Add 8 to accumulator, results to accumulator. If carry, one to status.	$A + 8 \rightarrow A$	4		01
	A10AAC	Y		Add 10 to accumulator, results to accumulator. If carry, one to status.	$A + 10 \rightarrow A$	4		05
	CPAIZ	Y		Complement accumulator and increment. If then zero, one to status.	(Two's complement)	4		2D
Arithmetic Compare	ALEM	Y		If accumulator less than or equal to memory, one to status.	$A < M(X,Y)$	4		29
	ALEC	Y		If accumulator less than or equal to a constant, one to status.	$A < I(C)$	2	C	7—
Logical Compare	MNEZ		Y	If memory not equal to zero, one to status.	$M(X,Y) \neq 0$	4		26
	YNEA		Y	If Y register not equal to accumulator, one to status and status latch.	$Y \neq A, S \rightarrow SL$	4		02
	YNEC		Y	If Y register not equal to a constant, one to status.	$Y \neq I(C)$	2	C	5—
Bits in Memory	SBIT			Set memory bit.	$1 \rightarrow M(X,Y,B)$	3	B	3—
	RBIT			Reset memory bit.	$0 \rightarrow M(X,Y,B)$	3	B	3—
	TBIT1		Y	Test memory bit. If equal to one, one to status.	$M(X,Y,B) = 1$	3	B	3—
Constants	TCY			Transfer constant to Y register.	$I(C) \rightarrow Y$	2	C	4—
	TCMIY			Transfer constant to memory and increment Y.	$I(C) \rightarrow M(X,Y); Y+1 \rightarrow Y$	2	C	6—
Input	KNEZ		Y	If K inputs not equal to zero, one to status.	$K \neq 0$	4		09
	TKA			Transfer K inputs to accumulator.	$K \rightarrow A$	4		0B
Output	SETR			Set R output addressed by Y.	$1 \rightarrow R(Y)$	4		0D
	RSTR			Reset R output addressed by Y.	$0 \rightarrow R(Y)$	4		0C
	TDO			Transfer data from accumulator and status latch to O outputs.	$A, SL \rightarrow O \text{ REG}$	4		0A
	CLO			Clear O-output register.	$0 \rightarrow O \text{ REG}$	4		0B
RAM 'X' Addressing	LDX			Load 'X' with a constant.	$I(B) \rightarrow X$	2	B	3—
	COMX			Complement 'X'.	$\bar{X} \rightarrow X$	4		00
ROM Addressing	BR			Branch on status = one.		1	W	—
	CALL			Call subroutine on status = one.		1	W	—
	RETN			Return from subroutine		4		0F
	LDP			Load page buffer with constant.	$I(C) \rightarrow PB$	2	C	1—

Figure 3.6: TMS 1000 standard instruction set [29, iii]

numbers, depending on if a button is pressed. We would not want to execute the multiplication code if the addition button is pressed, or vice versa. So we would need to go to two different portions of the program, depending on the button press. Alternatively, suppose we have some repetitive code, such as a strobe routine. It would be wasteful of ROM space to write out every instruction, instead of setting up a loop.

The TMS 1000 does this with the **BRANCH** and **CALL** instructions.

The **BRANCH** instruction, or **BR**, loads a value from the instruction into the program counter if status is 1. Additionally, it switches the page buffer and page address registers if status is 1 and the call latch is 0. Otherwise, it does nothing. This is useful for program flow control, creating loops, and conditional ‘if’ statements. Additionally, this is how we can switch between ROM pages; the **LDP** instruction loads a new value into the page buffer, and **BR** loads the page buffer into the page address. However, of note, **BR** does not save the previous program counter and page address values; unless we know where we came from, we can’t return to where we were from a **BRANCH**.

The **CALL** instruction allows for function calls. Say that we have a useful block of code, that we’re going to end up running over and over. Then instead of repeating it multiple times, we can **CALL** it as a subroutine. Like **BR**, **CALL** only does anything if the status is 1, and loads its operand into the program counter and loads the page buffer into the page address. However, **CALL** also saves the old values; the old program counter value is saved in the subroutine latch, and the old page address is stored in the page buffer. The call latch is set to 1 to prevent **BR** from unexpectedly switching in the old page buffer value.

When a subroutine is done, a **RETN** instruction restores the old page address and program counter values, and sets the call latch back to 0. That way, when our repetitive code is done executing, we can return to where we were in the program. If the call latch is already off, **RETN** does nothing.

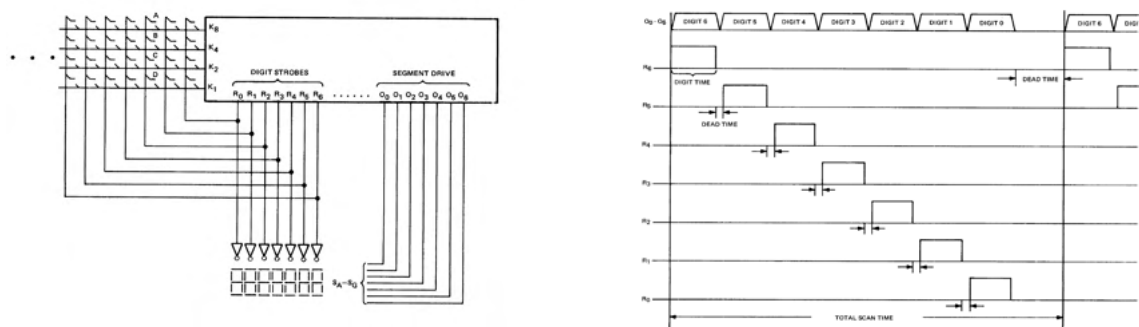


Figure 3.7: Hardware configuration and timing for display routine, for a calculator configuration [29, 13-3]

Some Actual Code

```

DISP TCY 7  PRESET Y=7 (SCAN FROM LEFT TO RIGHT)
DIS1 DYN      DECREMENT Y TO TRANSFER LOWER ORDER
      TMA      DIGIT TO THE ACCUMULATOR
      IYC      INCREMENT Y TO RESET HIGHER ORDER LINE
      RSTR
      DYN      DECREMENT Y TO SET LOWER ORDER LINE.
      TDO      TRANSFER ACCUMULATOR TO OUTPUT DECODE
      SETR
      KNEZ     TEST FOR K INPUT
      BR EXIT  IF K NOT ZERO, BRANCH TO EXIT
      YNEC 0   TEST FOR COMPLETION OF SCAN, Y=0
      BR DIS1  IF NOT COMPLETE, BRANCH TO DIS1
      BR DISP  IF COMPLETE, REINITIALIZE Y

```

Figure 3.8: TMS 1000 display-driving code [29, 13-3]

The code block in Figure 3.8 is an example of how the basic instructions of the TMS 1000 can accomplish useful tasks. This is code to strobe a keyboard and display a calculator output at the same time, in the hardware configuration depicted in Figure 3.7. Note that as this is assembly code, program counter values are represented by human-readable labels (DISP and DIS1) instead of their actual values, just as the instruction opcodes are represented by mnemonics.

Some previous routines have loaded seven numbers in rows 0 through 6 of the current RAM column, to be displayed, perhaps as a calculation's output. The value 7 is loaded into the Y-register (TCY, Transfer Constant [to] Y-register), and then immediately decremented to 6 (DYN, Decrement Y-register [and if there's] No [borrow, set status to 1]).

The sixth RAM column value is loaded in to the accumulator (TMA, Transfer Memory [to] Accumulator). The Y-register is incremented (IYC, Increment Y-register, Carry [sent to status]) so that the previous R-output can be turned off (RSTR, Reset R) and then decremented again (DYN) so that the current R-output can be turned on (SETR, Set R). The keyboard input is tested against 0 (KNEZ, K Not Equal [to] Zero). If there was a key input intersecting with the selected R-output line (6), this will result in a true status level, and the following branch statement (BR EXIT) will be taken. If the branch isn't taken, the program checks if the Y-register is down to 0, which will occur when every digit has been displayed (YNEC 0, Y-register Not Equal [to] Constant).²

²Eagle-eyed readers may notice that this does not line up completely with the timing diagram presented in Figure 3.7. This is because in the original code, the instruction here is YNEC 16. The Y-register will be equal to 15 if it is decremented from 0, due to how subtraction is implemented in the adder by reversing the bits stored in the accumulator to produce an unsigned, inverted number. As 15 is an invalid R-output address, this means that after displaying every digit of the stored number,

The two **BR** statements in a row function as an if-else statement. One peculiarity of the TMS 1000 is that a status value of 0 only lasts for one instruction cycle past when a value is sent to status, and then reset to 1. Thus, one of the two **BR** statements will always be taken: the first if **YNEC 0** is true, and the second if **YNEC 0** is false.

Thus, every loop, the seven-segment displays are powered and made to display the currently stored value, and the keyboard is scanned. This code is executed very quickly: even though the TMS 1000 has a very slow clock speed compared to modern microprocessors, it can execute a keyboard scan in barely a millisecond.³

Microcode

Of note, a lot of instructions do similar things. For instance, **TAM** transfers the accumulator value to memory, **TAMIY** transfers the accumulator value to memory and increments the Y-register, and **TAMZA** transfers the accumulator value to memory and zeroes out the accumulator. If we implemented these independently, it would be inefficient; we would have a lot of control circuitry that do the same few things.

The solution: microcode. We can create tiny instructions that do part of what a full instruction does, such as setting the inputs and outputs of the adder, setting the status based on the ALU's logical operations, or storing data in RAM. The TMS 1000 has 16 microinstructions, and most of its instruction set is created out of combinations of microcode instructions.⁴

In order to differentiate between instructions and microinstructions, we will refer to instructions like **THIS** and microinstructions like *THIS*. Note that the microinstruction convention is the same one we have been using for voltage signals; this is intentional, as microinstructions are implemented physically as powered or unpowered control lines.

To go back to **TAM**, **TAMIY**, and **TAMZA**, all of these instructions are made out of microinstructions. Instruction **TAM** just decodes into the *STO* microinstruction (store accumulator in current memory cell); **TAMIY** decodes to *YTP*, *CIN*, *STO*, and *AUTY*, which (in execution order) loads the Y-register into the adder P-input, adds one to the adder, stores the accumulator value in the current memory cell, and stores the adder output into the Y-register; **TAMZA** decodes to *STO* and *AUTA*, which stores the accumulator value in the current memory cell and transfers the adder result (0 because there are no inputs) to the accumulator, setting it to 0.

Decoding of instructions into microinstructions is done using a PLA. This is notable because it means the TMS 1000 is one of the few microcontrollers that are microprogrammable: programmers could create their own instructions if they chose to.

the TMS 1000 would do nothing for one loop iteration, with all of its R-outputs unpowered, likely to allow charge to drain from the display segments. I felt that this was a needless complication to add to the programming example.

³With a clock speed of 333 kHz, the TMS 1000 will execute 55,500 instructions per second. Each loop iteration executes 11 instructions except for the first and last, which will execute one more each; the loop will run for 7 times, so we can calculate that $\frac{(11 \cdot 4) + (12 \cdot 2)}{\frac{333000}{6}} = 0.0012$ seconds.

⁴Because of the CKI multiplexer, in effect some microinstructions can serve multiple functions.

Execution Sequence	Mnemonic	Logic Affected	Function
1	CKP	P-MUX	CKI to P adder input.
	YTP	P-MUX	Y-Reg to P adder input.
	MTP	P-MUX	Memory (X,Y) to P adder input.
1	ATN	N-MUX	Accumulator to N adder input.
	NATN	N-MUX	Accumulator to N adder input.
	MTN	N-MUX	Memory (X,Y) to N adder input.
	15TN	N-MUX	F ₁₆ to N adder input.
	CKN	N-MUX	CKI to N adder input.
1	CIN	Adder	One is added to sum of P plus N inputs (P+N+1).
	NE	Adder/Status	Adder compares P and N inputs. If they are identical, status is set to zero.
	C8	Adder/Status	Carry is sent to status (MSB only).
2	STO	Write MUX	Accumulator data to memory.
	CKM	Write MUX	CKI to memory.
3	AUTA	AU Select	Adder result stored into accumulator.
	AUTY	AU Select	Adder result stored into Y-Reg.
	STSL	Status Latch	Status is stored into status latch.

Figure 3.9: TMS 1000 microinstructions [29, 2-25]

The more complex or idiosyncratic instructions, such as **BR** and **LDX**, are decoded separately and not microprogrammable. These are termed **fixed instructions**, in contrast to the microprogrammable **programmable instructions**. Additional functionality can be added to fixed microinstructions by defining a new programmable instruction that executes on the same opcode.

3.3.6 A better functional diagram

The block diagram at the start of this section, taken from TMS 1000 promotional materials, was extremely simplified. Figure 3.10, taken from the programmer’s manual, is a more complete block diagram.

One component is worth mentioning. In order to give microcoded instructions proper versatility, a fixed multiplexer is included, the CKI logic, also known as the CKB logic in early documents. This either outputs the last four ROM bits, the keyboard inputs, a bit mask of the current ROM word, or 0, and somewhat functions as a secondary instruction decoder that way. The *CKP*, *CKN*, and *CKM* microinstructions respectively load the CKI logic output to the adder P-input, the adder N-input, and into the currently-selected RAM cell. Here is a table of the CKI logic’s outputs based on instruction word, represented in hexadecimal.

Instruction opcode	CKI/CKB output
0x00-0x07	ROM bits 4-7
0x08-0x0F	K-inputs
0x10-0x2F	0
0x30-0x3B	Bit selection “bitmask”
0x3C-0x3F	0
0x40-0x7F	ROM bits 4-7

Table 3.1: CKI/CKB logic behavior

3.3.7 Try it for yourself

As part of this thesis, I created a TMS 1000 emulator, which is accurate to the TMS 1000 at the microcode level. Using it, TMS 1000 ROM dumps can be executed, and stepped-through in a GDB-like manner, showing what each instruction is decoded to, and what microinstructions it executes. Additionally, a compiler and decompiler is included, allowing us to examine TMS 1000 source code, and to create our own programs. The emulator core has been speedtested, and found to operate at least as fast as an actual TMS 1000 in non-interactive mode.

The TMS 1000 emulator is available at github.com/atomicmoloch/TMS1000.

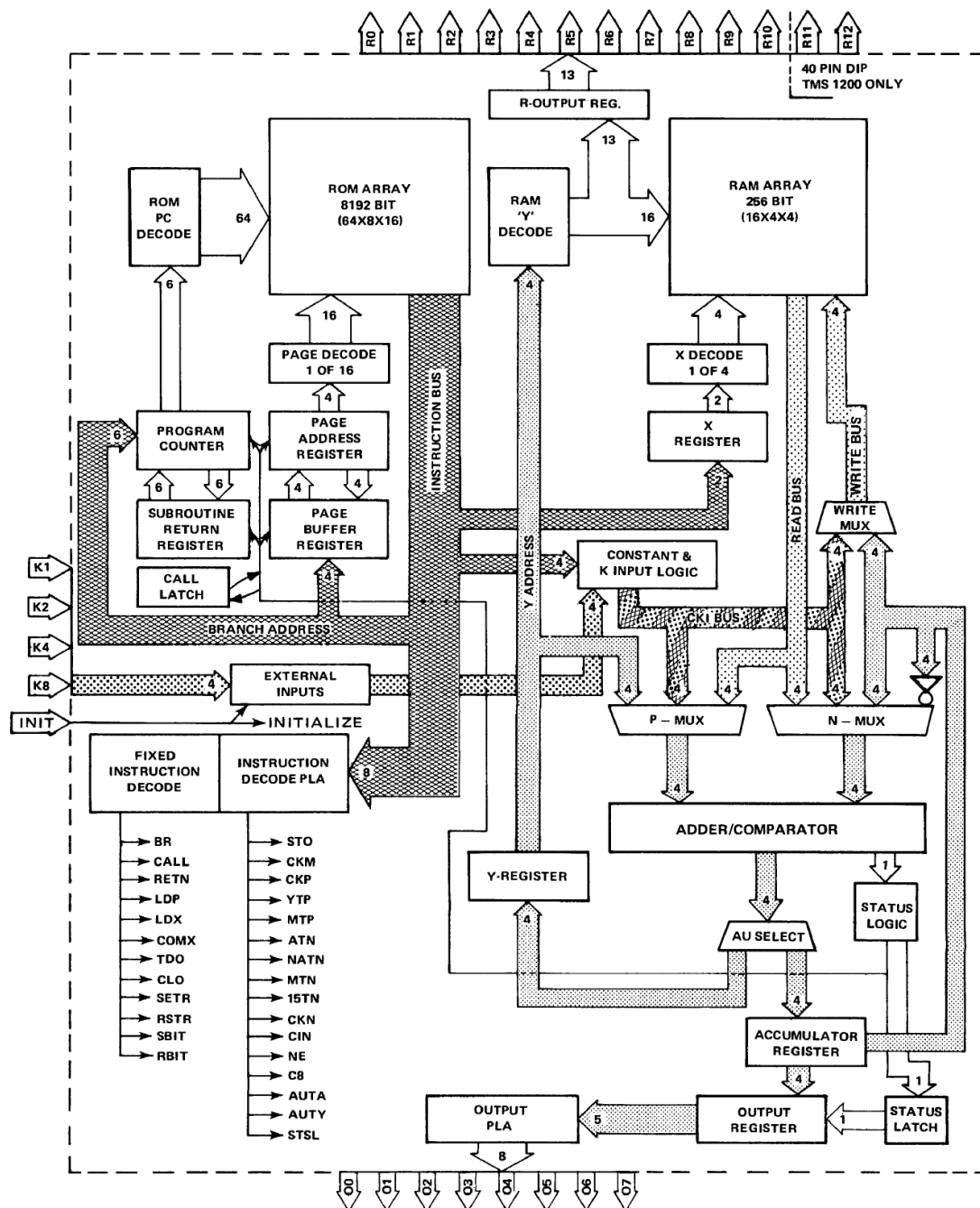


Figure 3.10: Less simplified block diagram [29, 1-6]

Chapter 4

TMS 1000 Components

4.1 Introduction

We now have the knowledge necessary to understand everything in the TMS 1000 microcontroller, almost down to every last line of p-diffused silicon.

This chapter is entirely dedicated to reverse-engineering the TMS 1000. Our primary resources are the patent diagrams and explications included with a series of repetitive patent applications relating to the TMS 1000, submitted throughout the 70s. We will term this standard patent application the TMS 1000 patent. As such, the TMS 1000 patent will be nearly the only source used, and rather than repetitively citing it at every component, we will only do so once here [6]. If a figure in this chapter does not have an explicit citation, it may be assumed to come from the TMS 1000 patent.

Of note, every contemporary patent relating to the the TMS 1000 patent has been in the public domain since the mid 1990s.

4.1.1 Notation

In many components we will discuss ‘stages’: these are repetitive structures that deal with different bits of a data input.

Due to a shortage of time, we will be using unaltered or slightly altered diagrams from the TMS 1000 patent. Texas Instruments submitted a large quantity of near-duplicate applications, in order to gain patent rights over various minor traits of the TMS 1000. We will primarily be using diagrams from U.S. Patent 4,024,386, patenting an ‘electronic calculator or digital processor chip having test mode of operation’.

Many important lines are given numeric labels by the patent. Thus, for instance, ‘146 lines’ will not refer to 146 different lines, but the lines labeled on the diagram as ‘146’. In order to hopefully lessen confusion, we will preface these numeric labels with a dollar sign, so that \$146 lines refers to the lines labeled 146. Additionally, any line that unconditionally carries V_{DD} is labeled as V_{dd} .

Some components in the patent are labeled differently from the production TMS 1000. The most notable differences are listed in Table 4.1.

Patent	Production
CKB logic	CKI logic
D-outputs	R-outputs
KC	INIT
LODS	TDO
S-outputs	O-outputs

Table 4.1: Differences between patent and production convention

4.1.2 Ordering

We would like to present and examine components in an order such that each component builds on previous components that we looked at. Unfortunately, the interconnectedness of the TMS 1000 means that almost everything is dependent on other components in a way that eludes beginnings. Additionally, sometimes a component is more understandable if we know what its outputs are going to be used for, so a dependency goes ‘both ways’.

We will attempt to resolve this by first examining the two components that are ‘independent’: the clock and power-up clear circuit; then we will examine the major components of the TMS 1000: the instruction decoder, ALU, ROM, RAM, and outputs; then, finally, we will examine the support components.

4.2 Clock and Timing Signals

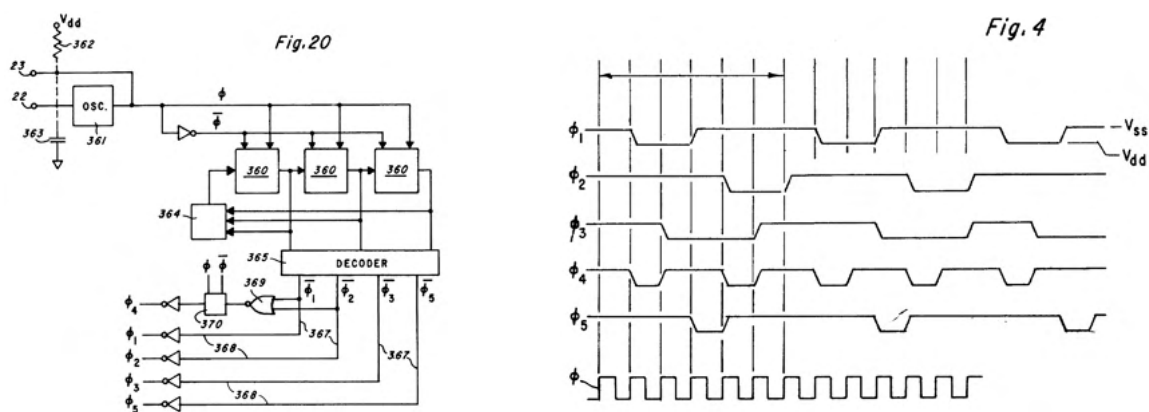


Figure 4.1: Clock circuit and resultant timing signals

4.2.1 Purpose

The clock is the ‘heartbeat’ of the TMS 1000, providing timing signals by which other components are clocked.

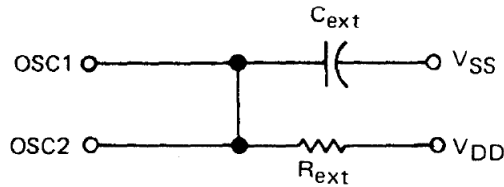
CONNECTION FOR INTERNAL OSCILLATOR

Figure 4.2: External oscillator pin connection [28, 15]

4.2.2 Inputs

Pins 22 and 23 correspond to the OSC2 and OSC1 pins on the exterior of the TMS 1000 package. The oscillator is connected in such a way so that the clock speed may be set, by adding resistance in the oscillator loop. Alternatively, the TMS 1000 can be clocked on an external oscillator, by shorting one of the oscillator pins and connecting the external clock to the other pin.

4.2.3 Outputs

Outputs consist of timing signals $\phi 1$ through $\phi 5$. These allow other components to synchronize their operations, and set up complex timing relationships with each other.

4.2.4 Function

The speed of the oscillator, being an imprecise ring oscillator, is highly variable. Texas Instruments suggests that it varies between 250 and 350 kHz, or thousands of cycles per second, when used with the recommended external resistance and capacitance.

Three counter stages with a feedback circuit take clock pulses and count up to six. A decoder decodes the six distinct states of the counter and produces $\phi 1$, $\phi 2$, $\phi 3$, and $\phi 5$ in inverted form, which are inverted again. Signal $\phi 4$ is active for one oscillator pulse on $\phi 1$ and one oscillator pulse on $\phi 2$.

Timing Scheme

The right-side diagram of Figure 4.1 depicts the timing signals used by the TMS 1000, as they are outputted over time, relative to oscillator pulses. The diagram is read, left to right. Each increment on the X axis depicts the clock and timing signals as they are being outputted at that moment of the clock cycle. The bottom line, labeled ϕ , is the clock, or oscillator signal. Note that, as V_{dd} is a negative voltage, each timing signal line is powered when the representation on this diagram dips.

The use of five separate timing lines is unusual, and even described as unique in a relevant patent application. The reason for choosing this arrangement was to allow complex operations to be implemented within a single machine cycle.

Timing signals $\phi 1$ and $\phi 2$ are the ‘primary’ timing signals. The full active clock cycle takes place between the first pulse of $\phi 1$ and the last pulse of $\phi 2$. Almost

everything that is clocked is clocked on $\phi 1$ or $\phi 2$.

Timing signal $\phi 3$ is largely used for precharging components activated on $\phi 1$ and $\phi 2$. Timing signals $\phi 4$ and $\phi 5$ are specialized, and used in memory operations.

4.3 Power Up and Clear

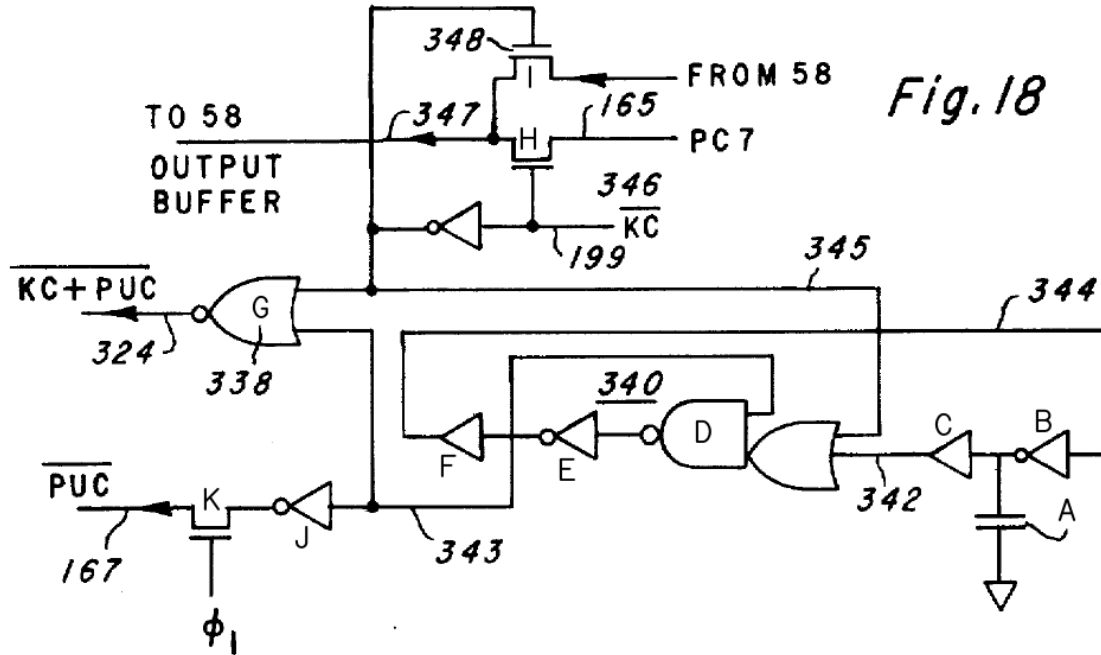


Figure 4.3: Power up and clear circuit

4.3.1 Purpose

The power-up and clear circuit is used to reset system components. This is done both when an external reset signal is received (such as from a 'clear' or 'reset' button), and when the microcontroller is first turned on, so that its program can start execution from a fresh state.

4.3.2 Inputs

Under ordinary conditions, inputs consist of the keyboard input $\overline{KC}$ and timing signal $\phi 1$. Additionally, a data line from the S output buffer is used in the chip's test functionality.

$\overline{KC}$ represents a keyboard reset, and is directly connected to an external pin. Additionally, if powered when the system is first turned on, it holds the $\overline{PUC}$ signal on.

This is so that the TMS 1000 can be delayed from starting its program routine. This is necessary when, for instance, a power supply takes an extended time to achieve a stable output, and attempting to turn it on immediately would possibly fail.

4.3.3 Outputs

The outputs primarily consist of two lines, $\overline{PUC}$ and $\overline{KC + PUC}$. A data line to the S output buffer is used in the chip's test functionality.

Breakdown

Line $\overline{PUC}$, short for Power Up and Clear, resets various components of the microcontroller when it is turned on.

As many components should be cleared both when a reset is requested, and when the microcontroller is first turned on, an additional $\overline{KC + PUC}$ line streamlines functionality by activating in both cases.

4.3.4 Control Lines

Control lines are exactly what they sound like: lines that control other components in the microcontroller. Every microinstruction is implemented as a control line.

Because this is a positive logic convention PMOS circuit, and *False* voltages activate transistors, many control lines represent the inverse of the values or microinstructions they implement. For instance, the control line for the keyboard clear/initialize signal is $\overline{KC}$; e.g., it logically represents the inverse of KC . Thus, if KC is active, $\overline{KC}$ is powered.

4.3.5 Function

Keyboard Reset Function

If keyboard reset (KC) is 1, we want to send out a $\overline{KC + PUC}$ signal.

When $\overline{KC}$ is 0, the signal is immediately inverted by a NOT gate, and becomes 1. This causes NOR gate G to power $\overline{KC + PUC}$, setting it to 0 (charging it to V_{DD}).

Additionally, $\overline{KC}$ facilitates some of the test functions of the TMS 1000, so that the content of the chip's ROM may be read out. When $\overline{KC}$ is powered, transistor H connects the last stage of the program counter to the input of output buffer S8, so its value may be read on an external pin. When $\overline{KC}$ is inactive, transistor I connects output buffer S8 to its respective register, allowing it to output its normal stored value.

Power Up and Clear Function

We want the Power Up and Clear circuit to send out a $\overline{PUC}$ signal, and a $\overline{KC + PUC}$ signal, whenever the system is powered on after being off.

The complexity of the PUC circuit comes from the fact that we cannot make assumptions about voltage levels, and thus logic levels. It must function the same if power has only been disconnected for a second (and so charge remains all across the system) or if it has been disconnected for years.

The two core components of the PUC circuit is Gate D and Capacitor A. Gate D implements the boolean expression $\overline{(\$342 \vee KC) \wedge \$340}$. Thus, it outputs 0 when line \$340 is 1 and either line \$342 or KC is 1, and otherwise outputs 1. Capacitor A is just a capacitor, but it gives us a guarantee: we know that capacitor A is drained. We will additionally assume as a guarantee that line \$340 is 1.¹

Because capacitor A is drained, it acts as a voltage sink. Thus, buffer C's input and output will be 1, and Gate D will output 0.

When Gate D outputs 0, NOT gate E outputs 1, which causes NOT gate J and NOR gate G to power $\overline{PUC}$ and $\overline{KC + PUC}$, respectively.² Additionally, E's output causes buffer F to output 1 on feedback line \$344, which is inverted by NOT gate B. The output of NOT gate B begins charging capacitor A. For the moment, capacitor A functions as a voltage sink, taking in all the current outputted by NOT gate B.

When Capacitor A is charged, it will no longer be a voltage sink, and NOT gate B's output will reach buffer C, causing it to output 0. If line \$345 is also 0, which will occur when $\overline{KC}$ is no longer powered, Gate D will start outputting 1.

When Gate D outputs 1, NOT gate E outputs 0, causing $\overline{PUC}$ and $\overline{KC + PUC}$ to fall to 1. Additionally, NOT gate E's output becomes Gate D's input on feedback line \$340, causing $\overline{(\$342 \vee KC) \wedge \$340}$ to always evaluate to 1 in a self-perpetuating manner. At this point, $\overline{PUC}$ cannot be activated again until the microcontroller is turned off and back on.

Since Gate D holds itself open, capacitor A no longer has any affect on the circuit's output. As buffer F outputs 0, NOT gate B outputs 1, and the charge stored in capacitor A is drained away. This is how we guarantee, at every startup, that capacitor A is drained.

4.4 Instruction Decoder

4.4.1 Purpose

The instruction decodes most of the instructions of the TMS 1000. Note that many of these are composed out of microinstructions, and decoded with a PLA. Others are decoded directly, as the instructions are implemented directly with their own control lines.

The instructions that the instruction decoder does not handle are program flow instructions: *BR*, *CALL*, and *RETN*. These are handled by a separate decoder,

¹This may possibly be achieved by using a high threshold voltage on the gate D driver transistor, which line \$340 connects to, so that a subthreshold drain current will quickly bring the charge below the threshold level when a voltage source is removed.

²Transistor K gates the PUC signal behind timing signal $\phi 1$. This is likely so that the system is guaranteed to start executing at $\phi 1$, rather than starting in the middle of a machine cycle. It may also be involved in pulling up voltage on power-on, as the Programmer's Manual faintly implies.

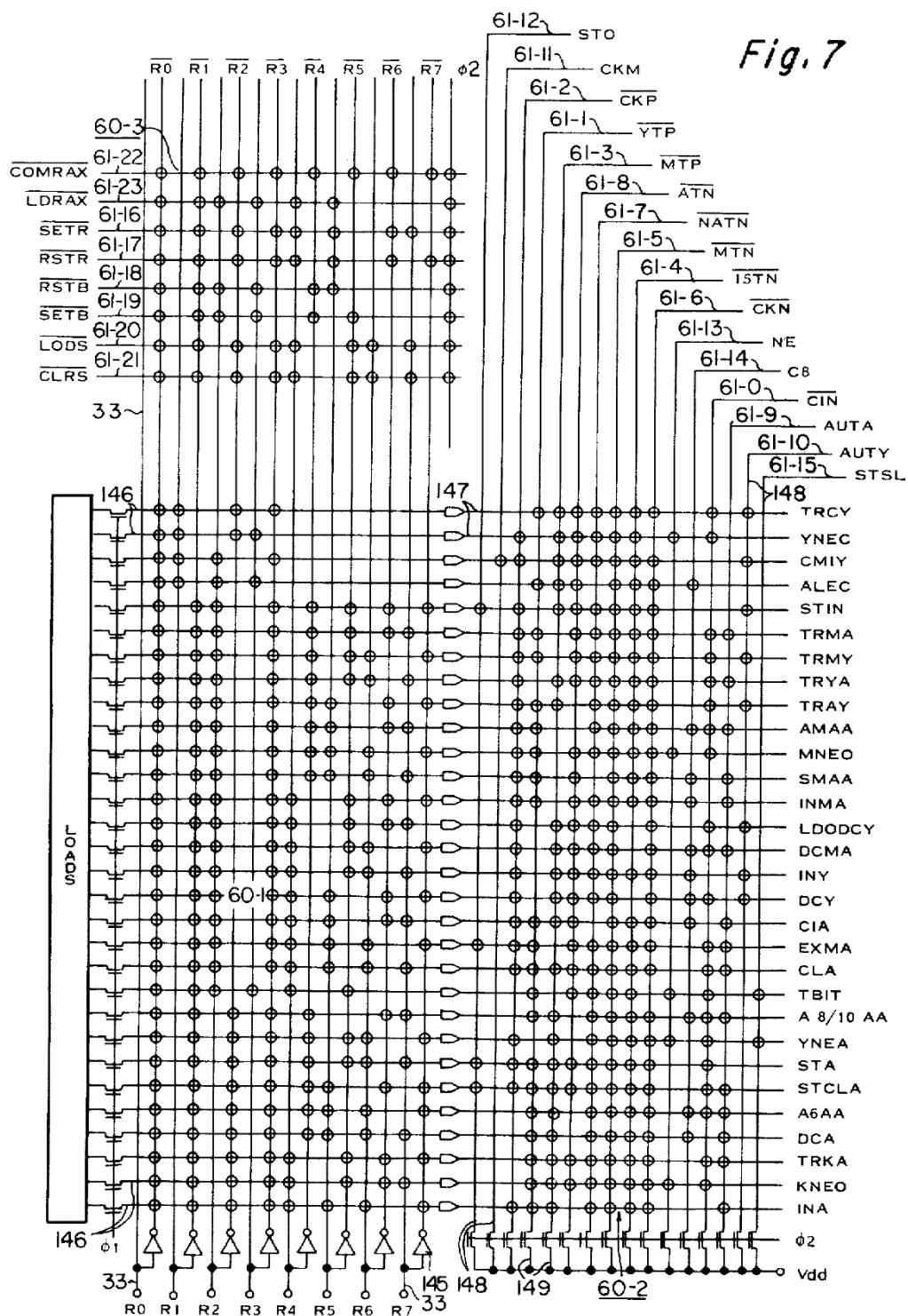


Figure 4.4: Instruction Decoder

labeled as the ‘address controls’.

4.4.2 Inputs

The inputs consist of data lines $R0$ to $R7$, and timing signals $\phi1$ and $\phi2$.

Breakdown

Data lines $R0$ to $R7$ are ROM lines. Each of these hold a voltage signal corresponding to the corresponding bit of the current ROM word. Such a setup may be called a **bus**; as this bus carries the current program instruction, we may say it is the instruction bus. The ROM lines become active (carry a voltage signal) on $\phi2$.

4.4.3 Outputs

Outputs consist of data lines $\overline{R0}$ through $\overline{R7}$; fixed instruction control lines $\overline{COMRAX}$, $\overline{LDRAX}$, $\overline{SETR}$, $\overline{RSTR}$, $\overline{RSTB}$, $\overline{SETB}$, $\overline{LODS}$, and $\overline{CLRS}$; and programmable instruction control lines $\overline{STO}$, $\overline{CKM}$, $\overline{CKP}$, $\overline{YTP}$, $\overline{MTP}$, $\overline{ATN}$, $\overline{NATN}$, $\overline{MTN}$, $\overline{15TN}$, $\overline{CKN}$, $\overline{NE}$, $\overline{C8}$, $\overline{CIN}$, $\overline{AUTA}$, $\overline{AUTY}$, and $\overline{STSL}$.

Breakdown

Data lines $\overline{R0}$ through $\overline{R7}$ are inverted ROM lines, and each contains the opposite bit as that on the corresponding ‘true’ ROM lines. The other lines are microinstruction control lines, which influence operation elsewhere in the TMS 1000.

4.4.4 Function

We want to decode instructions, in the form of ROM bits $R0$ to $R7$, into signals on the control lines that will actuate the desired effects of the instruction.

PLAs

To briefly recapitulate, PLAs are a structure that logically reduces to an array of NAND gates. It consists of two matrices, the AND matrix and the OR matrix, which are constructed the same way, where the outputs of the AND matrix are the inputs of the OR matrix.

In each matrix, the input lines are made out of a metal gate material, and the output lines are made out of p-diffused silicon, initially charged to V_{DD} (logical 0). In between adjacent output lines are p-diffused V_{SS} lines. The circles on the diagram are thin oxide areas, allowing the gate line to connect the intersecting output line with a V_{SS} line through the MOSFET gate effect, discharging it to 1.

Instruction PLA

Programmable microinstructions are decoded using the instruction PLA.

On $\phi 1$, each of the \$146 lines of the AND matrix are precharged to V_{DD} . On $\phi 2$, the ROM lines become active. The gates between the ROM lines and the \$146 lines discharge all, or all but one, of the \$146 lines.³

Buffers are used to convert the silicon lines \$146 into metal strip lines \$147. If a \$146 line is left charged, the corresponding \$147 line will set all of the output lines it has gate intersections with to 1, discharging them.

Of note, this means that if no instruction is decoded by the PLA, every control line will be 0. As a number of the control lines function in inverted form, this effectively means that all the adder instructions *CKP*, *YTP*, *MTP*, *ATN*, *NATN*, *MTN*, *15TN*, *CKN*, and *CIN* are all on ‘by default’. However, without an *AUTA*, *AUTY*, *C8*, or *NE* instruction, these adder instructions do nothing; inputs go into the adder, but the output goes nowhere.

The patent application warns that no programmable instruction starting with 1 should be created, since execution would occur concurrently with **BR** and **CALL** instructions and would lead to undefined behavior; the current instruction could change midway through decoding.

Fixed Instructions

The fixed instructions, $\overline{COMRAX}$, $\overline{LDRAX}$, $\overline{SETR}$, $\overline{RSTR}$, $\overline{RSTB}$, $\overline{SETB}$, $\overline{LODS}$, $\overline{CLRS}$, are decoded directly by the AND matrix.

One side of these control lines are connected to DD through a load transistor (this is not shown in the diagram). If any of the gated ROM lines are 0, the instruction line is discharged to 1; otherwise, it is charged to 0. These can be partial expressions: for instance, *SETB* requires that the first through fifth ROM bits are 001100, and the last 2 bits can be any values.

Of note, these lines are not clocked; thus, they will activate as soon as a ROM word is outputted on the instruction bus, and may stay active until the ROM instruction changes an entire machine cycle later.

4.5 Adder

4.5.1 Purpose

The adder allows two four-bit numbers to be added together, or logically compared. It has two asymmetrical inputs, termed the P-input and the N-input.⁴ The N- and P-inputs are components called **multiplexers**, which allows multiple variables to use the same line, by allowing the input of that line to be selected.

³It is possible to set it up such that multiple \$146 lines are left charged by certain ROM words. In that case, the most restrictive subset of the two lines will be left charged. In the TMS 1100 instruction set, the **CLA** instruction overlaps with the **AnAAC** instruction, but this behavior is not present in the TMS 1000 instruction set.

⁴Standing for positive- and negative-inputs, likely because in subtraction, the negative input comes from the N-input. There is no relation to p- or n-type silicon.

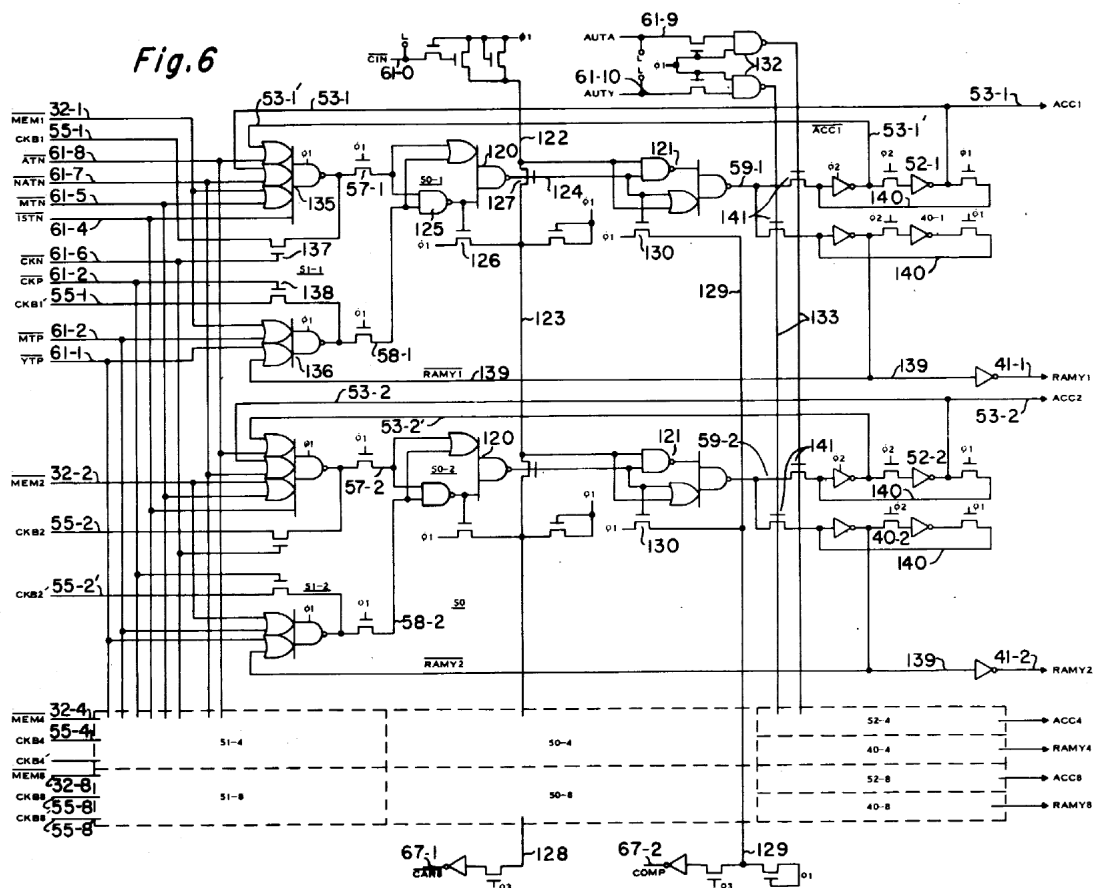


Figure 4.5: Adder. On the original diagram, the memory lines were incorrectly displayed as non-inverted.

The adder utilizes four identical stages, each of which adds and compares together one bit of the N-input, and one bit of the P-input. Each stage represents a digit of a four-bit binary number.

4.5.2 Inputs

Inputs consist of eleven control lines, twelve data lines, and timing signal lines $\phi 1$ and $\phi 2$.

The eleven control lines can be broken up into five lines that affect the N-input, $\overline{ATN}$, $\overline{NATN}$, $\overline{MTN}$, $\overline{15TN}$, and $\overline{CKN}$; three which affect the P-input, $\overline{CKP}$, $\overline{MTP}$, and $\overline{YTP}$; one that affects the carry, $\overline{CIN}$; and two which control where the output is stored, $AUTA$ and $AUTY$.

The twelve data lines consist of $\overline{MEM1}$, $\overline{MEM2}$, $\overline{MEM4}$, and $\overline{MEM8}$, which contain each of the bits of the currently-loaded RAM word; $CKB1$, $CKB2$, $CKB4$, and $CKB8$, which contain each of the bits of the current CKB multiplexer output; and $CKB1'$, $CKB2'$, $CKB4'$, and $CKB8'$, which are duplicates of their respective CKB lines.

Breakdown

$\overline{ATN}$ represents an ‘accumulator to N-input’ microinstruction.

$\overline{NATN}$ represents an ‘inverse of accumulator to N-input’ microinstruction.

$\overline{MTN}$ represents a ‘current RAM word to N-input’ microinstruction.

$\overline{15TN}$ represents a ‘15 to N-input’ microinstruction, which loads the constant 15, or in binary 1111, into the N-input.

$\overline{CKN}$ represents a ‘CKB output to N-input’ microinstruction.

$\overline{CKP}$ represents a ‘CKB output to P-input’ microinstruction.

$\overline{MTP}$ represents a ‘current RAM word to P-input’ microinstruction.

$\overline{YTP}$ represents a ‘Y-register value to P-input’ microinstruction.

$\overline{CIN}$ represents a ‘carry in’ microinstruction. If the line is powered, 1 is added to the first adder stage.

$AUTA$ represents a ‘store adder output in accumulator’ microinstruction.

$AUTY$ represents a ‘store adder output in Y-register’ microinstruction.

4.5.3 Outputs

Outputs consist of ten data lines: ACC lines 1 through 8, $RAMY$ lines 1 through 8, $CAR8$, and $COMP$.

Breakdown

$ACC1$, $ACC2$, $ACC4$, $ACC8$ carry each of the bits of the currently-stored accumulator value.

$RAMY1$, $RAMY2$, $RAMY4$, $RAMY8$ carry each of the bits of the currently-stored Y-register value.

$\overline{CAR8}$ is the inverse of the carry value of the last stage of the adder.

$COMP$ represents a comparison of the adder inputs, and is set to 1 if they are equal, and 0 otherwise.

4.5.4 Function

Each of the four adder stages is constructed the same, and thus will have corresponding component labels. On the diagram, two out of the four stages are depicted.

N-input function

We want the N-input to output the value selected by a powered N-input control line. This is done with a single complex gate per stage: gate \$135.

For adder stage n , complex gate \$135 implements the expression:

$$\overline{(ACCn \vee ATN)} \wedge \overline{(ACCn \vee NATN)} \wedge \overline{(MEMn \vee MTN)} \wedge \overline{(15TN)}$$

Each of the terms in the NAND expression consists of a control instruction and a corresponding data line. Assuming that one and only one of the control lines is active (0), the gate outputs the value of its corresponding data line. For example, if $\overline{MTN}$ is the control line at logical 0, the gate reduces to $(1) \wedge (1) \wedge (\overline{MEMn}) \wedge (1)$, which reduces to $\overline{MEMn}$.

Thus, if $\overline{ATN}$ is 1 ($\overline{ATN}$ is 0), the gate will output 1 if $\overline{ACCn}$ is 1, and 0 otherwise. $\overline{NATN}$ causes an output of $\overline{ACCn}$; $\overline{MTN}$ causes an output of $\overline{MEMn}$; and $\overline{15TN}$ will always cause an output of 1.

If multiple control lines are active, the gate will effectively AND together the data lines corresponding to the multiple gates. If no control lines are active, the output will be 0.

The $\overline{CKN}$ instruction bypasses the logic gate. If $\overline{CKN}$ is 0, $\overline{CKBn}$ is directly connected to the gate's output line. The reason for this is that the $\overline{CKBn}$ data lines are not inverted. If $\overline{CKN}$ is active, the gate is essentially used to precharge its output line. If $\overline{CKBn}$ is 1, the charge is not affected, but if $\overline{CKBn}$ is 0, the charge is drained.

P-input function

We want the P-input to output the value selected by a powered P-input control line. This is done with a single complex gate per stage: gate \$136.

Complex gate \$136 implements the expression $\overline{(\overline{MEMn} \vee \overline{MTP})} \wedge \overline{(\overline{YTP} \vee \overline{RAMYn})}$. Like the N-input gate, the gate outputs the value of each control line's corresponding data line, so that $\overline{MTP}$ causes an output of $\overline{MEMn}$ and $\overline{YTP}$ causes an output of $\overline{RAMYn}$.

The $\overline{CKP}$ instruction bypasses the logic gate. If $\overline{CKN}$ is 0, $\overline{CKBn'}$ is directly connected to the gate's output line, functioning the same as the N-input equivalent.

Adder function

The N- and P-inputs are both active (outputting a valid output) on $\phi 1$. We will designate the outputs of stage n 's N- and P-inputs as Nn and Pn respectively.

We want to add together the N- and P-input values of each stage. How can we do this? If both inputs are 0, when added together, they will equal 0. If one input is 1 and one is 0, then when added together, they will equal 1. If both inputs are 1, they will equal 10 (in decimal: 2); so the current digit will be 0, and a 'carry' signal must be set so that 1 will be added to the next digit.

First we will add together the inputs without worrying about carrying. This is done with complex gate \$120 of each stage, which implements the expression $(Nn \vee Pn) \wedge \overline{(Nn \wedge Pn)}$. Thus, it outputs 1 if both the P-input and the N-input are the same value (either 0 or 1), and otherwise outputs 0. In mathematical terms, gate \$120 outputs the inverse of $Nn + Pn \bmod 2$.

Carry

Now we have to figure out if a carry occurs at each stage, and if so, send a signal to the next stage to add an extra 1 to the value it already computed.

Line \$123, between each stage, carries carry values. It is precharged by $\phi 1$, and so begins at 0. Transistor \$126 connects the carry line \$123 with the ground side of $\phi 1$. A line connects the output of intermediate gate \$125 to the gate of transistor \$126.

Intermediate gate \$125, which is a component of complex gate \$120 and implements $\overline{(Nn \wedge Pn)}$. Essentially, it is the part of gate \$120 that checks if both Nn and Pn are 1 (so that $Nn + Pn = 10$). This is the circumstance that would cause a carry!

Gate \$121 is a second adder gate, similar to gate \$120, which adds the carry value with the output of gate \$120, and inverts it back to its true numerical value. For adder stage 1 value An and carry-in value Cn , complex gate \$121 implements the expression $\overline{(\overline{An} \wedge Cn) \wedge (\overline{An} \vee Cn)}$. Thus, it outputs 1 if An and Cn are different values, and 0 if they are the same value. This is equivalent to $Nn + Pn + Cn \bmod 2$.

Cascading Carry

We aren't done yet! There is one case that we haven't dealt with: what if a carry input causes the next adder stage to also 'overflow' and create another carry value. If $Nn + Pn = 1$, and the carry input is 1, then adding the carry bit will cause a second 'cascading' carry to be required.

This is facilitated by transistor \$124, which connects the input and output carry lines of each stage. If the first adder gate outputs 0 (e.g. if its non-inverted output is 1), the stages are connected. We might think about this as the fact that if the adder inputs sum to 1, we know that if a carry input is received, the carry will overflow. Thus, carry inputs are passed through to the next stage. Additionally, we know that if the first stage adder inputs sum to 1, then the first stage could not be outputting a carry signal.

Thus the input carry line is at 0, no effect will occur, but if it is at 1, charge from

the output carry line will drain into the input carry line, setting it to 1 as well.

Accumulator and Y-register function

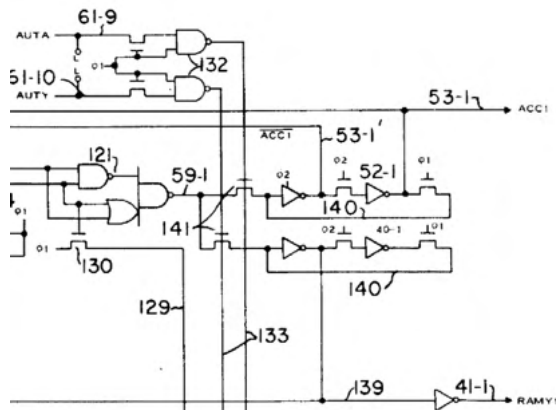


Figure 4.6: Accumulator and Y-register

Now we have added together the inputted values, and accounted for carrying. There is only one thing left to do: storing the result.

We have two options for where we want to store it: the Y-register and the accumulator. These are both implemented as two almost-identical recirculating shift registers on each stage, both labeled 140. They can best be identified by seeing where their output lines lead.

We select where we want to store the adder result with two control signals, *AUTA* and *AUTY*. When *AUTA* is powered on $\phi 1$, each stage's adder output is connected to the accumulator latch input. When *AUTY* is powered on $\phi 1$, each stage's output is connected to the Y-register latch input.

The accumulator value is outputted on *ACC1* through *ACC8* on $\phi 2$, while the Y-register is outputted on *RAMY1* through *RAMY8* on $\phi 1$.

CIN, CAR8, and COMP function

Control line $\overline{CIN}$ represents a 'carry in' microinstruction. Basically, it adds 1 to whatever the adder value would be. When powered, activates a transistor connecting the input line of adder stage 1 (the first adder stage) with $\phi 1$, adding 1. An additional transistor only allows $\overline{CIN}$'s signal to reach the aforementioned transistor on $\phi 1$.

The carry output of adder stage 8 (the fourth and last adder stage) is clocked on $\phi 3$, inverted, and outputted on line $\overline{CAR8}$. This is used in status operations and allows conditional branches and logic statements to be made by comparing numbers. For instance, if we want to test if a number in memory is greater or equal to 10, we can add 6 to it and check $\overline{CAR8}$. In this case, $\overline{CAR8}$ will be 0 if 6 added to the mystery number is 16 or greater, and 1 otherwise.

But what if we want to compare for precise values, like if a mystery value is exactly equal to 10? This is done with the *COMP* output, which is implemented as follows:

Line 129 is precharged by ϕ_1 , charging it to 0. If adder gate 1 outputs 0 in any stage, indicating that the N- and P-inputs are not the same value, transistor 130 is activated, connecting line 129 to ground and setting it to 1. On timing signal ϕ_3 , line 129 is inverted and outputted on *COMP*. Thus, *COMP* is 1 if the N- and P-input values are identical, and 0 otherwise.

4.6 ROM array

Fig. 14

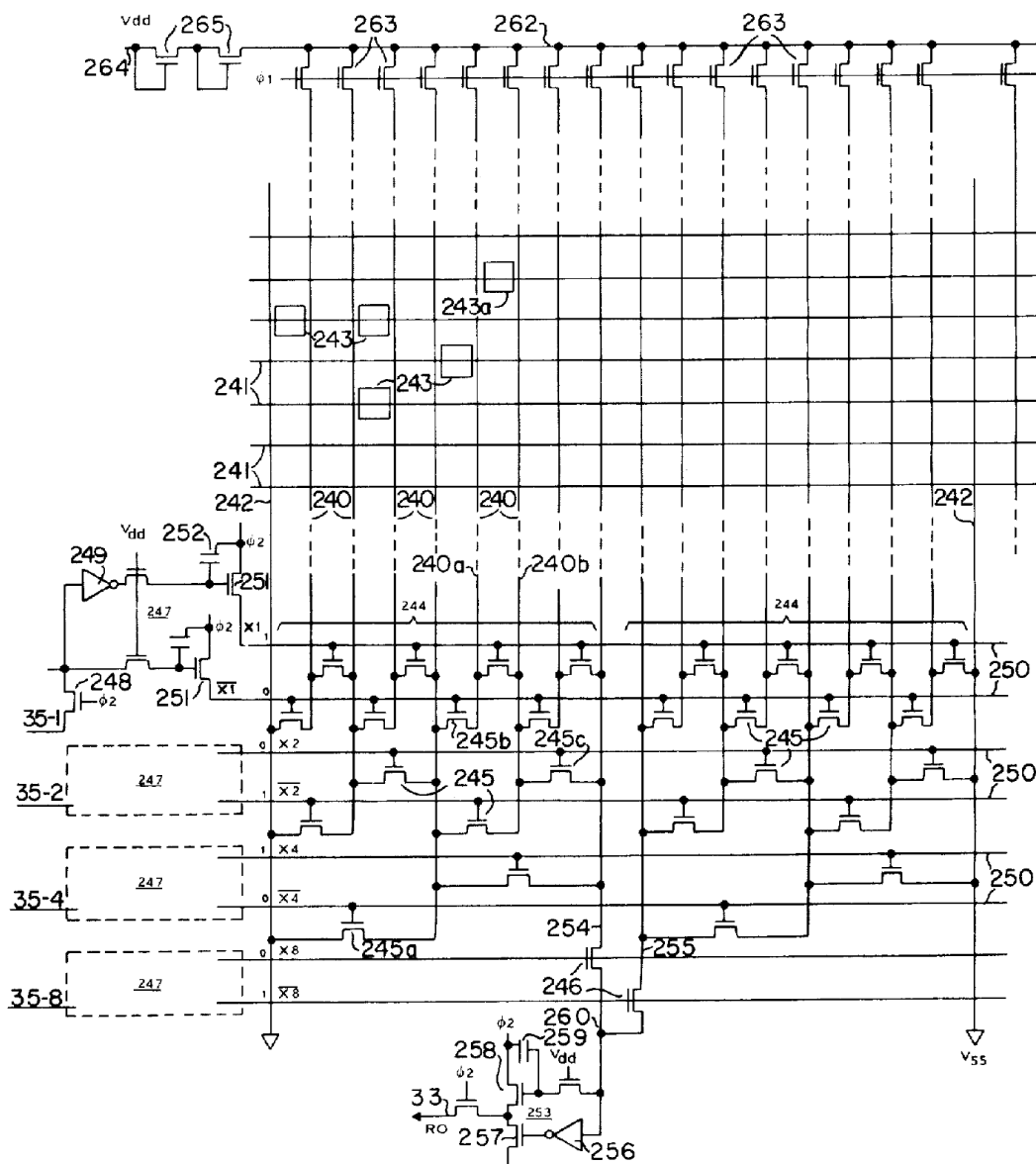


Figure 4.7: ROM array

4.6.1 Purpose

The purpose of the ROM array is to hold the program ran on the TMS 1000. It is an instance of read-only memory, so it cannot be reprogrammed after it is manufactured.

In order to run a program, we must have some way of choosing what instruction to read and reading out the bits of that instruction.

The ROM is organized into 16 ‘pages’, each of which contain 64 instructions, which are each 8 bits long. The current page that we want to read is stored as a value in the page address register. The current instruction that we want to read on the selected page is stored as a value in the program counter register, which also handles incrementing the instruction.

As the ROM is organized as a grid, these values effectively choose what row and column we are reading. These values need to be ‘decoded’ into a row and a column to send power to.

The page address value is decoded by the page address decoder, depicted in this circuit diagram. However, the program counter value is decoded by the RAM/ROM word decoder, which is depicted in its own circuit diagram, and not shown here. The RAM/ROM word decoder will power one of 64 address lines, labeled \$241, that we will take as inputs.

4.6.2 Inputs

Inputs consist of page address lines \$35-1 through \$35-8, ‘row’ address lines \$241, and timing signals $\phi 1$ and $\phi 2$.

Breakdown

Page address lines \$35-1 through \$35-8 contain the page address value as a four-bit number.

There are 64 \$241 address lines. For each possible value of the program counter, a different line will be powered (logical 0). At any program counter value, only one line will ever be powered.

4.6.3 Outputs

Outputs consist of eight lines, $R0$ to $R7$, which carry each bit of the currently-loaded ROM word. These appear as on the diagrams of other components. Only $R0$ is depicted in the diagram.

4.6.4 Function

The diagram only depicts the circuit to read one of the seven bits of the current ROM word. Each read circuit is identical; the full circuit consists of what is pictured, repeated seven times.

Representing bits with transistors

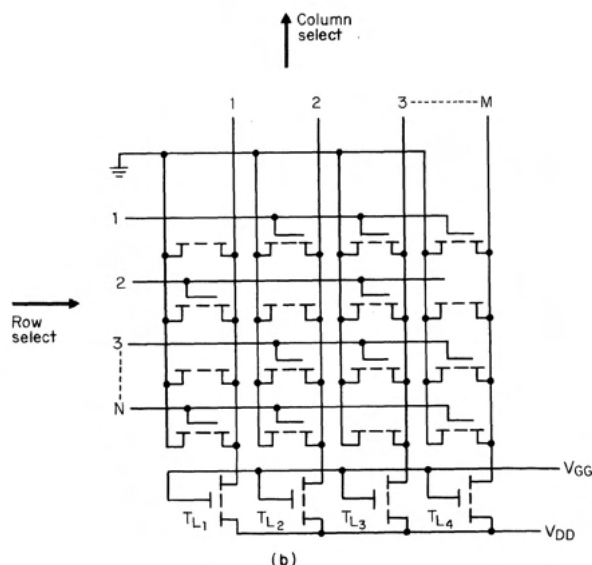


Figure 4.8: Basic ROM array [5, 198]

We want to be able to permanently store instructions in the ROM. Thus, we need some way to physically encode bits. This is done using transistors; the ROM functions similarly to a programmable logic array (PLA) matrix.

‘Row’ lines intersect with ‘column’ lines in a grid pattern.

The row lines are metal strips. The column lines are created out of p-diffused silicon. If a thin oxide region exists between two silicon column lines, beneath a metal row line, then negatively charging the row line will connect the two column lines, through the MOSFET action.

In a basic masked ROM array, every other column would be connected to ground, and the rest would be precharged to logical 0. That way, by placing a transistor between two adjacent columns, the column line will be discharged to logical 1 if the address line is powered, and by not placing a transistor there, the column line will remain at logical 0.

The TMS 1000, to save space on the chip, uses a clever system to achieve the same effect. For any ROM page address value, half of the column lines are connected to ground, but a different selection of lines is grounded for each ROM page address value. This means that a different transistor decides the bit output value for each different ROM page address value.

Page Address Decoder

As mentioned, the program counter is decoded by another TMS 1000 component, and we will not worry about it yet. However, we need to decode the sixteen possible page address values, so that the ROM outputs the bit on the correct page.

The page address is decoded into the column lines of the grid. There are 17

column lines. The first and last column lines are grounded, and the middle two lines (the 9th and 10th lines) are used as output lines.

Each bit n of the page address is split into two lines, Xn and $\overline{Xn}$.

If $X1$ is 0, transistors every other column line is connected together. If $\overline{X1}$ is 0, the lines which $X1$ did not connect together are connected together, effectively also connecting every other column together.

If $X2$ is 0, transistors connect every two columns together. If $\overline{X2}$ is 0, the opposite selection of columns is connected together, like with $\overline{X1}$.

Lines $X4$ and $\overline{X4}$ likewise connect every four columns together.

Finally, lines $X8$ and $\overline{X8}$ connect either the ninth or tenth column lines to the ROM output line for that bit (in the diagram, $R0$).

Together, these bits ‘select’ one unique column line. If an active transistor connects it to its neighbor, it will be brought to ground (1), and the ROM output will be 1. Otherwise, the selected column line will remain charged to 0, and the ROM output will be 0.

This is confusing and hard to understand! An example of how it works will make the operation more clear.

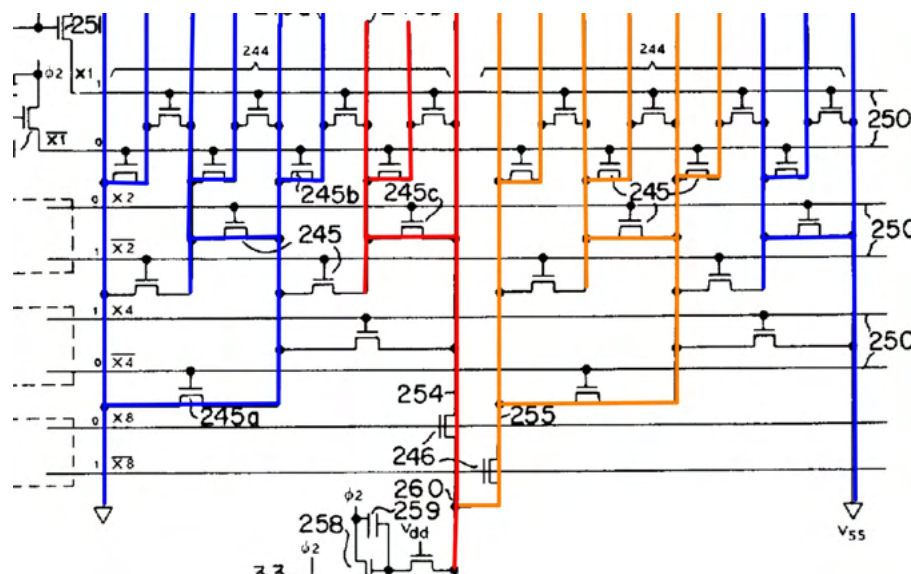


Figure 4.9: Page address demonstration

This diagram depicts the column lines that are charged to 1 and 0 for page address values of 0101 and 1101. Transistors connect every column line left of the seventh column (from the left), and every column line right of the fifteenth column (from the left) to ground. Additionally, it connects the seventh column to the ninth column, and the fifteenth column to the tenth column.

Columns that are at ground are colored blue. Either column nine (colored red) or column ten (colored orange) will be connected to the ROM output, depending on if the first page address bit is 1 or 0.

If a transistor in the ROM array connects two of the columns connected to ground, or two of the columns connected to the output lines, it will do nothing. Those columns

are already connected, and already have the same charge on them.

However, if a transistor connects a ground line and an active line (such as a transistor between the sixth and seventh columns), it will discharge the output line to 1. Thus, for instance, at a page address value of 0101, the ROM output of that bit is entirely dependent on if a transistor exists between the sixth and seventh columns, on the single row line that is powered. At a page address of 1101, the output value is dependent on the presence of a transistor on the active row line between the fifteenth and sixteenth columns. We have successfully stored data as a matrix of transistors!

Note that the tenth and eleventh columns should never be connected together, or else the ROM output would be dependent on two transistors (two memory locations) instead of one.

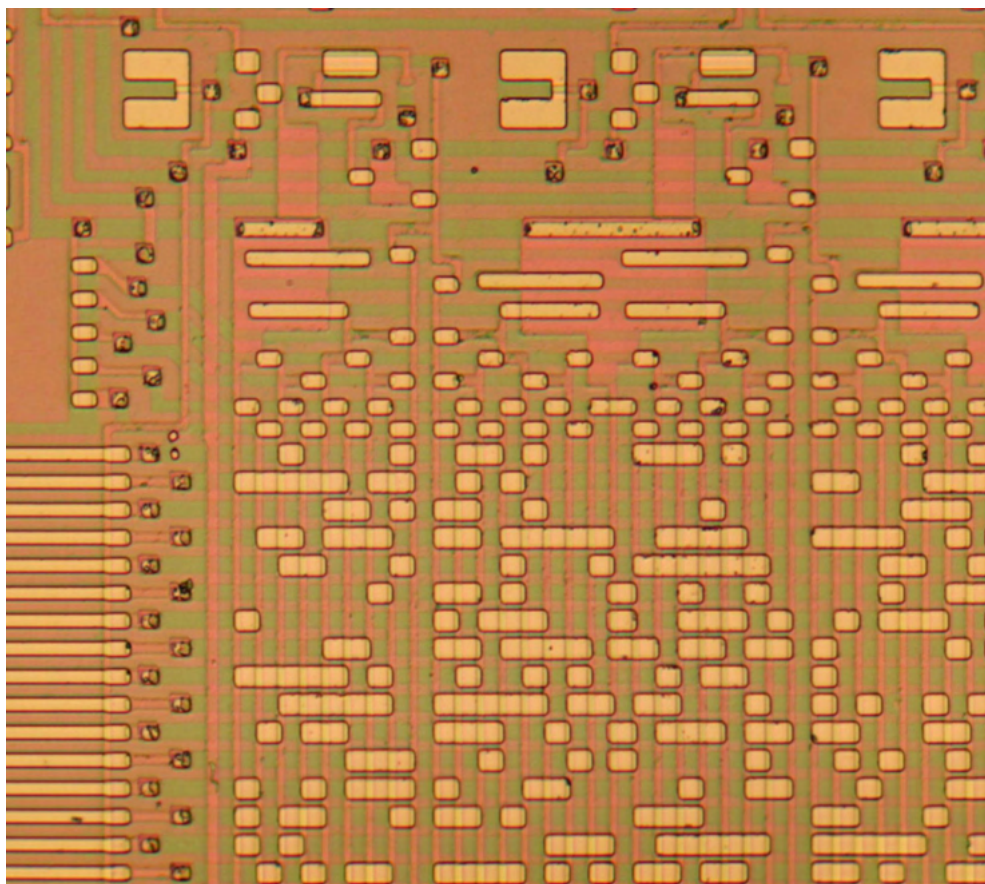


Figure 4.10: One corner of the silicon layer of the ROM array, featuring two stages of the Page Address decoder and part of the ROM array matrix below. Row lines are horizontal. Column lines are vertical. The beige regions are thin oxide regions, effectively MOSFET transistor connections. Note that the ninth and tenth column lines of each page address stage are never connected by a thin oxide region. [25]

Fig. 5

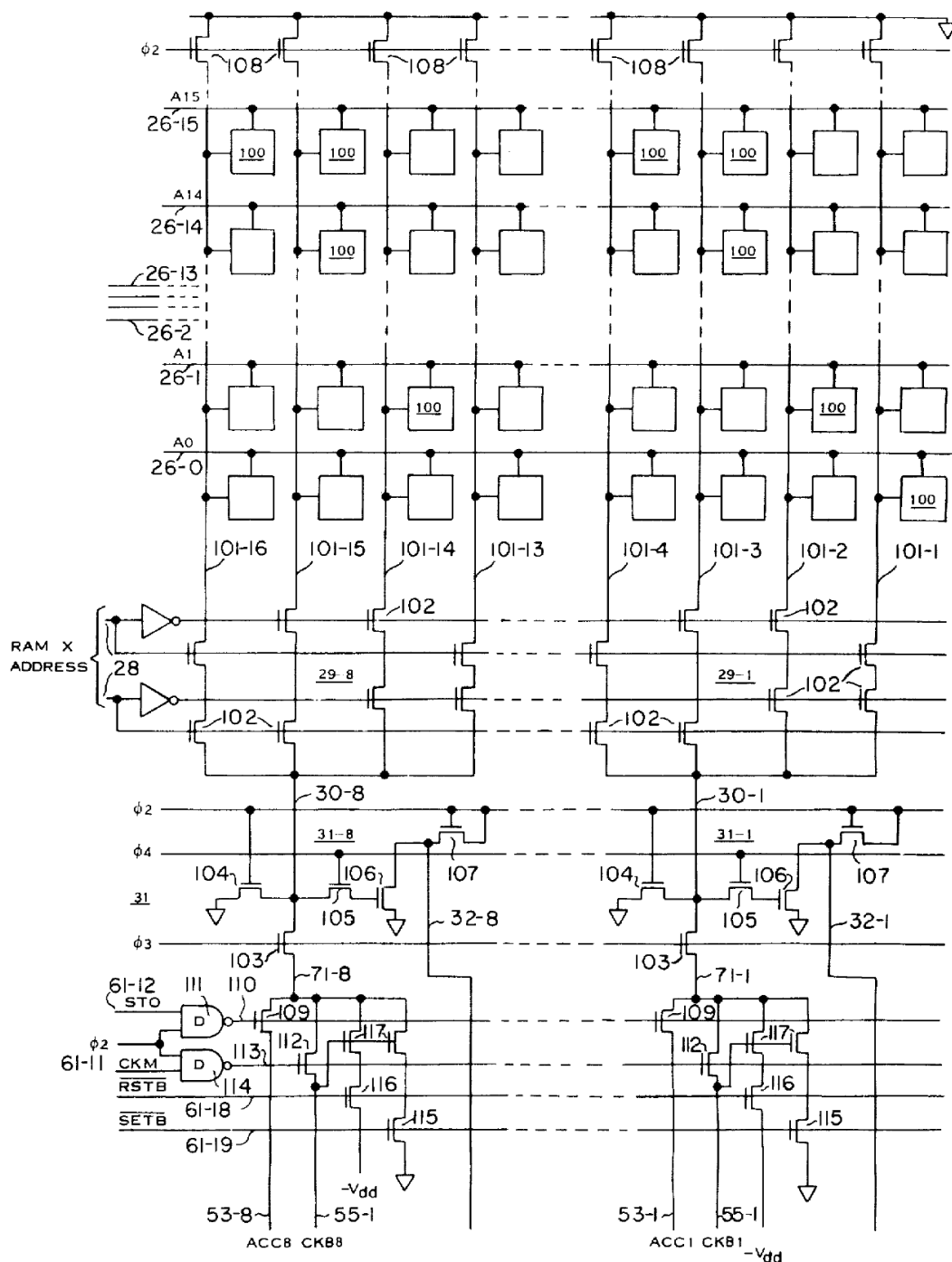


Figure 4.11: RAM array and controls

4.7 RAM array and controls

4.7.1 Purpose

The RAM array is the primary ‘working memory’ storage area on the TMS 1000. It is composed out of 256 RAM ‘cells’, each of which can store one bit of memory (this is not a patent component label; there are actually 256 RAM cells). Its word size is 4 bits, so 4 bits are selected at any one time.

In order to use the RAM, we must have

1. A way of selecting a specific cell.
2. A way of reading the value in the selected cell.
3. A way of storing a new value in the selected cell.

Like in the ROM array, RAM cells are organized in a grid. A cell is selected by selecting a row and column. The column is selected by the 2-bit X-register value, from 0 through 3. The row is selected by the 4-bit Y-register value, from 0 through 15.

Also like in the ROM array, the row value is decoded by a separate component, the RAM/ROM word decoder, which is depicted in its own circuit diagram. We will assume that the correct row line is already powered, and take it as an input. The X-register decoder is pictured on this circuit diagram, and is very simple.

4.7.2 Inputs

The active RAM cell is selected by decoded row selection lines $A0$ through $A15$, and by two lines which contain the value in the X-register, $RAX0$ and $RAX1$. The row selection lines are active at all times except for $\phi2$.

In order to allow data to be stored in the RAM array, four control lines select what data to store: STO , CKM , $\overline{SETB}$, and $\overline{RSTB}$. Data lines $ACC1$ through $ACC8$ and $CKB1$ through $CKB8$ contain the data itself.

Additionally, the RAM array is the only TMS 1000 component which utilizes every timing line: $\phi1$, $\phi2$, $\phi3$, $\phi4$, and $\phi5$.

Breakdown

STO carries a ‘store accumulator value in memory’ microinstruction.

Lines $ACC1$ through $ACC8$ carry the value stored in the accumulator. Data from these lines is stored in the current RAM word in event of a STO microinstruction.

CKM carries a ‘store CKM output in memory’ microinstruction.

$\overline{SETB}$, and $\overline{RSTB}$ carry microinstructions for bitwise operations, corresponding to the $SBIT$ and $RBIT$ instructions. They allow individual bits of a RAM word to be set to 1 and 0, respectively.

Why is this needed? Suppose that we were writing a program that needed to save some boolean values for later reference. For example, we could be making a

calculator, and need to keep track of if the current number is negative or not. While we could use an entire RAM word to save this value, that is wasteful: a RAM word contains four bits, and we only need to save one bit of data. If there are a lot of boolean values we need to keep track of, it could amount to a lot of wasted space.

However, with the **SBIT** and **RBIT** instructions, and their corresponding microinstructions, we can store our values in a single bit of a RAM word. Four boolean variables can be saved in the same RAM word. Such single-bit variables are sometimes known as ‘bit flags’. **SBIT** takes an operand, between 0 and 3, and sets the corresponding bit of the current RAM word to 1. **RBIT** takes an operand, between 0 and 3, and sets the corresponding bit of the current RAM word to 0. Control lines $\overline{SETB}$ and $\overline{RSTB}$ are powered if their corresponding instructions are decoded by the instruction decoder. The **TBIT** instruction, which is decoded by the control PLA, allows bit flags to be checked, so we know if the stored value is 0 or 1.

Lines *CKB1* through *CKB8* carry outputs from the *CKB* multiplexer. This is used directly by *CKM*, and less directly by the bitwise operations.

4.7.3 Outputs

The RAM outputs the contents of the current RAM word, in inverted form, on lines 32-1 through 32-8. These appear as $\overline{MEM1}$ through $\overline{MEM8}$ in the adder circuit.

4.7.4 Memory Cell

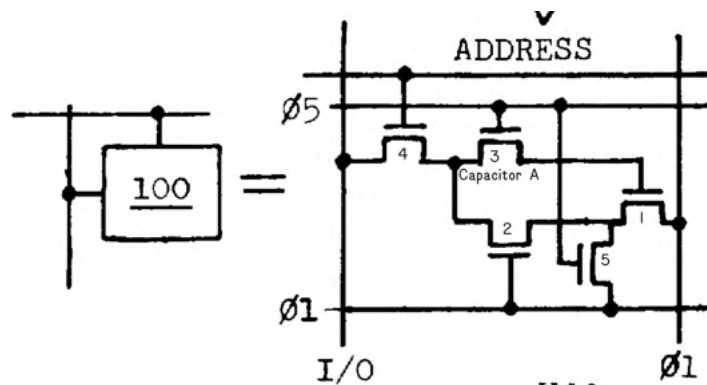


Figure 4.12: Single RAM cell

Before we examine how the RAM array works, we will look at how a single memory cell works.

We basically want every cell to behave as a latch. We want to be able to store a value in the cell indefinitely, and to be able to access that value at a later point. Using the standard latch design would be possible here, but would be very space-inefficient, power-inefficient, and would use far more transistors than are necessary. Since we have so many RAM cells, we have to seriously concern ourselves with efficiency.

Inputs and Outputs

Each cell takes as inputs an *ADDRESS* line, which is one of the RAM array ‘row’ lines, selected based on the Y-register value. Additionally, each cell uses timing signals $\phi 1$ and $\phi 5$. Note that $\phi 5$ is powered after $\phi 1$ and before $\phi 2$, not overlapping with either.

Finally, the *I/O* line serves as both an input line and an output line. At different parts of the RAM cell’s cycle, the stored value is outputted on the output line, and a new stored value can be inputted through it.

Storing a Value

The RAM cell’s connection to the *I/O* line is controlled by transistor 4, which is controlled by the *ADDRESS* line. The *ADDRESS* line will be powered on all timing signals except for $\phi 2$ if the RAM cell’s row is selected by the Y-register value. Otherwise, it will remain unpowered, and the cell will remain disconnected.

Charge is stored in each memory cell in two places: Capacitor A, and the gate of transistor 1, which is constructed to have a lot of surface area, and thus function as another capacitor.

If *ADDRESS* is powered, and $\phi 5$ is powered, then both capacitors are connected to the *I/O* line. Thus, if the *I/O* line is connected to a voltage source (*DD* or ground) in this circumstance, both capacitors will be charged or discharged to the voltage of *I/O*, and a new value will be stored in the cell.

Refresh Circuit

If the cell is not being written to (if *ADDRESS* is not powered, or *I/O* is not connected to a voltage source on $\phi 5$), then the stored value will be recirculated or refreshed, allowing it to be stored indefinitely.

When $\phi 5$ is powered, transistor 3 is activated, allowing capacitor A to charge or discharge the gate of transistor 1, equalizing the voltage levels. If capacitor A is charged to logical 0, transistor 1’s gate will be charged to logical 0; if capacitor A is charged to logical 1, transistor 1’s gate will be charged to logical 1.

When $\phi 1$ is powered, transistor 2 connects the source of transistor 1 to capacitor A. If the charge stored in transistor 1’s gate is V_{IL} or greater, transistor 1 will be active. As $\phi 1$ is powered (at V_{DD}), the voltage at transistor 1’s source will be barely below V_{DD} , and capacitor A will be charged to logical 0.

Otherwise, if transistor 1 is not active, no voltage will pass through transistor 1, and capacitor A will be charged to logical 1. How is this possible, as no ground voltage source is connected? This is effected by transistor 5, connecting the source of transistor 1 with $\phi 1$, and activated by $\phi 5$. When $\phi 5$ is powered, $\phi 1$ will be at ground. Thus, transistor 5 precharges the source of transistor 1 to logical 1.

Now capacitor A is charged to the voltage value that transistor 1’s gate was charged to by capacitor A. We can say that the value has been refreshed.

Reading a value

If *ADDRESS* is powered on $\phi 1$, and *I/O* is not connected to a voltage source, then when capacitor A is refreshed, *I/O* will also be charged to the stored value, allowing us to read it.

4.7.5 RAM controls

The RAM controls consist of four identical circuits (stages), each of which handles one bit of the current RAM word.

We want to be able to read the value stored in the RAM cell chosen by the X and Y-register values, and to store data of our choosing in each cell.

Of note, RAM cells are not zeroed out by any $\overline{KC}$ or $\overline{PUC}$ line. Thus, at power up, they may contain invalid values, and should be set by a program before they are read from.

X-register Decoder

Lines \$101 are the *I/O* lines of each RAM cell, and form the ‘columns’ of the RAM array. The X-register decoder connects one out of every four \$101 lines to lines \$30-1 through \$30-8.

It does this in what may be called a ‘brute force’ method. The X-register input lines are split into their true and inverted forms. Each \$101 line is connected to its corresponding \$30 line through two transistors. One of these transistors is gated on either *RAMX0* or $\overline{RAMX0}$, and one is gated on either *RAMX1* or $\overline{RAMX0}$. Thus, for a given *RAMX0* value, half of the \$101 lines will be disconnected, and for a given *RAMX1* value, half of the \$101 lines will be disconnected, leaving only one \$101 line connected to each \$30 line output.

Reading

On $\phi 2$, transistors \$104 and \$108 are opened, discharging all \$30 and \$101 lines to 1 (note that as *ADDRESS* lines are never powered on $\phi 2$, this does not affect the values stored in each cell). Additionally, on $\phi 2$, transistor \$107 is opened, precharging each stage’s \$32 output line to 0.

On $\phi 1$ of the next machine cycle, the contents of the chosen RAM cell are outputted on line \$30 of each stage, through the X-register decoder. On the first $\phi 4$ pulse of the cycle, which overlaps with $\phi 1$, each \$30 line is connected to the gate of transistor \$106. If the chosen RAM cell contains a 0, transistor \$106 will be activated, draining the output line \$32 to logical 1. Otherwise, the \$32 line will remain precharged to logical 0.

On the next $\phi 4$ pulse, which overlaps with $\phi 2$, the gate of transistor \$106 is connected back with line 30, which is concurrently being drained, allowing it to precharge.

Writing

Input to each stage n comes on input line $\$71-n$, which is connected to line $\$30-n$ on $\phi 3$. As $\phi 3$ overlaps with $\phi 5$, a voltage source connected to $\$71-n$ will thus write a new value into the selected RAM cell. The reason $\phi 3$ is used instead of $\phi 5$ is for the sake of precharging.

If no voltage sources are connected to $\$71-n$, the charge on the I/O line of the selected cell will not be sufficient to overwrite the stored value. If a voltage source (V_{DD} or V_{SS}) is connected, the stored value will be replaced by that of the voltage source.

Gate $\$111$ and transistor $\$109$ connect each line $\$71-n$ to the corresponding $ACCn$ input on $\phi 2$ if STO is 1. This is accomplished by NANDing together STO and $\phi 2$, and connecting the output to transistor $\$109$.

Gate $\$114$ and transistor $\$112$ connect line $\$71-n$ to the corresponding $CKBn$ input on $\phi 2$ if CKM is 1. This is accomplished the same way as STO .

The bitwise operations are a bit more involved. The CKB multiplexer separately decodes $SBIT$ and $RBIT$ instructions, in addition to the instruction decoder. On these instructions, the CKB multiplexer will then decode the last two bits of the instruction into a selection of an output line, in the same way that the X-register decoder does. Thus, if $\overline{SETB}$ or $\overline{RSTB}$ is active, we know that one $CKBn$ line is powered, and all others are not. This allows us to select a bit within the RAM word.

Thus, $\overline{RSTB}$ is connected to the gate of transistor $\$116$, and $CKBn$ is connected to transistor $\$117$ in series. If the voltages applied to both transistors are logical 0, line $\$78-n$ will be connected to V_{DD} , setting the selected bit of the currently-selected word to 0.

$\overline{SETB}$ works similarly, but with a ground line in place of a DD line. Transistors $\$115$, controlled by $\overline{SETB}$, and $\$117$, controlled by $CKBn$, in series connect $\$71-n$ to ground. If $\overline{SETB}$ and $CKBn$ are both logical 0, 1 will be written into the selected bit of the current RAM word.

Note that as write operations occur after read operations, it is possible to do both operations at once. This property is utilized in the XMA instruction (exchange memory and accumulator).

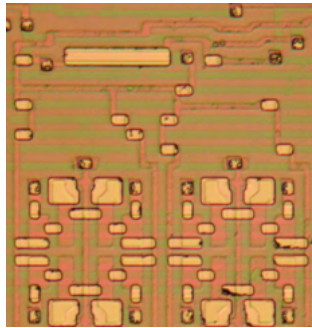


Figure 4.13: The silicon layer of a small part of the RAM array, featuring part of an X-register decoder circuit. The large, square beige areas are the silicon portion of transistor 1 in the memory cell circuit. [25]

4.8 S-Outputs

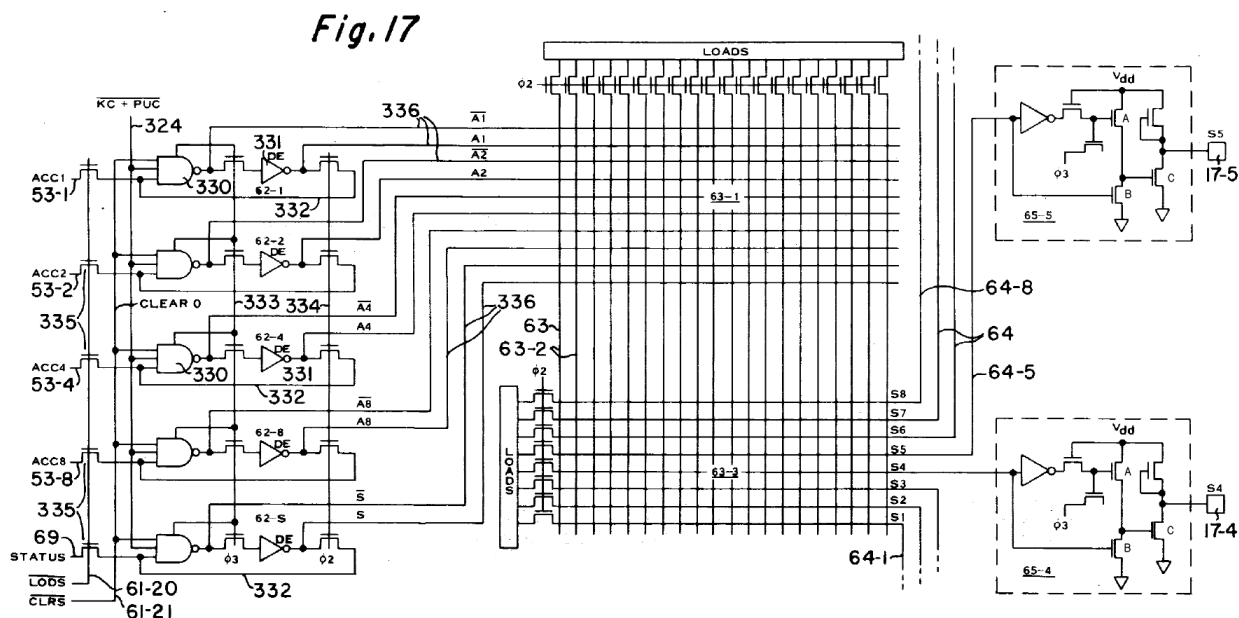


Figure 4.14: S-outputs

4.8.1 Purpose

The S-outputs (called the O-outputs in production) are data-oriented outputs. They allow the value stored in the accumulator and status latch to be outputted, through a PLA.

The S in S-outputs stands for 'segment', as they can be used to send segment data to a seven-segment display.

4.8.2 Inputs

Inputs consist of control lines $\overline{CLRS}$, $\overline{KC + PUC}$, and $\overline{LODS}$; data lines $ACC1$ through $ACC8$ and $STATUS$ line 69; and timing signals $\phi 2$ and $\phi 3$.

Breakdown

$ACC1$ through $ACC8$ contain the accumulator value. $STATUS$ line 69 contains the status latch value.

$\overline{KC + PUC}$ is keyboard or power-up clear.

$\overline{LODS}$ carries a 'load accumulator and status latch into S-outputs' microinstruction.

$\overline{CLRS}$ carries a 'clear S-outputs' microinstruction.

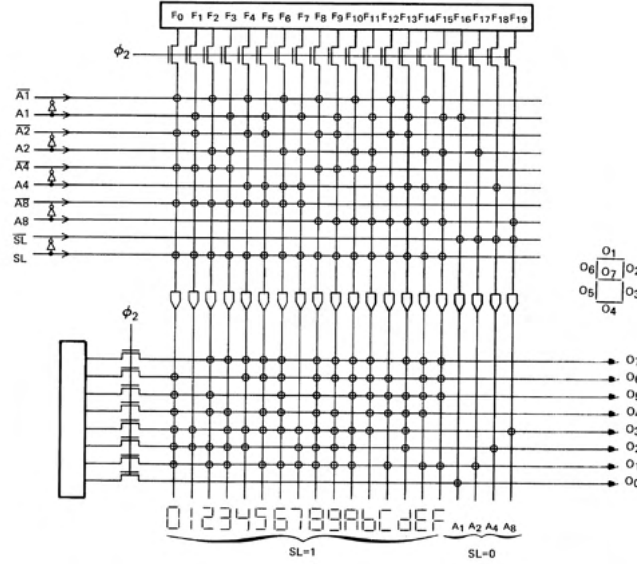


Figure 4.15: Output PLA in use as a seven-segment decoder [29, 2-21]

4.8.3 Outputs

The outputs of the S-output circuit are the S-output pins *S1* through *S8*, on the exterior of the TMS 1000 die.

4.8.4 Function

We want to be able to load values into the S-outputs, and have output pins *S1* through *S8* correspondingly turned on and off, so that we can output the value we loaded in.

Values are loaded in and stored on an O-output register, and outputted to the pins through a series of buffers that ensure that a clean output is achieved.

In between these components is a programmable logic array, which is an idiosyncratic feature of the TMS 1000 that reflects its design history as a calculator chip.

Register

When we load data into the O outputs, we want to be able to store it there, so we don't have to re-load data in every cycle. This is done by using a variant recirculating shift register for every data input bit.

The input of each latch comes from gate \$330, which implements the expression $\overline{CLRS} \wedge \overline{KC} + PUC \wedge In$, where *In* is the combination input line and feedback line input of latch *n*. As $\overline{CLRS}$ and $\overline{KC} + PUC$ are normally 0, the gate will ordinarily output $\overline{In}$, storing *In* in the latch. If $\overline{CLRS}$ or $\overline{KC} + PUC$ are 0, each latch stage gate outputs 1, setting the stored value to 0.

Each latch's input is active on $\phi3$, and the stored value is recirculated on $\phi2$. Note that these timing signals overlap. Thus, the active period for inputs is between the end of $\phi2$ and the start of $\phi2$. Timing signal $\phi3$ was used here in order to give the

latch inputs a large active period.

On $\phi 2$, the latch's feedback line is connected to In , recirculating the value. If $\overline{LODS}$ is 0, the corresponding $ACCn$ or $STATUS$ line is connected to In , overwriting the stored value. The fact that the feedback line may be active when $\overline{LODS}$ is powered does not seem to be a concern, as unlike many other instructions, $\overline{LODS}$ may be powered for an entire machine cycle: if it is unable to overwrite the stored latch value in the first latch input active period, it will be able to do so on the next.

S-Output PLA

A programmable logic array, or PLA, is used to decode the buffer outputs into outputs. It functions as described in Chapter 2. The inputs of the PLA are the S-output register outputs, in both true and inverted form. The outputs are connected to the S-output buffers.

Texas Instruments had seven-segment calculator displays in mind with the S-outputs, and the PLA was primarily included so that a calculator could be made with just a TMS 1000, a keyboard, and a display. However, it can be put to more creative use as well: for instance, Parker Brothers' "Merlin" electronic game uses the S-outputs to conduct keyboard scans, and to power a speaker (using three lines together for the latter task in order to supply sufficient current).

Each of the lines of both PLA matrices are precharged to 0 on $\phi 2$, and may or may not be discharged to 1 by the PLA's action.

Output Buffers and Pins

We now have the output pin logic values that we want on each of the PLA's output lines. However, if we were to connect these lines directly to the output pins, we would not be able to draw substantial current through them: they are thin, have bad conductivity properties (as p-diffused silicon has higher resistance than a metal conductor), and are only connected to a voltage source on timing signal $\phi 2$.

Thus, in between the PLA and the output pins, resides the 'output buffers'. The eight output lines of the PLA are connected to buffers \$65-1 through \$65-8 (of which only two, \$65-4 and \$65-5, are shown).

Each of these buffers may be considered a basic voltage amplification circuit. It uses two NOT-gate-like structures, of which the second is connected to each output pin.

If a 1 is received by an output buffer, transistor B will be inactive, and via a NOT gate, transistor A will activate. This will charge the gate of transistor C with V_{DD} , activating it, and charging the output pin with V_{SS} .

If a 0 is received by an output buffer, transistor A will be inactive, and transistor B will be active. This will charge the gate of transistor C with V_{OL} , deactivating it, and allowing the output pin to connect to, and charge to, V_{DD} .

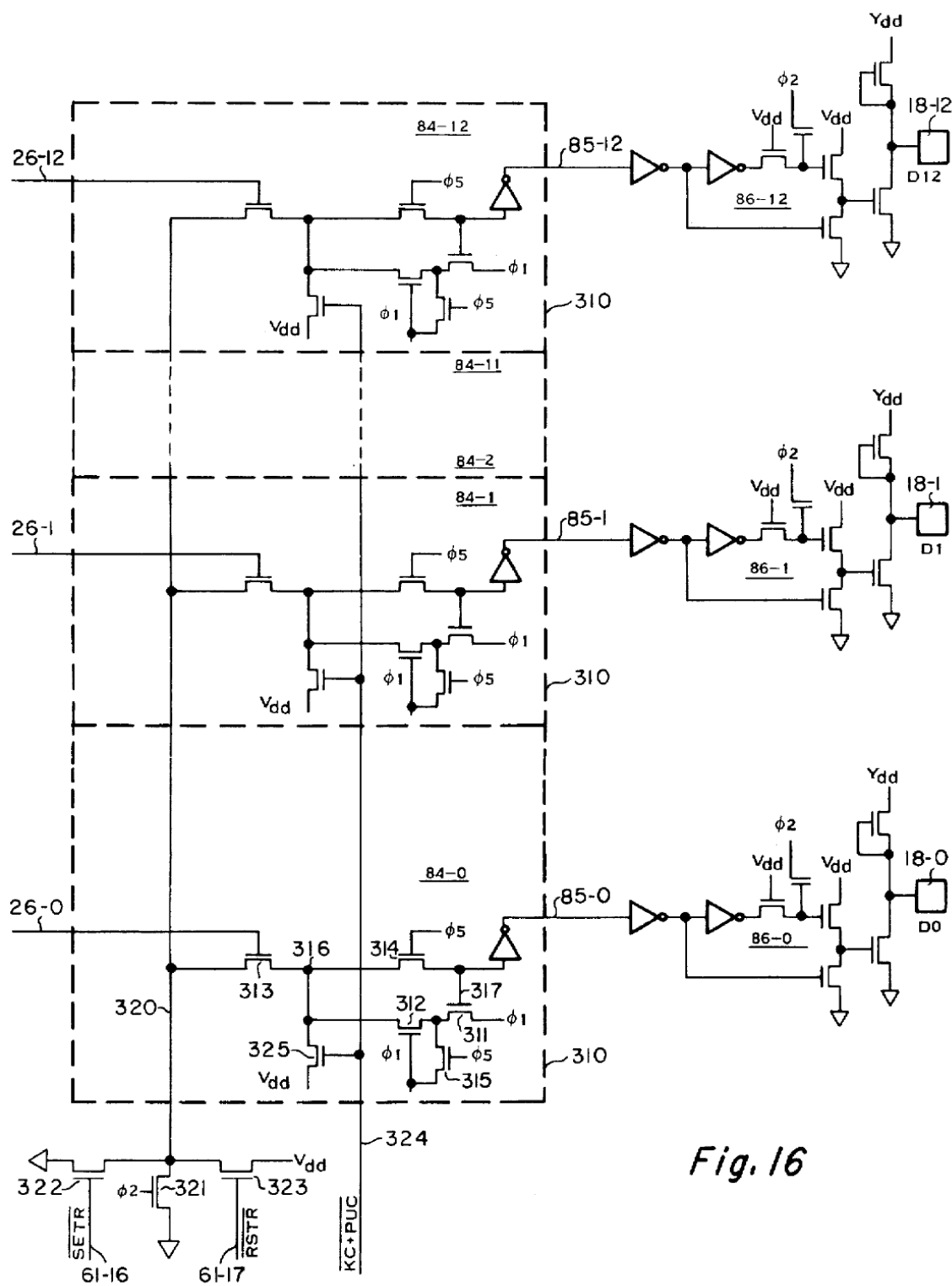


Fig. 16

Figure 4.16: D-outputs

4.9 D-outputs

4.9.1 Purpose

The D-outputs (known as the R-outputs in production) provide directly-addressable output lines, which are of use in keyboard scanning and controlling peripheral devices.

4.9.2 Inputs

Inputs consist of three control lines: $\overline{SETR}$, $\overline{RSTR}$, $\overline{KC + PUC}$; thirteen address lines, \$26-0 through \$26-12; and timing signals $\phi 1$, $\phi 2$, and $\phi 5$.

$\overline{SETR}$ represents a ‘set R output to 1’ microinstruction.

$\overline{RSTR}$ represents a ‘set R output to 0’ microinstruction.

$\overline{KC + PUC}$ represents a ‘clear/reset’ signal, from the power-up and clear circuit.

Address lines \$26-0 through \$26-12 control which D-output is set. This is just like the row select lines of the ROM and RAM arrays that we have already discussed: the RAM/ROM word decoder decodes the value stored in the Y-register, and powers the corresponding address line. Only one address line will ever be powered per instruction cycle.

We may note that these are the same lines, outputted by the ROM/RAM word decoder, that are used in RAM addressing. However, note that there are only 13 D-outputs (of which some packages only make accessible 11), but 16 Y-register address lines. Thus, if the Y-register is larger than 13, none of the lines that the D-outputs ever see will be powered.

4.9.3 Outputs

Outputs consist of thirteen pins, $D0$ through $D12$, on the exterior of the TMS 1000.

4.9.4 Function

Like the S-outputs, the D-outputs consist of a register, where values to be outputted are stored, and a buffer, to amplify the output voltage and current.

Each register stage functions similarly to a RAM cell. In brief: lines \$26-0 through \$26-12 function as the memory cell *ADDRESS* line, opening transistor \$313 to connect input line \$320 to capacitor \$316 and capacitive gate \$317, allowing a value to be stored. On $\phi 5$, capacitor \$316 is connected to capacitive gate \$317. Additionally, on $\phi 5$, the region between transistor \$311 and \$312 is precharged to ground. If the stored bit is 0, transistor \$317 is open, so that on $\phi 1$, transistor \$312 connects the V_{dd} side of $\phi 1$ to capacitor \$316. If the stored bit is 1, transistor \$317 is closed, so that on $\phi 1$, transistor \$312 connects capacitor \$316 to the ground-precharged region.

One of the primary differences from the RAM cell design is that there is a separate output line, connected to capacitive gate \$317. Each output line goes into circuit \$86, which functions very similarly to the S-output buffer circuit. It electrically insulates

the output pins from the rest of the circuit, amplifies the voltage, and allows for greater current output. Circuit §86 directly connects to the D-output pins.

When $\overline{KC} + \overline{PUC}$ is powered, transistor §325 opens, connecting capacitor §316 of each stage to DD , setting the stored value to 0.

When $\overline{SETR}$ is 0, transistor §322 connects line §320 to ground, discharging it to 1. When $\overline{RSTR}$ is powered, transistor §323 connects line §320 to DD , charging it to 0. On timing signal ϕ_2 , §320 is discharged to ground through §321, preventing residual charge from accumulating and causing problems.

4.10 X Register

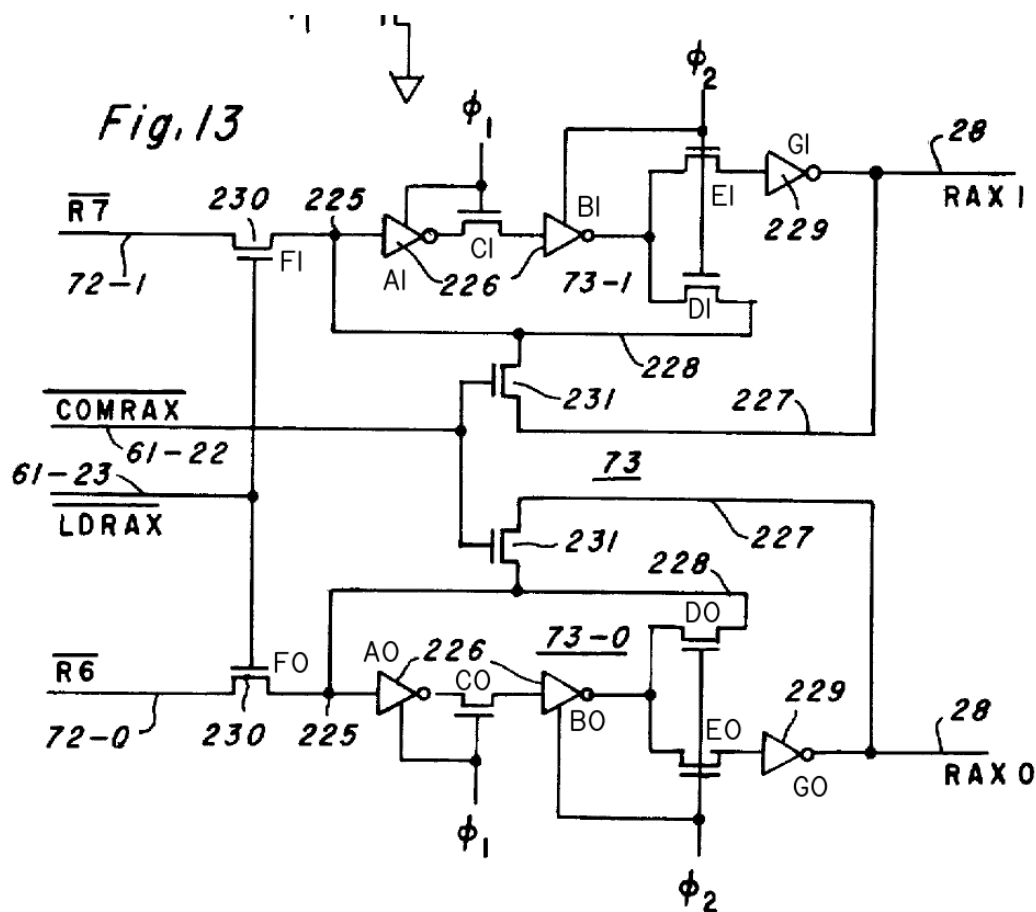


Figure 4.17: X-register

4.10.1 Purpose

The X-register stores a two-bit address (from 0 to 3) used to select RAM cells to read and write to.

4.10.2 Inputs

Inputs consist of two control lines, $\overline{LDRAX}$ and $\overline{COMRAX}$; two data lines, $\overline{R6}$ and $\overline{R7}$; and two timing signals, $\phi1$ and $\phi2$.

Breakdown

$\overline{LDRAX}$ represents a ‘load value into X-register’ microinstruction, and allows values to be loaded into the X-register from ROM.

$\overline{COMRAX}$ represents a ‘complement X-register value’ microinstruction, and inverts the bits stored in the X-register,

The data lines, $\overline{R6}$ and $\overline{R7}$, contain the sixth and seventh bits of the current ROM word, in inverted form.

4.10.3 Outputs

Outputs consist of two address lines, $RAX0$ and $RAX1$, which contain the respective values of each of the X-register bits.

4.10.4 Function

The X-register stores a two-bit value using two identical register stages. On each stage, NOT gates A and B, together with transistors C and D and feedback line \$228 form the typical latch design which appeared in Chapter 2. Timing signal $\phi2$, which activates NOT gates B0 and B1, also opens transistors E0 and E1, allowing the stored value of the register to be outputted on lines $RAX0$ and $RAX1$. Of note, due to NOT gates G0 and G1, the output is inverted compared to the value stored in the register. This is why the input data lines are connected to the inverted forms of the relevant ROM bits.

When $\overline{LDRAX}$ is powered, transistors F0 and F1 are opened, allowing new values to be loaded into the register from $\overline{R6}$ and $\overline{R7}$ from timing signal $\phi3$ to timing signal $\phi1$.

When $\overline{COMRAX}$ is powered, an additional feedback line is opened, from the inverted output line of each latch to the same latch stage’s input. If powered after timing signal $\phi2$, NOT gate G’s output will overwrite the recirculated stored value, inverting it.

Of note, the $\overline{PUC}$ and $\overline{KC}$ lines have no effect on the X-register. Thus, when the system is reset or turned on, it must be manually set to a known value with a $\overline{LDRAX}$ instruction.

Fig.12

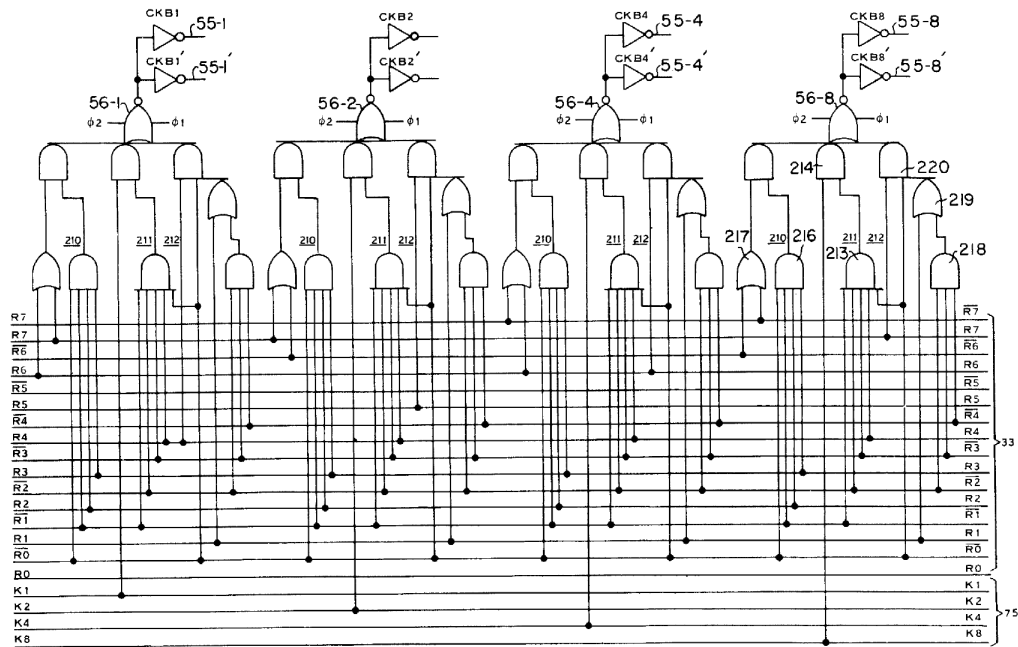
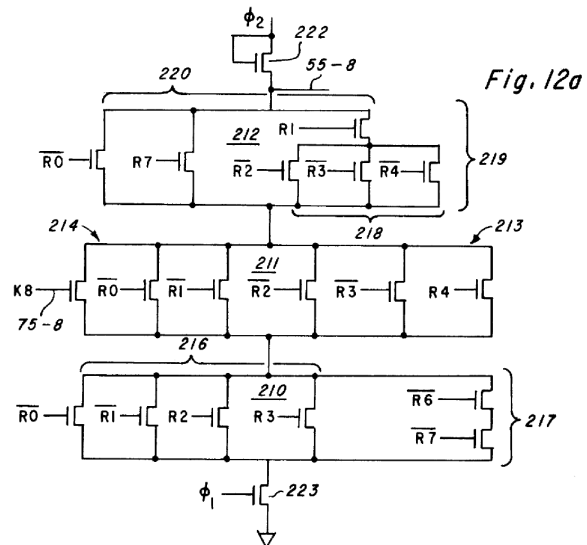


Figure 4.18: The CKB multiplexer

Figure 4.19: A close-up of the rightmost gate, which outputs $CKB8$

4.11 CKB logic

4.11.1 Purpose

The CKB logic serves as a multiplexer, and a kind of secondary instruction decoder. It makes microinstructions across the TMS 1000 much more versatile, at the cost of making microprogramming less versatile.

Instruction opcode	CKB output
0x00-0x07	ROM bits 4-7
0x08-0x0F	K-inputs
0x10-0x2F	0
0x30-0x3B	Bit selection “bitmask”
0x3C-0x3F	0
0x40-0x7F	ROM bits 4-7

Table 4.2: CKB logic behavior

We have already seen how the CKB multiplexer allows for additional functionality with bitwise RAM operations. Here is another example:

Recall that microinstruction $\overline{CKN}$ loads the CKB output into the adder. If the current instruction opcode is between 0x08 and 0x0F, this will load the K-inputs into the adder. If the instruction opcode is between 0x40 and 0x7F, it will load a constant into the adder. Thus, the same microinstruction, without additional circuitry, can accomplish two very different tasks. Microinstruction $\overline{CKN}$ can then be used by, for instance, both the TKA (transfer K-inputs to accumulator) and TCY (transfer constant to Y-register) instructions.

4.11.2 Inputs

Inputs consist of 20 data lines: true and inverted ROM lines $R0$ through $R7$, and $\overline{R0}$ through $\overline{R7}$, and K input lines $K1$, $K2$, $K4$, $K8$; and timing signals $\phi1$ and $\phi2$.

Breakdown

K inputs $K1$ through $K8$ carry the value inputted on the TMS 1000’s four input pins.

4.11.3 Outputs

The outputs consist of eight data lines: $CKB1$, $CKB2$, $CKB4$, and $CKB8$, which contain each of the bits of the current CKB output; and duplicate lines $CKB1'$, $CKB2'$, $CKB4'$, and $CKB8'$.

4.11.4 Function

We want it so that instructions with opcodes between 0x00 and 0x07 and between 0x40 and 0x7F cause the last ROM bits to be outputted; that instructions with opcodes between 0x08 and 0x0F cause the K-inputs to be outputted; that instructions between 0x30 and 0x3B will cause the CKB logic to decode the last bits of the ROM word into a selected bit address; and that all other instructions will cause the CKB output to be 0.

The CKB logic is implemented in four nearly identical complex gates, 56-1 through 56-8. Each consists of three complex AND gates, which are OR'd together.⁵ Thus, for each stage, if either component gate \$210, \$211, or \$212 is 1, 1 will be outputted, and 0 will be outputted otherwise. Each one of these AND gates decodes bits from the ROM, such that at most one of them will be 1 for a given ROM word.

AND gate \$211 of each stage n implements the expression $Kn \wedge (\overline{R0} \wedge \overline{R1} \wedge \overline{R2} \wedge \overline{R3} \wedge \overline{R4})$. Thus, if the first five ROM bits are 00001 (if the current ROM word is between 0x08 and 0x0F), gate \$214 will output the value of corresponding K-input line Kn . Otherwise, it will output 0.

AND gate \$212 of each stage n implements the expression $Rn \wedge (\overline{R0} \wedge (R1 \vee (\overline{R2} \wedge \overline{R3} \wedge \overline{R4})))$. Thus, if the first two ROM bits are 01 (if the current ROM word is between 0x40 and 0x7F) or the first five ROM bits are 00000 (if the current ROM word is between 0x00 and 0x07), gate 212 will output the value of the corresponding ROM bit Rn . Otherwise, it will output 0.

AND gate \$210 combines the logic used in the X-register decoder with an instruction decoder, in order to handle bit addressing for *SBIT* and *RBIT* instructions. For each stage n , it implements the expression $(R6 + R7 == n) \wedge (\overline{R0} \wedge \overline{R1} \wedge R2 \wedge R3)$. Thus, if the first four ROM bits are 0011 (if the current ROM word is between 0x30 and 0x3B), the gate decodes $R6$ and $R7$, such that only one CKB line will be powered, corresponding to the value of $R6$ and $R7$. Gate \$210-1 will be 1 when $R6$ and $R7$ are both 1; Gate \$210-2 will be 1 when $R6$ is 0 and $R7$ is 1; Gate \$210-3 will be 1 when $R6$ is 1 and $R7$ is 0; and finally Gate \$210-4 will be 1 when $R6$ and $R7$ are both 0.

4.12 Status Logic

4.12.1 Purpose

The status logic generates a signal (0 or 1) on the status line, allowing *BRANCH* and *CALL* instructions to conditionally execute. The status latch allows a status value to be stored and outputted.

4.12.2 Inputs

Inputs consist of four control lines, $C8$, NE , $STSL$, and $\overline{KC}$; two data lines, $\overline{CAR8}$ and $COMP$; and three timing signals, $\phi1$, $\phi2$, and $\phi3$.

⁵Strictly speaking, they are NOR'd together, with the output inverted.

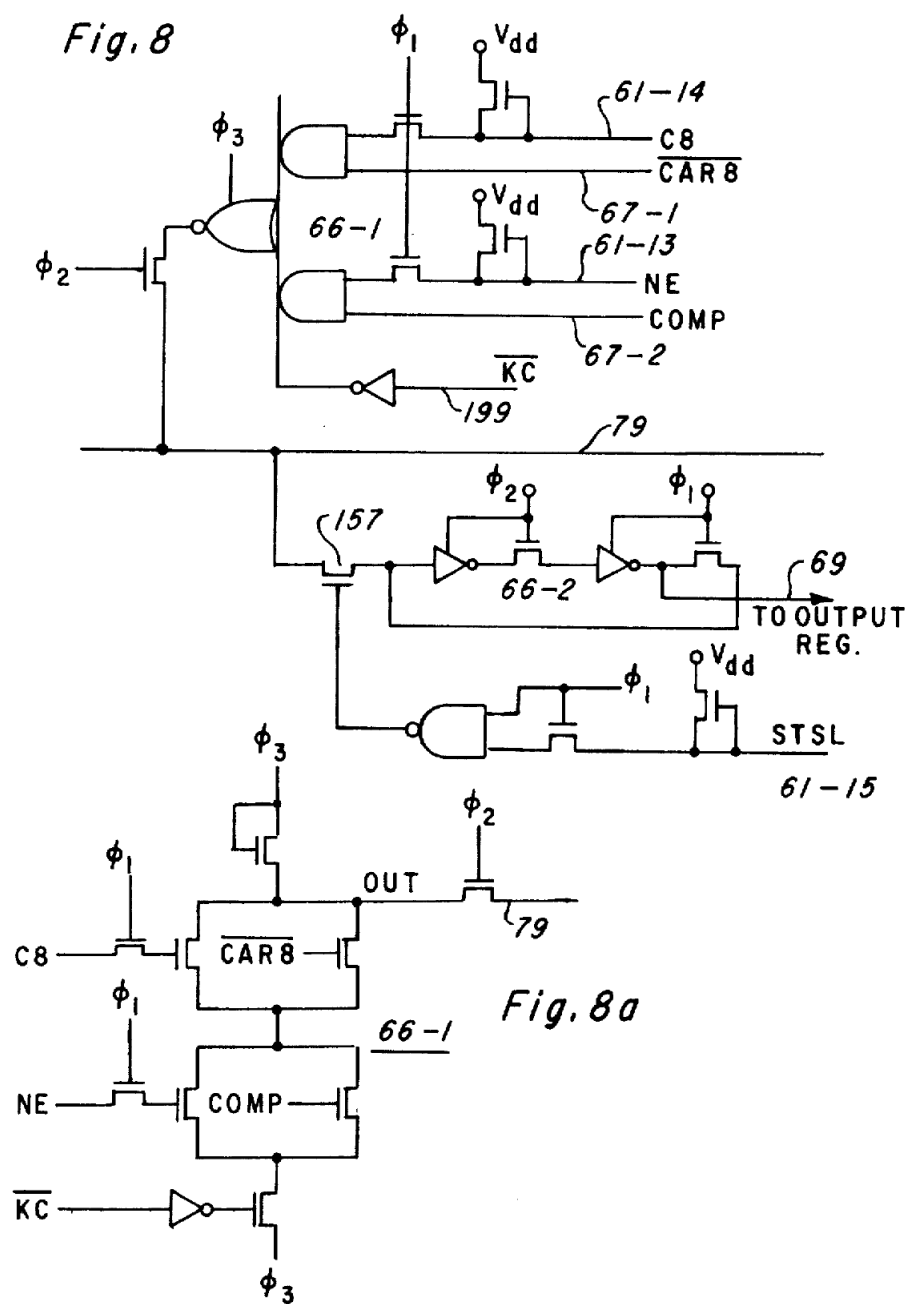


Figure 4.20: Status logic

C8 represents a ‘send carry to status’ microinstruction, which sets the status to 1 if the adder overflowed, and 0 otherwise.

NE represents a ‘compare adder inputs’ microinstruction, which sets the status to 0 if the adder inputs are identical, and 1 otherwise.

STSL represents a ‘store status in status latch’ microinstruction, which sets the status latch value equal to the current status value.

$\overline{KC}$ represents a keyboard clear instruction.

$\overline{CAR8}$ is the inverse of the carry from the adder.

COMP represents a comparison of the adder inputs, and is set to 1 if they are equal, and 0 otherwise.

4.12.3 Outputs

Outputs consist of two data lines, *STATUS* lines \$79 and \$69.

STATUS line \$79 outputs the current status value, and is used in logic determining if a branch or call instruction executes or not.

STATUS line \$69 outputs a saved status value, and is connected to an S-output buffer, where it may be outputted along with the value stored in the accumulator.

4.12.4 Function

Status Function

We want the status to be 1 for most instructions. However, if a *C8* microinstruction is active, we want the status to be equivalent to the adder carry value, and if an *NE* microinstruction is active, we want the output to be the inverse of the *COMP* value.

This functionality is implemented by complex gate \$66-1. It implements the boolean expression $(C8 \wedge \overline{CAR8}) \vee (NE \wedge COMP) \vee KC$. Thus, it will output 0 on *STATUS* line \$79 if a *KC* signal is received, if a *NE* instruction is active and *COMP* is 1, or if a *C8* instruction is active and $\overline{CAR8}$ is 1, and 1 otherwise.

On timing signal $\phi3$, voltage is applied to the gate, precharging the OUT region to V_{DD} . When $\phi3$ deactivates, the OUT region becomes electrically isolated.

On timing signal $\phi1$, *C8* and *NE* charge their respective gates. If enough transistors are opened, so that OUT is electrically connected to the ground end of $\phi3$, then OUT discharges, and on $\phi2$, the gate will output 1. If no path to ground is opened, then OUT remains charged at V_{DD} , and 0 is outputted on $\phi2$.

Status Latch Function

The status latch follows the typical latch design discussed in Chapter 2, with its input coming from *STATUS* line \$79, and its output going to the *STATUS* S-output register stage. The input is active on timing signal $\phi2$, and the output is active on timing signal $\phi1$. The *STSL* control line, if powered, connects *STATUS* line \$79 to the input of the status latch. A NAND gate, precharged by *STSL* on $\phi1$, prevents the latch input from being connected at the same time that the feedback line is active.

4.13 Address Controls

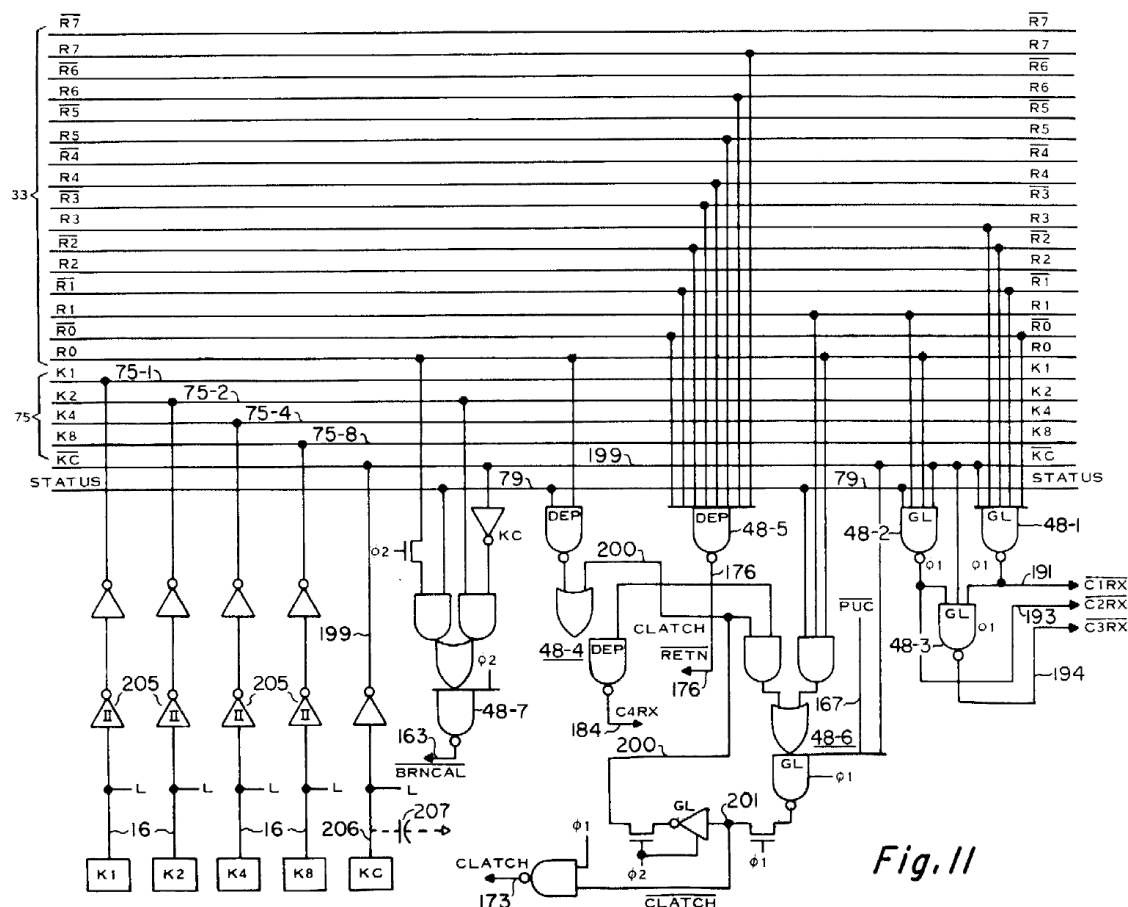


Figure 4.21: Address Controls

4.13.1 Purpose

Almost all TMS 1000 instructions are decoded by the instruction decoder. As we mentioned in that section, the primary exceptions are program flow commands, which are handled by the address controls. The address controls decode **BRANCH**, **CALL**, **RETN** and **LDP** commands.

Additionally, this is where the K-input lines are connected to the K-input pins. This function is basically independent from the function of the address control logic.

4.13.2 Inputs

Inputs consist of control line $\overline{PUC}$; seventeen data lines: *STATUS* line \$79, and true and inverted ROM lines *R0* through *R7* and $\overline{R0}$ through $\overline{R7}$; five input pins, K1, K2, K4, K8, and KC; and timing signals $\phi 1$ and $\phi 2$.

Breakdown

$\overline{PUC}$ represents a ‘power up clear’ command, and is outputted by the power up and clear circuit at startup.

$STATUS$ line \$79 is generated by the Status Logic circuit.

K input pins K1, K2, K4, K8, and KC are external pins of the TMS 1000.

4.13.3 Outputs

Outputs consist of six control lines: $\overline{BRNCAL}$, $\overline{RETN}$, $\overline{C1RX}$, $\overline{C2RX}$, $\overline{C3RX}$, and $C4RX$; and five data lines: $CLATCH$, and K inputs $K1$, $K2$, $K4$, $K8$.

Breakdown

$\overline{BRNCAL}$ represents a ‘branch/call’ microinstruction. We want to power it on BR and CALL instructions.

$\overline{RETN}$ represents a ‘return’ microinstruction. We want it to be powered on RETN instructions.

$\overline{C1RX}$, $\overline{C2RX}$, $\overline{C3RX}$, and $C4RX$ control the page address buffer and the page address register, which store the ROM page address.

$\overline{C1RX}$ functions as a ‘load value from ROM into page buffer’ control signal, or a ‘LDP’ microinstruction. We want it to be powered on LDP instructions.

$\overline{C2RX}$ functions as a ‘load page address into page buffer’ control signal, effectively saving it so that long ‘CALL’ subroutines can be returned from. We want it to be powered on CALL instructions.

$\overline{C3RX}$ functions as a ‘recirculate page buffer register’ instruction, and should be on whenever the page buffer is not being actively modified. We want it to be powered whenever LDP and CALL aren’t being called.

$C4RX$ functions as a ‘load value from page buffer into page register’ control signal. We want it to be powered on BR, CALL, and RETN instructions.

$CLATCH$ contains a bit representing whether or not the program is currently in a function call. We want $CLATCH$ to be 1 if a CALL instruction has been executed, and a RETN instruction has not yet been executed, and otherwise we want it to be 0.

K inputs $K1$, $K2$, $K4$, $K8$ carry signals received from the K-input pins.

4.13.4 Function

Address Control Logic

A series of complex gates ensures that we get the outputs that we want on the right inputs.

Gate \$48-7 outputs onto $\overline{BRNCAL}$. It implements the expression

$$\overline{(R0 \wedge STATUS) \vee (K2 \wedge KC)}$$

Thus, if $\overline{KC}$ is not powered, gate \$48-7’s output is 0 when a BRANCH or CALL instruction is the current ROM word, and $STATUS$ is 1. As part of the TMS 1000’s

test functionality, if $\overline{KC}$ is 1, $\overline{BRNCAL}$ can be powered by powering input line $K2$.

Gate \$48-5 outputs onto control line $\overline{RETN}$. It implements the expression:

$$\overline{R0 \wedge R1 \wedge R2 \wedge R3 \wedge R4 \wedge R5 \wedge R6 \wedge R7}$$

Thus it outputs 0 when the current instruction is 0x0F, that is, a **RETN** instruction.

Gate \$48-1 outputs onto control line $\overline{C1RX}$. Gate \$48-1 implements the expression:

$$\overline{\overline{KC} \wedge \overline{R0} \wedge \overline{R1} \wedge \overline{R2} \wedge R3}$$

Thus, it outputs 0 if the current ROM word is an *LDP* instruction (0x10-0x1F) and $\overline{KC}$ is 0.

Gate \$48-2 outputs onto control line $\overline{C2RX}$.⁶ It implements the expression:

$$\overline{STATUS \wedge \overline{KC} \wedge R0 \wedge R1}$$

Thus, it outputs 0 if the current ROM word is a *CALL* instruction (0xC0 to 0xFF), *STATUS* is 1, and $\overline{KC}$ is 0.

Gate \$48-3 outputs onto control line $\overline{C3RX}$. Gate \$48-3 NANDs together $\overline{KC}$ and the outputs of gate \$48-1 and \$48-2. Thus, its output is 0 if the instruction word is neither *CALL* nor *LDP*.

Gate \$48-4 outputs onto control line $\overline{C4RX}$. It implements the expression:

$$\overline{\overline{RETN} \wedge ((\overline{STATUS \wedge R0}) \vee \overline{CLATCH})}$$

Thus, \$48-4 outputs 1 if *RETN* is the current instruction, or if the call latch is 0 and *STATUS* and *R0* are both 1. Thus, it outputs 1 if a *RETN* instruction is executed, or if a *BRANCH* or *CALL* instruction is executed and the program isn't in a function call.

Call Latch

The call latch uses a variant of the typical recirculating shift register design discussed in Chapter 2. Input comes into gate \$48-6 on $\phi1$ and is inverted, charges output gate \$173 (also clocked on $\phi1$), and is fed back to the input on $\phi2$.

Gate \$48-6 implements the expression:

$$\overline{\overline{KC} \wedge \overline{PUC} \wedge ((STATUS \wedge R0 \wedge R1) \vee (CLATCH \wedge \overline{RETN}))}$$

Thus, if $\overline{KC}$ and $\overline{PUC}$ are 0, \$48-6 outputs 0 if either *CALL* is the current instruction and *STATUS* is 1, or if the call latch is 1 and the current instruction is *RETN*. Otherwise, it outputs 1. Thus, if a *CALL* instruction is successfully executed, *CLATCH* becomes 1, and will continue being 1 until *RETN* sets it to 0, or if a keyboard or power-up-clear signal resets the computer.

⁶The patent application contains a typo regarding this.

Keyboard Inputs

Each K-input pin is connected to its corresponding K-input line. Two measures are in place to ensure that all inputs are valid.

The L lines on the diagram are loads, connected to DD through a depletion-load transistor. This means that when no signal (neither 0 nor 1) is applied to an input pin, the voltage will be pulled down to logical 0, ensuring that the K-inputs are always a valid value.

Gates \$205 are Schmitt triggers, which are used in converting analog signals to digital ones, ensuring that the outputted signal will be a clean 0 or 1.

A second set of NOT gates inverts the signals back to their true forms, except for the input from the keyboard-clear pin, which is connected in its inverted form to $\overline{KC}$.

4.14 Page Address and Page Buffer Registers

4.14.1 Purpose

The page address register holds the current ROM page address, and is used in selecting ROM outputs.

The page buffer register holds a stored ROM page address. This is either a new page address, loaded in by a LDP (load page) instruction, or the page to return to after a long CALL.

We want to be able to set the page buffer (with LDP instructions), to switch the page buffer and page address (as part of CALL instructions), or to move the page buffer into the page address without saving the latter (as part of RETN and BR instructions). This is done with several specialized address control microinstructions.

4.14.2 Inputs

Inputs consist of address control microinstructions $\overline{C1RX}$, $\overline{C2RX}$, $\overline{C3RX}$, and $C4RX$; $\overline{KC}$; ROM lines $R0$ through $R7$ and K inputs $K1$, $K2$, $K4$, $K8$, coming from the main data bus; and timing signals $\phi1$ and $\phi2$.

Breakdown

$\overline{C1RX}$ functions as a ‘load value from ROM into page buffer’ control signal, or a ‘LDP’ microinstruction. It is powered on LDP instructions.

$\overline{C2RX}$ functions as a ‘load page address into page buffer’ control signal, effectively saving it so that long ‘CALL’ subroutines can be returned from. It is powered on CALL instructions.

$\overline{C3RX}$ functions as a ‘recirculate page buffer register’ instruction, and should be on whenever the page buffer is not being actively modified. It is powered whenever LDP and CALL aren’t being called.

$C4RX$ functions as a ‘load value from page buffer into page register’ control signal. It is powered on **BR**, **CALL**, and **RETN** instructions.

4.14.3 Outputs

Outputs consist of PA output lines \$35-1 through \$35-8, which are used in ROM addressing.

4.14.4 Function

Inputs $\overline{C1RX}$, $\overline{C2RX}$, $\overline{C3RX}$, and $C4RX$ are all clocked on $\phi 1$

The page buffer and page address registers are each composed of four recirculating shift registers, as discussed in Chapter 2, with inputs active on $\phi 1$ and feedback lines active on $\phi 2$.

We will use PAn to refer to the value stored in the page address register of stage n , and PBn to refer to the value stored in the page buffer register of stage n .

The page buffer register stages, which are located above the page address register stages on the diagram, get their input from each stage’s respective complex gate \$47, which implements:

$$\overline{(PBn \vee \overline{C3RX}) \wedge (PAn \vee (\overline{C2RX} \wedge \overline{KC})) \wedge (Rn \vee \overline{C1RX})}$$

The symbol Rn designates the ROM line attached to page buffer stage n ($R4$ for stage 1, through $R7$ for stage 4). Note that the logic in the address controls that generates $\overline{C1RX}$, $\overline{C2RX}$, and $\overline{C3RX}$, ensures that only one is at 0 at a time. Thus, the value inputted to stage n ’s page buffer is PBn if $\overline{C3RX}$, PAn if $\overline{C2RX}$ (and no KC signal is being received), and Rn if $\overline{C1RX}$. Gate \$47 is clocked on $\phi 3$, so its output is active on both $\phi 1$ and $\phi 2$.

The page address register stages receive their input from each stage’s respective complex gate \$46, which implements:

$$\overline{(PAn \vee C4RX) \wedge (PBn \vee \overline{C4RX})}$$

Thus, if $C4RX$ is 1, gate \$46’s output is PBn , and if $C4RX$ is 0, gate \$46’s output is PAn . Whatever value is outputted, will be stored in the page address register.

Additionally, $\overline{PUC}$ activates transistors \$190 and \$182 in each Page Buffer and Page Address stage respectively, connecting the inverted part of each register to ground, and so setting the output of every register stage to 1. This means that when the TMS 1000 powers up, it will start executing instructions on ROM page 1111, or in decimal, page 15.

4.15 Program Counter and Subroutine Return

4.15.1 Purpose

The program counter stores the address of the current ROM word. It consists of eight identical stages, each of which stores one bit and of which six are used in

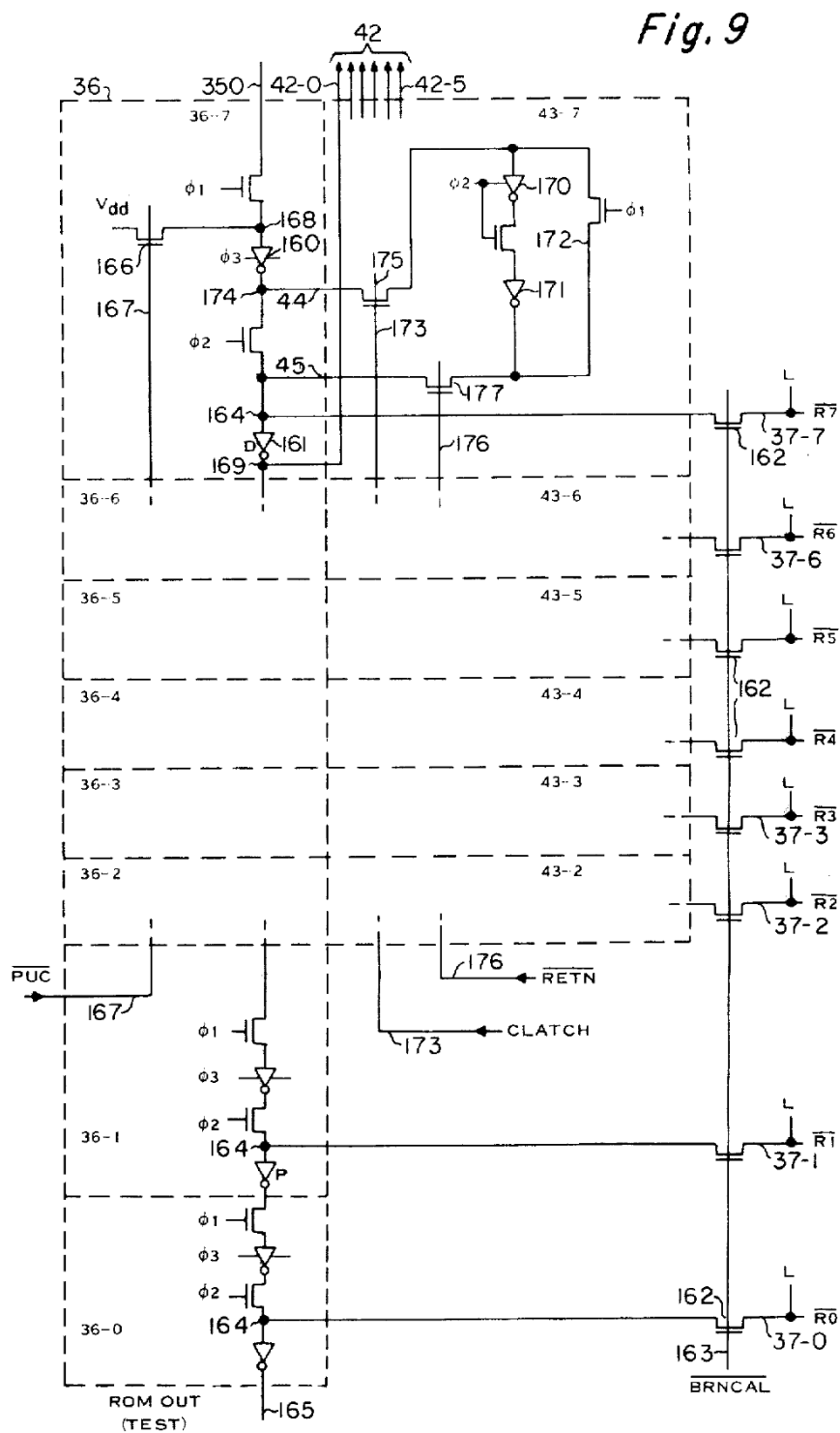


Figure 4.23: Program counter

normal operation, as the ROM word address is six bits long (storing a value from 0 to 63). The remaining two stages are used in the test functionality of the TMS 1000.

We want the program counter to increment every machine cycle, so that the next instruction in the program will be loaded. Thus, we need it to cycle through its possible values. In order to achieve this effect, the program counter is a true shift register, so that every cycle, each bit is shifted ‘up’ one stage. A feedback circuit feeds in a new bit to the first stage every cycle.

When a **CALL** instruction is executed, we need to save the previous program counter value, so that we can return to it. This is done with the subroutine register. Each program counter stage has a corresponding subroutine register stage, which can store the value of the bit in its associated program counter stage when a **CALL** is executed.

4.15.2 Inputs

Inputs consist of three control lines: $\overline{BRNCAL}$, $\overline{RETN}$, and $\overline{PUC}$; nine data lines: inverted ROM lines $\overline{R0}$ through $\overline{R7}$ and $CLATCH$; and timing signals $\phi1$, $\phi2$, and $\phi3$. Additionally, feedback line \$350 provides inputs to the first stage of the program counter, with a signal generated from the program counter output.

Breakdown

$\overline{BRNCAL}$ represents a ‘branch/call’ microinstruction, which overwrites the program counter with a value from ROM to allow it to jump to another part of the program.

$\overline{RETN}$ represents a ‘return’ microinstruction, which restores the program counter to its previous value after a **CALL** instruction.

$\overline{PUC}$ represents a ‘power up and clear’ command, which zeroes out the value stored in the program counter.

$CLATCH$ carries the value of the call latch, and will be 1 if a **CALL** instruction is being or has been executed and a **RETN** has not been executed, and 0 otherwise.

$\overline{R0}$ through $\overline{R7}$ carry the inverted values of each ROM bit.

4.15.3 Outputs

Outputs consist of seven data lines: Lines \$42-0 to \$42-5, and Line \$165.

Breakdown

Lines \$42-0 to \$42-5 output the current program counter value to the ROM/RAM word decoder, where they will be used to select an instruction for the ROM to output. Lines \$42-0 to \$42-5 are also used by the feedback circuit. We will refer to these lines as $PC0$ through $PC5$.

Line \$165 is used in the chip’s test functionality, and allows the stored ROM program to be serially read out via S-output 8. This line is $PC7$ on the power-up clear circuit diagram.

4.15.4 Function

Program Counter Function

The program counter saves our place in the program as a 6-bit number that can be decoded to a ROM word. We must be able to change the program counter value with address control instructions, such as **BRANCH** and **CALL**. Additionally, the program counter value must update every cycle, so that the next instruction in the program is outputted by the ROM every machine cycle. The latter function is implemented by shifting every bit up every machine cycle, and by feeding in a new bit to the first program counter stage through a feedback circuit.

Each program counter stage functions as a shift register, from its input line to its output line.

On $\phi 1$, the stage's input line is connected to NOT gate \$160, charging the gate. On $\phi 3$ (which overlaps with $\phi 1$ and $\phi 2$), NOT gate \$160 is activated. The region between NOT gate \$160 and the transistor clocked on $\phi 2$ will become charged according to the gate. On $\phi 2$, a transistor connects that charged region with the output line, where unclocked NOT gate \$161 inverts it a second time, converting the input back to its original form.

When $\overline{PUC}$ is 0, DD is connected to the input of NOT gate \$160 on each stage, setting each stage to 0. The Programmer's Manual states that keyboard clear signals also zero out the program counter, suggesting that in production, $\overline{PUC}$ was replaced by $\overline{KC + PUC}$.

When $\overline{BRNCAL}$ is 0, $\overline{R0}$ through $\overline{R7}$ are connected to each program counter stage in the region between the transistor clocked on $\phi 2$ and NOT gate \$161. Because of NOT gate \$161, the program counter stage will output the true (non-inverted) value of the corresponding ROM bit.

Subroutine Register Function

We want the subroutine register to save the location in the program, represented by a program counter value, to return to after a subroutine call, when a subroutine call is executed.

Each subroutine register stage is a simple latch, of the arrangement discussed in Chapter 2. The input is active on $\phi 2$, and the feedback line is active on $\phi 1$. When $CLATCH$ is 0, the output of NOT gate \$160 is connected to the subroutine register stage input. When this is the case, the value in the subroutine register will be equivalent with the value in the program counter. When $CLATCH$ becomes 1, the subroutine register stage is cut off, and will recirculate the last program counter value before $CLATCH$ became 1.

As $CLATCH$ will become 1 when a **CALL** instruction is executed, the stored value will be the word address of the next instruction after the executed **CALL** instruction. This is the location in the program that we want to return to after a subroutine call finishes.

When $\overline{RETN}$ is 0, the output of each subroutine register stage is connected to the corresponding program counter stage by transistor \$177, in the same region where

ROM bits are loaded in. This replaces the program counter value with the stored subroutine register value.

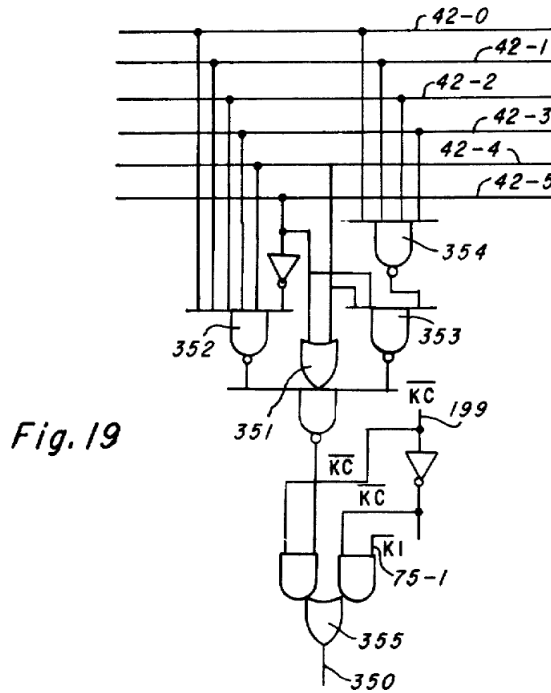


Figure 4.24: Program counter feedback circuit

Feedback Circuit function

A complex series of logic gates generates an input bit for the program counter from the output of the program counter. The purpose of this is to sequentially step through instructions, so that every ROM word address is entered into the program counter in a predictable, defined order, and that one ROM address will not appear twice until all 63 other ROM addresses have appeared.

The basic idea is to implement an inverted logical XOR on the top two lines of the program counter, so that it outputs 1 if $PC4$ and $PC5$ are both 0 or both 1, and to output 0 if they differ. However, this would cause the program counter to be unable to reach the value 111111 from 000000, as it would output 0 when 111110 is reached, and would cause it to get stuck on 111111, as this would cause it to perpetually output 1. Thus, additional logical gates correct these two problems.

The feedback circuit is primarily implemented by Gate \$351. For the most part, \$351 implements the expression:

$$\overline{(PC4 \vee PC5) \wedge \overline{PC4} \wedge \overline{PC5}}$$

This is the inverted XOR gate. However, it's full functionality is much more complex, to account for the places where the inverted XOR breaks down.

To allow 111111 to be reached from 111110, gate \$352 is included in NAND gate \$351. Gate \$352 implements the expression:

$$\overline{PC0 \wedge PC1 \wedge PC2 \wedge PC3 \wedge PC4 \wedge \overline{PC5}},$$

It will output 0 if the program counter reads 111110, and 1 otherwise. Thus, if the program counter is 111110, gate \$352 will output 0, and gate \$351 will output 1, causing 111111 to be loaded into the program counter.

To force 111111 to output a 0, gate \$354 is included in the NAND gate \$353. Gate \$353 now implements the expression:

$$\overline{((\overline{PC0 \wedge PC1 \wedge PC2 \wedge PC3}) \wedge PC4 \wedge PC5)},$$

Gate \$353 NANDs $PC4$ and $PC5$, unless $PC0$ through $PC3$ are all 1, in which case it always outputs 0. Thus, if the program counter is 111111, gate \$353 will output 0.

Thus, the program counter steps through instructions in a nonlinear, but deterministic and ordered fashion, eventually reaching every possible program counter value.

Step	PC value	Feedback circuit output
0	000 000 (0)	
1	000 001 (1)	1
2	000 011 (3)	1
3	000 111 (7)	1

Table 4.3: Progression of program counter values, starting from 0

The additional gate \$350 is used in the test functionality of the TMS 1000. It implements the expression $(\overline{KC} \wedge \$351) \vee (KC \wedge K1)$. Thus, if a KC is not being generated, gate \$355 will output the feedback gate output, and the feedback circuit will function as normal. But if a KC signal is being generated, gate \$355 will output the value of keyboard input pin $K1$.

In short, here are the expressions implemented by each gate. These are presented in their non-minimized form, so that they exactly correspond to the circuit diagram.

\$351:

$$\overline{((\overline{PC0 \wedge PC1 \wedge PC2 \wedge PC3 \wedge PC4 \wedge \overline{PC5}}) \wedge ((\overline{PC0 \wedge PC1 \wedge PC2 \wedge PC3}) \wedge PC4 \wedge PC5) \wedge (PC4 \vee PC5))}$$

\$352:

$$\overline{PC0 \wedge PC1 \wedge PC2 \wedge PC3 \wedge PC4 \wedge \overline{PC5}}$$

\$353:

$$\overline{(PC0 \wedge PC1 \wedge PC2 \wedge PC3) \wedge PC4 \wedge PC5}$$

\$354:

$$\overline{PC0 \wedge PC1 \wedge PC2 \wedge PC3}$$

\$355:

$$(\overline{KC} \wedge 351) \vee (KC \wedge K1)$$

4.16 RAM/ROM Word Decoder and Data Select

4.16.1 Purpose

The RAM/ROM word decoder uses a PLA-like structure to decode the values stored in the program counter and Y-register into a selection of an output line to power, for the purpose of ROM, RAM, and D-output addressing.

This circuit is an ingenious cost-saving measure. By decoding both registers in one circuit, fewer components and less die space is used than two separate decoders. However, that functional complexity also makes it a hard component to understand!

First, we need a way to select which register we are decoding at a time. This is done by the ‘data select’ circuit at the bottom of the diagram. Then, we need a way to separate the outputs. This is done by putting one set of outputs on either side of the decoder array’s row lines. The ROM output is clocked on $\phi 1$, while the Y-register output is clocked on $\phi 2$.

4.16.2 Inputs

Inputs consist of lines \$42-0 through \$42-5, containing the bits of the program counter; lines \$41-1 through \$41-8, which contain the bits of the Y-register; and timing signals $\phi 1$, $\phi 2$, and $\phi 4$.

Recall that timing signal $\phi 4$ is powered twice: once on the first pulse of $\phi 1$, and once on the first pulse of $\phi 2$.

4.16.3 Outputs

Outputs consist of 64 ROM address lines, labeled \$241, and 16 RAM address lines, labeled \$26-0 through \$26-15.

4.16.4 Function

Data select

There is one set of input lines to the decoder array, labeled Y1 through Y6 and $\overline{Y1}$ through $\overline{Y6}$.

The data select chooses whether the program counter lines or the Y-register lines will be connected to the array input lines. It does this simply by using timing signals.

The program counter lines are connected to the array inputs through a transistor clocked on ϕ_1 , and the Y-register lines are connected through a transistor clocked on ϕ_2 . Thus, on ϕ_1 , program counter values are passed through, and on ϕ_2 , Y-register values are passed through.

ROM decoding

Timing signal ϕ_4 's first pulse comes at the same time ϕ_1 is powered. Transistors \$287 and \$288 connect each of the \$281 lines with ϕ_4 allowing them to be charged negative, but as they are enhancement mode transistors, when ϕ_4 is grounded, these transistors will close, preventing the \$281 lines from discharging through \$287 and \$288 to ϕ_4 .

Lines Y_6 and $\overline{Y_6}$ are connected to transistors \$289 and \$290 respectively, each of which alternately connect half of the \$281 'row' lines with ϕ_4 . Thus, when ϕ_4 is grounded (halfway through ϕ_1), half of all \$280 lines will be grounded and discharged.

Transistors \$280' connect adjacent \$280 'row' lines, and have their gates connected to true or inverted Y lines. Because of the aforementioned Y_6 and $\overline{Y_6}$ behavior, we know that one of the two lines that each of these transistors connects will be negatively charged, and one will be connected to the ground charge source. If even one of these transistors is connected, the charged line will be drained to V_{SS} .

These are arranged such that each line \$281 has a unique set of five such \$280' gates, connected to different Y and $\overline{Y}$ gates.⁷ Thus, for each program counter value, only one \$281 line will remain charged, with all others grounded.

On ϕ_1 , each \$281 line charges corresponding capacitor \$284, and when ϕ_1 is discharged, this capacitor will hold gate \$282 open or closed. On the line which is 'selected' (left charged) by the program counter value, gate \$282 will allow ϕ_2 to power its output line \$241. On all other lines, the output line will remain unpowered.

Y-register

As the Y-register is only four bits, compared to the program counter's six bits, Y_5 is shorted to ground and Y_5 is shorted to DD on clock signal ϕ_2 . This reduces the number of possibly powered output lines by a factor of 4, bringing it down to 16, the number of possible Y-register values.

Like with the ROM decoding, ϕ_4 charges all lines to V_{DD} . The one line with all \$280' transistors closed will remain charged when ϕ_4 is connected to ground, while all others will be discharged, in the same manner in which this occurs for ROM decoding.

On ϕ_2 , each of the possible Y-register output lines (one out of every four \$281 lines) are connected to the gate of transistor \$294. Capacitor \$295' ensures that on the charged line, sufficient charge is collected and held in the gate to hold it open for a machine cycle.

On ϕ_2 , transistor \$304 is opened, allowing ϕ_2 to power transistor \$302, connecting each output line \$26 to V_{SS} , ensuring that all output lines start discharged.

⁷The patent diagram depicts an invalid arrangement, with duplicate \$280' gates.

If transistor \$294 is activated, $\overline{\phi 2}$ is connected with the output line \$26, charging it to V_{DD} . Additionally, when line \$28 is charged, transistor \$303 is opened, ensuring that transistor \$302's gate is connected to the ground side of $\phi 2$ and discharged, and does not drain the output line until next machine cycle. On unselected lines, line \$26 is not connected to a negative voltage source, so transistor \$302 remains active, keeping it connected to ground. This is referred to as a zero-keeping circuit, as it keeps the unselected lines at V_{SS} , while allowing the selected line to be charged.

The Y-register decoding cannot affect the ROM decoding, and vice versa, as the decoded ROM outputs are gated on $\phi 1$, while the decoded Y-register outputs are gated on $\phi 2$.

Conclusion

4.1 What's left to do?

I could write a whole chapter on the ways that I wanted to expand this thesis, but ran out of time for. In fact, I will.

- Making a large table of every control and data line in the TMS 1000, where they originated from, what components they affect, and what timing relationships they possess would be a useful and interesting exercise.
- I feel that this thesis is incomplete by one chapter: we have examined the construction of the TMS 1000 from the electrons up, but haven't looked at the circuits that the TMS 1000 was embedded in, and how it was actually used.
- The clock circuit is still a bit of a black box. I would like to fully reverse-engineer it.
- Another chapter (or appendix) I was planning on writing was one to bring the technology more 'up to date'. For instance, by discussing how CMOS logic differs from PMOS/NMOS logic, and the advantages (logic gates being able to output V_{SS} and V_{DD} perfectly, less energy consumption and voltages required)
- I wanted to make the TMS 1000 circuit diagrams interactive. A group at visual6502.org creates interactive transistor-level simulations of old microprocessors; I would love to do something similar, or more basically, implement them in Logisim or an equivalent.
- I also wanted to recreate the TMS 1000 on an FPGA. One such project exists (github.com/mikeakohn/tms1000_fpga/), but it is done with instruction-level emulation and does not emulate the microcode .

4.2 What I learned

In writing the TMS 1000 emulator, I learned Rust. I wanted my emulator to be able to run TMS 1000 code at comparable speeds to an actual TMS 1000. I additionally wanted to, as much as possible, use similar data structures. These factors motivated me to write the emulator in Rust, although I had no experience in that language hitherto, but had heard many good things.

When I first started this thesis, I knew relatively little about low-level computer architecture. It was a fascinating but also terrifying topic.

I set out just trying to write an emulator. But I kept finding that I needed to understand things at a lower level. When constructing my emulator, based primarily on information from the TMS 1000 Programmer's Manual, I kept running into unanswerable questions. Why does the program counter step through its possible values out of order? What happens if we have two overlapping PLA terms? My emulator wasn't working, and I could only guess at why.

I turned to the circuit diagrams, but found them unintelligible. Why were there NOT gates all over the place, inverting lines multiple times for no apparent reason? I had to go deeper. I had to finally really understand how a transistor works, and to do that I had to get a real understanding of how electricity works.

So, in a sense, my thesis is a prelude to a thesis: it's a document of the things I wish someone had told me. I am hoping that it will be of use for someone else, so that I can be the proverbial giant on whose shoulders people stand, and see far and distant vistas.

And I personally feel like I've conquered an old fear. Computer hardware is no longer a scary, mysterious region. Now it's just where my old friend the TMS 1000 microcontroller lives.

There's some notion of control, it seems to me, that you can derive in a world full of confusion if you at least understand how things get put together. Even if you can't understand every little part, how infernal machines get put together.

Tom West

Appendix A

Other Transistors

A.1 Bipolar Junction Transistors

BJTs have three terminals: the collector, the base, and the emitter. The collector is connected to a section of highly-doped p-region, the base is connected to the center slice of barely-doped n-region, and the emitter is connected to the final section of medium-doped p-region. Its operation relies on two circuits: one between the base and the emitter, and one between the collector and the emitter.

A negative voltage is applied to the collector, and a positive voltage is applied to the emitter in the collector-emitter circuit. However, there is no way for current to flow through the two p-n junctions.

By applying a voltage to the base-emitter circuit, such that negative voltage is applied to the base and positive voltage to the emitter, the base-emitter p-n junction is forward biased, and a current flows. Because the base-emitter p-n junction is narrowed by the base-emitter voltage, electron holes from the emitter p-region can diffuse into the base n-region. Because the n-region is only lightly doped, there are relatively few electrons for these electron holes to recombine with. When these electron holes diffuse to the base-collector p-n junction, they are swept across by the pull of the negatively-charged collector terminal. Remember, even though the base-collector p-n junction is reverse-biased, minority carriers can still flow between them; and what we have created is a preponderance of minority carriers in the base n-region. Thus, by applying a voltage between the base and emitter junctions, current between the emitter and collector is allowed. By varying the voltage applied to the base, the base-emitter p-n junction is weakened a variable amount, and a corresponding amount of electron holes can diffuse through it; thus, the current can be fairly precisely controlled by the base voltage.

BJTs are often described as current amplifiers, as the current which flows between the collector and emitter is roughly equivalent to some constant times the amount of current flowing between the gate and the emitter. However, because a current must exist to weaken the base-emitter p-n junction, maintaining an adequate base-emitter current becomes a concern.

BJTs can be run ‘backwards’ by switching the collector and emitter voltages.

However, because the emitter is more heavily doped than the collector, the current gain will be lessened in this mode of operation [1].

A.2 Junction FETs

Junction FETs are called such because they rely on a direct p-n junction to operate. They are constructed by embedding an area of n-type silicon in an area of p-type silicon, such that a channel of p-type silicon still goes around the n-type silicon. The gate terminal is connected to the n-type silicon, while the drain and source terminals are connected to the p-type silicon on either side of the channel.

When no voltage is applied to the gate terminal, current may easily pass between the drain and the source, through the continuous region of p-type silicon.

When a voltage is applied to the gate terminal which is positive compared to the source, the p-n junctions on all sides of the embedded n-type silicon becomes reverse-biased, meaning that the depletion regions expand. With enough of a voltage differential, the depletion region becomes large enough to partially or fully block off the source-drain channel, preventing current from passing between them [13, 134-5].

References

- [1] William Beaty. How do transistors work? <http://amasci.com/amateur/transis.html>, 1995.
- [2] binarysequence. TMS1100 in Merlin, 2018. Available from: <https://commons.wikimedia.org/wiki/File:Tms1100-merlin.png>.
- [3] Gary Boone. Oral-history: Gary boone. https://ethw.org/Oral-History:Gary_Boone, 1996. Interview conducted by David Morton on June 22, 1996. IEEE History Center.
- [4] Gary W. Boone and Michael J. Cochran. Variable function programmed calculator. <https://patents.google.com/patent/US4074351A/>, 1977. US patent 4,074,351.
- [5] William N. Carr and Jack P. Mize. *MOS/LSI Design and Application*. Texas Instruments Electronics Series. McGraw-Hill Book Company, 1972. https://archive.org/details/bitsavers_tiTexasInsSeriesCarrMOSLSIDesignandApplication1972_23323581/.
- [6] Edward R. Caudel and Joseph H. Raymond Jr. Electronic calculator or digital processor chip having test mode of operation. <https://patents.google.com/patent/US4024386A/>, 1974. US patent 4,024,386.
- [7] Robert H. Crawford. *MOSFET in Circuit Design*. Texas Instruments Electronics Series. McGraw-Hill Book Company, 1967. https://archive.org/details/bitsavers_tiTexasInsSeriesCrawfordMOSFETInCircuitDesign1967_10607502/.
- [8] Dataquest, Inc. *Microcontroller Historical Shipment Data*. Dataquest, Inc., 1989. https://archive.org/details/bitsavers_dataquestMtAnalysis19751989_2759382.
- [9] Elcap. Electrolytic Capacitors, 2014. Available from: https://commons.wikimedia.org/wiki/File:Electrolytic_capacitors-P1090328.JPG.
- [10] Electronic Design. First true computer on a chip finally reaches marketplace. *Electronic Design*, 23(13):19, 1975. https://archive.org/details/bitsavers_ElectronicignV23N1319750621_98066899/.

- [11] Electronic Design News. Fourth annual microprocessor directory, November 1977. https://archive.org/details/bitsavers_ednEDN4thadirectoryNov201977_48996250/.
- [12] Robert G. Hibberd. *Integrated Circuits: A Basic Course for Engineers and Technicians*. Texas Instruments Electronics Series. McGraw-Hill Book Company, 1969. <https://archive.org/details/integratedcircui0000hibb/>.
- [13] Paul Horowitz and Winfield Hill. *The Art of Electronics*. Cambridge University Press, 3rd edition, 2015.
- [14] Sung-Mo Kang and Yusuf Leblebici. *CMOS Digital Integrated Circuits: Analysis and Design*. WCB McGraw-Hill, 3rd edition, 2003. <https://rajeev2007.github.io/VLSI/0072460539cmos.pdf>.
- [15] Guy Kewney. Today translators, tomorrow the world. *Personal Computer World*, page 40, December 1979. https://archive.org/details/bitsavers_ElectronicignV23N1319750621_98066899/.
- [16] Tracy Kidder. *The Soul of a New Machine*. Avon Books, 1981.
- [17] Robert H. Kushman. Learn the peculiarities of low-end microcomputer architectures. *EDN*, 24(7):99–108, April 1975. https://archive.org/details/edn-1979_04_05/.
- [18] Leisure Time Electronics. Image from Leisure Time Electronics*, June 1980. <https://archive.org/details/198006LeisureTimeElectronics/page/n11/>. Image from page 512 of the June 1980 issue. No attribution available.
- [19] Hung Chang Lin. *Integrated Electronics*. Holden-Day Series in Information Systems. Holden-Day, 1967. <https://archive.org/details/integratedelectr001inh/>.
- [20] Herbert Maisel. *Introduction to Electronic Digital Computers*. McGraw-Hill Book Company, 1969. <https://archive.org/details/introductiontoel0000mais/page/n7/mode/2up>.
- [21] John Maushammer. Diode Closeup, 2006. Available from: <https://commons.wikimedia.org/wiki/File:Diode-closeup.jpg>.
- [22] Carole Spearin McCauley. *Computers and Creativity*. Praeger Publishers, 1974. <https://archive.org/details/computerscreativ00mcca/>.
- [23] James Redin. Calculator wars, 2002. Available from: <http://www.datamath.org/Story/Calculator-War.htm>.
- [24] Retired electrician. Mos transistor, 2023. Available from: https://commons.wikimedia.org/wiki/File:IRFIBC30_MOS_transistor_02.jpg.

- [25] Sean Riddle. Sean Riddle's Home Page. Available from: <https://seanriddle.com>.
- [26] Ken Shirriff. The Texas Instruments TMX 1795: the (almost) first, forgotten microprocessor. <https://www.righto.com/2015/05/the-texas-instruments-tmx-1795-first.html>, 2015.
- [27] Texas Instruments. *1967 Annual Report*. Texas Instruments, 1967. https://archive.org/details/Texa1295_1967.
- [28] Texas Instruments. *TMS 1000 Series MOS/LSI One-Chip Microcomputers*, 1975. https://archive.org/details/bitsavers_tiTMS1000TneChipMicrocomputers1975_1257483/.
- [29] Texas Instruments. *TMS 1000 Series MOS/LSI One-Chip Microcomputers Programmer's Reference Manual*, 1975. https://archive.org/details/bitsavers_tiTMS1000T_10154027/.
- [30] Texas Instruments. *Microsystems Designers Handbook*, 2nd edition, 1981. <https://archive.org/details/texas-instruments-microsystems-designers-handbook-2nd-edition/>.
- [31] VectorVoyager. Silicon Doping, 2024. Available from: https://commons.wikimedia.org/wiki/File:Silicon_doping_-_Type_P_and_N.svg.